Seeing what's missing

Graphical and computational evaluation of imputation methodology

[TODO: create cover, add boilerplate stuff]

Acknowledgements

[TODO: write; may be added after uploading dissertation to MyPhD]

Table of contents

[TODO: fix which sections are included]

Samenvatting

[TODO: write; may be added after uploading dissertation to MyPhD]

Abbreviations and symbols

n	number of units in a sample or rows in a dataset, indexed $i = 1, \ldots, n$
p	number of variables in a sample or columns in a dataset, indexed $j = 1, \ldots, p$
Y	partially observed sample data (sized $n \times p$), also called the (hypothetically) **complete data**
R	response indicator matrix (sized $n \times p$), which stores the locations of the observed ($R_{ij} = 1$) and missing ($R_{ij} = 0$) parts of Y
Y_{obs}	**observed data** (elements of Y_{ij} where $R_{ij} = 1$), also called the **incomplete data**
Y_{mis}	**missing data** (elements of Y_{ij} where $R_{ij} = 0$)
Q	quantity of scientific interest; the scientific estimand we would like to estimate if we had complete data
$P(Q\|...)$	**complete-data model**; the analysis model we would like to fit if we had complete data, also called the 'model of scientific interest'
$P(R\|...)$	**missing data model**; the model that relates Y to R, which is interpreted as the 'probability to be missing'; also called the 'response model'
X	fully observed sample data for covariate(s) of Y, used in missing data models
ψ	parameter(s) of the missing data model that represent any cause of the missingness unrelated to X and Y
MCAR	'Missing Completely At Random'; missing data mechanism with missing data model $P(R\|\psi)$, also called 'not data dependent'
MAR	'Missing At Random'; missing data mechanism with missing data model $P(R\|Y_{\text{obs}}, X, \psi)$, also called 'seen data dependent'
MNAR	'Missing Not At Random'; missing data mechanism with missing data model $P(R\|Y_{\text{obs}}, Y_{\text{mis}}, X, \psi)$, also called 'unseen data dependent'
LWD	list-wise deletion
CCA	complete case analysis
MI	multiple imputation
JM	joint modeling; imputation technique
FCS	fully conditional specification; imputation technique
SMC-FCS	substantive model compatible fully conditional specification; imputation technique
MICE	multivariate imputation by chained equations algorithm
`mice`	multivariate imputation by chained equations software
m	number of imputations, indexed $\ell = 1, \ldots, m$
$P(Y, X, R)$	joint distribution of the incomplete data, fully observed covariate(s), and the missingness

$P(Y_{\text{mis}}\|...)$	joint (imputation) model, where $P(Y_{\text{mis}}\|Y_{\text{obs}}, R)$ denotes the posterior distribution of the missing data conditional on the observed data and the missingness
Y_j	univariate subset of Y (sized $n \times 1$); an incomplete variable
Y_{-j}	all data in Y except for column j (sized $n \times (p-1)$)
$P(Y_{\text{mis},j}\|...)$	**imputation model** for column j, where $P(Y_{\text{mis},j}\|Y_{\text{obs},j}, Y_{-j}, R)$ would indicate all other variables are imputation model predictors
$\dot{Y}_\ell$	**imputed data** in imputation number ℓ (of m total)
Y_ℓ	**completed data** in imputation number ℓ, consisting of $\{Y_{\text{obs}}, \dot{Y}_\ell\}$
$\hat{Q}_\ell$	estimate of Q calculated from completed data in imputation number ℓ
$\bar{Q}$	pooled estimate based on m imputations
T	total variance of $Q - \bar{Q}$
U	within-imputation variance due to sampling
B	between-imputation variance due to nonresponse
SE	standard error
CI	confidence interval
γ	fraction of information missing due to nonresponse, also called 'fraction of missing information' (FMI)
PMM	predictive mean matching; imputation method
V&V	verification and validation of (software) systems, where verification refers to the process of evaluating a system's output, and validation assesses its fit for purpose.

1 Introduction

TL;DR (or: the quality of the solution)

This dissertation is about algorithms and the quality of their output. I investigate how one specific type of data-generating algorithm can be evaluated for use in statistical inferences. The type of algorithm in question are imputation algorithms: algorithms with the purpose of generating plausible data values, based on a model of some observed information. This type of algorithm is popular for solving missing data problems.

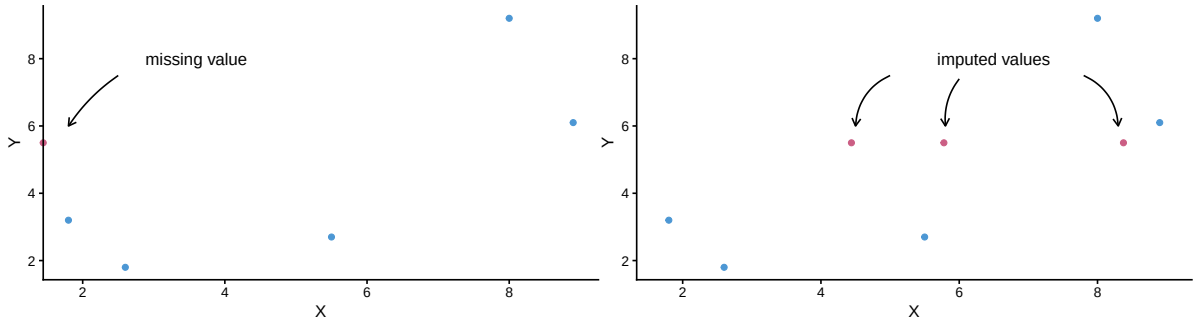
Missing data problems are timeless, but technological advances have made treating missingness easier than ever. A single line of code can seem to solve your missing data problem nowadays. But is that solution really what you expect? That is what my dissertation is about. I want to contribute methodology that facilitates drawing valid inferences from incomplete data.

Missing data are ubiquitous and cannot be ignored without risking bias. Missing data occur often in scientific disciplines with human subjects, such as the social sciences and biomedical research. A participant in a study might, for example, drop out of a longitudinal trial or refuse to answer a sensitive question, resulting in gaps in the research data. These gaps complicate data analysis and may lead to biased and invalid study results. Imputation algorithms are designed to 'impute' (i.e., fill in) missing values in incomplete datasets. Imputation is a popular and flexible procedure that separates the missing data problem from the analysis of scientific interest.

Imputation allows data analysts to first solve for the missingness, to then be able to analyze the completed data as if they were completely observed. If done several times in parallel, imputation can yield estimates that are both unbiased as well as confidence valid. The practice of drawing inference by replacing missing values with several synthetic substitutes is called 'multiple imputation' (MI). Figure 1 provides an example of multiple imputation, where a missing value is replaced several times with plausible substitutes. The variability between imputations will reflect the uncertainty due to non-response in the final estimate.

Although imputation is widely supported by software and many implementations of the method have convenient defaults, generating good imputations is not a trivial task. **How can we know when to trust the output of imputation algorithms?** There are three approaches. The ideal route is to study the formal statistical properties of the method via analytical derivations. Unfortunately, this is often impossible for imputation algorithms, many of which do not have formal proofs. Instead, the de facto standard is to estimate their empirical performance via simulation studies. These studies, however, are not standardized and are therefore hard to compare. Moreover, simulation study results might not always translate to applied use. For real-world applications, there is no systematic framework for evaluating the operational utility of imputation algorithms, such as their algorithmic convergence. Ergo, evaluating whether imputations should be trusted remains challenging.

Figure 1: Multiple imputation of a missing value



In this dissertation, I set out to investigate what is needed to develop, apply, and evaluate data-generating algorithms. I thereby focus on the imputation procedure 'MICE' (van Buuren & Groothuis-Oudshoorn, 1999) as implemented in the R package `mice` (van Buuren & Groothuis-Oudshoorn, 2011).

This dissertation in a nutshell:

- Q: Why do I study the evaluation of imputation methodology?
  - A: Because current evaluation practices (such as computational evaluation via simulation studies, and graphical evaluation via diagnostic plots) are fragmented and may be insufficient.
- Q: How will I contribute to the evaluation of imputation methodology?
  - A: By bridging inferential evaluation and data- or algorithm-level diagnostics for imputation workflows.

1.1 Background

Why this dissertation? (or: certainty with uncertainty)

My academic tagline has long been "statistician, interdisciplinarian, open scientist". And while there's no doubt I am an interdisciplinarian and an open scientist, I am no longer keen on calling myself a statistician necessarily. I am more inclined to say I am a methodologist rather than a statistician. As a methodologist, I am interested in how science is done, and how it can be done better. Especially in the social and biomedical sciences, research should not be purely quantitative. Statistics is just *one* way of doing science.

Don't get me wrong, statistics is still a favorite of mine. I love stats (I even have a mug that says so). Statistical inference allows us to distinguish between real-world effects and random noise (Rosenberg, 2018). It gives scientists a tool to get certainty about uncertainty.

Rooted in mathematical probability theory, which deals in absolute truths, statistical inference is a framework for inferring with uncertainty from noisy real-world data. Epistemologically speaking, statistical inference may therefore be classified either as deductive or inductive inference; there is no consensus in the literature about which pattern of scientific reasoning suits statistical inference (de Ruiter & Albert, 2017). Gelman & Shalizi (2013) argue that statistical inference is deductive in the context of hypothesis testing (i.e., when general laws from mathematical probability theory are used) and inductive in the context of parameter estimation (i.e., when sample statistics are extrapolated to the population). In other words, statistics is the science of uncertainty and extracting information from data (Hand, 2025). The latter can feel magical at times: the process of turning data into insights. But I am most a fan of the uncertainty aspect.

I am motivated to contribute to trust in science by making it easier to reflect uncertainty in data analysis efforts, inferences, and graphs. In this dissertation, I hope to achieve that for missing data methodology. We will start out with a theoretical background, and slowly zoom in: from statistical inference under ideal circumstances to inference with incomplete data, then from missing data methods in general to imputation methods specifically, and finally to multiple imputation with `mice`. The introduction section ends with a framework for evaluating imputation methodology, the scope of this dissertation, and a description of each of the chapters in this dissertation. But first, let's start with some terminology and notation.

Why are missing data problematic? (or: don't ignore the missingness)

Generally speaking, statistical inference works really well. Statistical analyses are a means to infer conclusions about a population after observing only a subset of this population: a sample. If the sample is representative, sample estimates will reflect the population characteristics plus some uncertainty surrounding it. The analysis model used for estimation will also be referred to as the 'model of scientific interest' or 'complete data model' (when the sample is incomplete) throughout this disseration. The analysis model yields sample estimates that ought to match the population of interest.

In technical terms, we want our estimation procedure to be an unbiased and confidence valid estimator for the quantity of scientific interest, Q, at the population level. If our estimate of Q matches the true population parameter, we have an unbiased estimate. If the variance surrounding the estimate correctly reflects the uncertainty in the estimation procedure, it could be considered confidence valid. Statisticians call uncertainty around estimates 'error'. There is, for example 'sampling error' due to observing only a subset of the population. This uncertainty is often expressed in terms of the standard error (SE) of an estimate. Standard errors can get translated into decision tools for inference such as p-values and confidence intervals (CIs),

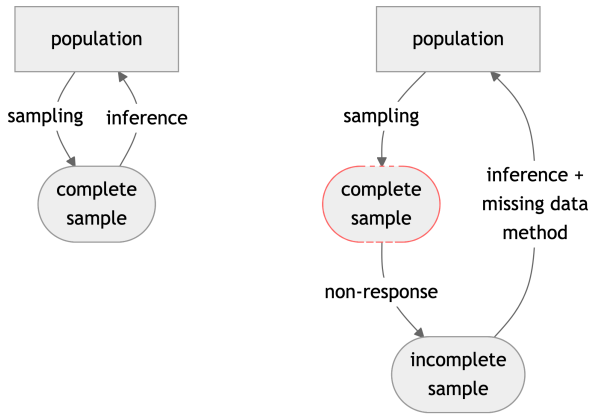
which help researchers distinguish between signal and noise. CIs, for example, should cover the the true population parameter at the frequency of the nominal confidence level (e.g., 95%). A statistical procedure with a coverage probability that matches the nominal confidence level is called 'confidence valid' (Neyman, 1934). The concept of confidence validity provides an operational link between the mathematics of probability and empirical practice. Confidence valid statistical methods give us certainty about the uncertainty in our inferences from a sample to the population. So, statistics is a means of estimating effects within a sample and inferring back to the population with some expression of uncertainty. To illustrate these concepts, I will use an empirical example.

When sociologists wanted to study trust in the government during the COVID-19 pandemic, it was not feasible to survey all Dutch citizens. Instead, they used a representative sample. Some thousands of people were asked to fill in a questionnaire about their trust in the government over the course of the pandemic. By repeating the questionnaire over time, the researchers wanted to gain insight into the development of (dis)trust among different subgroups in Dutch society. The estimates from the sample would then be used to inform policy, by inferring back to the population with some expression of uncertainty. This uncertainty, however, does not only stem from the sampling process.

Uncertainty in statistical inferences can stem from many sources, such as non-response. The 'Total Survey Error' framework (TSE, Deming, 1944), uncertainty due to non-response is called 'non-response error'. Non-response error occurs when participants do not (fully) respond to data collection efforts (Groves, 2002). We speak of 'unit non-response' for intended study participants who do not provide any data at all (for example because they could not be reached, or refused to participate). 'Item non-response' refers to patterns of missingness where some–but not all–data are missing for a participant (for example because they skipped a survey question). Of course, non-response can lead to additional uncertainty in our inferences, because we are missing some information. But even the way we treat non-response may introduce error. For unit non-response, the non-response treatment is typically weighting, which may introduce 'weighting error' Alsalti et al. (2026). Item non-response treatment can also produce error, such as 'imputation error' (Federal Committee on Statistical Methodology, 2001). Although imputation error is not typically part of the TSE framework, it is the focus of this dissertation. The aim of this dissertation is to study the way we handle incomplete observations in partially observed data and how that affects our inferences, and to provide methodology for minimizing non-response error and imputation error.

Of course, non-response error and imputation error can be circumvented by preventing missingness altogether. A truism attributed to Gertrude Mary Cox posits that the best way to treat missing data is by not having any. But anyone who analyses data will run into a missing data problem at some point (Allison, 2002). Missing data are a nuisance because it is impossible to do statistical inferences with incomplete data. When there is just one missing value in a dataset, statistical inference cannot be performed. With incomplete data, even simple statistics such as the mean of variables are mathematically undefined. To obtain results from your incomplete dataset, you will always employ a missing data strategy—whether deliberately or not.

Figure 2: Inference with incomplete data



Note. Schematic view of statistical inference in the presence of non-response. Left: no missing data in the sample from the population (i.e., ideal circumstances for inference). Right: incomplete sample due to non-response. If a complete sample cannot be obtained, a missing data method has to be employed for inferring to the population.

Whenever missing data cannot be prevented, the only way forward is to treat the missing data problem. Figure 2 shows that some kind of missing data method has to be employed to get a population estimate from an incomplete sample. Even without specifying a missing data method explicitly, there will be handling of the missing data. The default strategy in most statistical software packages is to ignore all cases that have missing values. This practice is called 'list-wise deletion' (LWD), also referred to as 'complete case analysis' (CCA). Ignoring missingness, however, lowers statistical power and may yield invalid inferences (e.g., biased estimates, spurious significant results, see e.g., van Buuren, 2018). Under many missing data conditions, list-wise deletion is ill-advised (for exceptions see e.g., van Buuren, 2018 § 2.7; Carpenter & Smuk, 2021, § 3.1). With substantive missingness, list-wise deletion often results in bias: "results from case deletion may be biased, because the complete cases can be unrepresentative of the full population" (Schafer & Graham, 2002, p. 155). LWD assumes that the observed part of the data is equivalent to the missing part of the data, which is not usually the case.

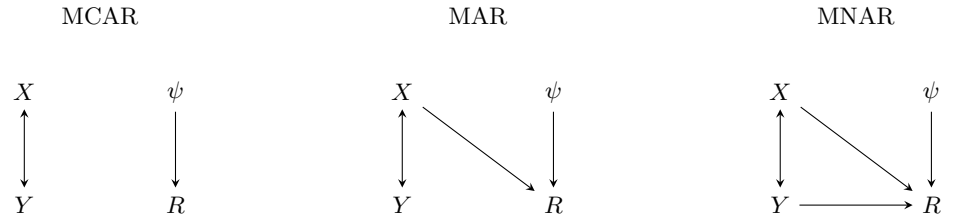
Why are the missing and observed parts not equivalent? (or: the missing data mechanisms)

List-wise deletion assumes that the observed part of the data is representative of the missing part of the data, which seldom holds in practice. In practice, the missing part of the data is often missing due to some non-trivial reason, other than pure chance (or 'bad luck'). If there is a non-chance cause for the missingness across observations, the observed part of the data is not representative of the missing part of the data. This relationship can be formulated in terms of a missing data model. Missing data models capture the causal relation between the

data (Y) and the associated missingness indicator (R). Missing data models thereby reflect the 'probability to be missing' for any given observation. Missing data models are used to describe and classify the causes of the missingness, which determines how the missingness should be handled to arrive at unbiased and confidence-valid results.

There are three types of missing data mechanisms described by missing data models. If there is no data-related cause cause of the missingness, the missingness is said to be 'completely at random' (MCAR), or 'not data dependent' (respective terminology by Rubin, 1976; and Hand, 2020). If there is an association between the missingness and the observed part of the data, the missingness is 'at random' conditional on the observed data (MAR) or 'seen data dependent'. If the cause of the missingness is missing too, the missingness is 'not at random' (MNAR), or 'unseen data dependent'. Figure 3 provides a graphical representation of these three mechanisms, where the missingness (R) in incomplete data Y either depends on Y itself (i.e., MNAR), on observed covariate(s) X (i.e., MAR), or none of these (i.e., MCAR). Such a graphical representation of missingness mechanisms was first published by Schafer & Graham (2002). Since then, many others have extended the use of causal diagrams for missing data methodology (e.g., Daniel et al., 2012; Ding & Li, 2018; Moreno-Betancur et al., 2018).

Figure 3: Missing data mechanisms



Note. Incomplete data Y and fully observed covariate X covary. Under missing completely at random missingness (MCAR), the missingness indicator R is only affected by ψ, causes of the missingness unrelated to X and Y. Under MAR, the missingness also depends on observed information (X), whereas MNAR entails missingness that is dependent on the missing data itself (Y). Figure adapted from Schafer & Graham (2002).

Modeling the non-response can aid the analysis of incomplete data. Together with complete-data models (i.e., the model of scientific interest), missing data models inform data analysts how the missingness might affect the scientific estimates. But from the observed data alone, it is impossible to definitively determine the missingness mechanism. We can only make assumptions about the missingness mechanisms that are most likely to hold in our incomplete data. We need external information (e.g., about the study design) to substantiate the assumption about the missingness mechanism. For example, if there was a glitch in the survey software, affecting all participant equally likely, a reasonable assumption would be MCAR. In human subjects research, however, pure MCAR missingness is unlikely. Only truly random missingness, independent of any individual characteristics, would amount to missing completely at random missingness mechanism.

In the empirical example about trust in the government during the COVID-19 pandemic,

MCAR was not a reasonable assumption. It was not random who had missing data on the outcome variable (the question in the survey about trust in the government). Non-response was associated with demographic traits of respondents with lower trust. People with lower trust in institutions were also less likely to respond to follow-up data collection waves of the questionnaire. This resulted in higher missingness rates across participants with lower trust in the government. If we would use list-wise deletion as missing data method, we would over-estimate trust in the government at the population level.

There were signs of non-MCAR missingness in the COVID survey data because participants with complete data differed from participants with non-response. Using only the observed data to infer to the population leads to biased estimates if the missingness is not MCAR. But if we can model the missingness probability based on known data, we have MAR missingness, which can yield valid inferences if this model if adequate. Then, conditional on this information (such as previous observations or demographic data), observed and missing values have the same probability to be missing. That is why we also speak of 'ignorable' missingness. Ignorable missing-data mechanisms meet two requirements: 1) conditional on the observed data, the probability that an entry is missing does not depend on the unobserved value itself (i.e., MCAR or MAR missingness), and 2) the parameters in the missing data model are distinct from the parameters of the substantive model (Little & Rubin, 2020). "The assumption of ignorability is essentially the belief on the part of the user that the available data are sufficient to correct for the effects of the missing data" (van Buuren, 2018, p. 2.2.6). If treated correctly, ignorable missingness will yield valid inferences.

This is in contrast to the MNAR missingness mechanism, where the missingness depends the unobserved values themselves. MNAR missingness is often called 'non-ignorable', which roughly means that given the observed data, you cannot treat the missing-data mechanism as irrelevant for the subsequent inferences. If the missingness is MNAR, the validity of the inferences is at risk. For example, when people who distrust the government are less likely to respond to the trust item, and we have no way of correcting for this. This kind of missingness cannot be resolved from the observed data alone and requires external information to amend. In practice, it is impossible to know from the observed part of the data alone whether we are dealing with MNAR, so you always have to make an assumption about the mechanism underlying the missingness in your data. The missing data mechanisms and our assumptions about them in practice determine how the missingness should be treated to achieve unbiased and confidence valid statistical inferences.

How to treat the missing data problem? (or: ignore, model, fill in – the missing data methods)

This section describes three popular routes for treating missing data. I will not pretend to provide an encompassing overview on treating missing data problems. Others have written much more extensive reference works on the matter, see for example Rubin (1996), Schafer (1997), Allison (2002), Schafer & Graham (2002), Graham (2012), Molenberghs et al. (2014),

van Buuren (2018), and R. J. A. Little & Rubin (2019). The three missing data treatments I will briefly describe are deletion-based methods, likelihood-based methods, and imputation, before further expanding on imputation methodology.

The first missing data treatments are deletion-based methods, such as list-wise deletion. With list-wise deletion, we only analyze rows in the data without missing values (i.e., complete cases). Deletion-based methods are easy to apply, and sometimes even applied by default in statistical analysis software. If the missingness level is low, deletion-based methods do little harm, and under some conditions they might even be the best missing data method to apply (see e.g., van Buuren, 2018 § 2.7; Carpenter & Smuk, 2021, § 3.1). Deletion-based methods, however, do not accommodate differences between the observed and missing parts of the data, making them mostly applicable to MCAR. Under most kinds of missingness, deletion-based methods will therefore risk biasing the statistical inferences. Moreover, these methods discard incomplete observations, resulting in lower power and potential resource waste compared to non-deletion-based methods. In short, deletion-based methods do not treat the missing data as much as ignore them, and one should be careful of bias when applying them.

Secondly, there are methods that incorporate the missingness into the model of scientific interest, such as likelihood-based methods. These missing data methods jointly specify the complete-data model and missing data model per analysis. Thereby, these modeling methods do accommodate MAR, but only for a single, specified analysis model. For example, 'expectation-maximization' (EM) treats the missing values as latent variables in the complete data model. Another option that incorporates the missingness in the analysis model are via Bayesian methods. Both of these techniques are tied to a specific analysis model, which may be a problem when working with applied researchers, and which might not be suitable for contemporary practitioners who run several models in a data science workflow.

Thirdly, there are imputation methods. Imputation methods does not have the drawbacks of being analysis-model specific. Imputation methods first solve for the missingness, to then analyze the completed data 'as if' they were fully observed. This allows the use of familiar complete-data methods instead of bespoke incomplete-data models. Imputation splits the missing data problem from the analysis of scientific interest. Imputation is useful when different people are responsible for different parts of a project (e.g., a missing data methodologist vs. domain expert). And when many different analyses are planned on the same data, it is possible to run several analyses without having to incorporate the missingness into the scientific model each time. This makes imputation better suited for contemporary data science workflows than missing data methods that incorporate the missingness in the substantive model. Compared to missing data methods that incorporate the missing data like Bayesian methods or likelihood-based approaches, imputation is more flexible and often less complex for non-experts. Compared to list-wise deletion, imputation methods offer the ability to accommodate MAR mechanisms, and they reduce research waste by retaining incomplete cases.

How to generate imputations? (or: univariate, joint, chained – the imputation flavors)

With imputation, we fill in the missing part of the data based on the observed part. We model the distribution of this missing part based on what is observed, using the observed part of the data to fix the missing part of the data. Which types of model or models do we use?

The simplest imputation model is no model at all. We can, for example, impute the missing values by filling in a random other person's observed data, called ' hot deck imputation'. This method is quick and easy. The problem of this method is that it does not accommodate MAR mechanisms, and, moreover, disrupts the multivariate distribution of the data, biasing any multivariate statistics downwards. Another simple technique is to impute a univariate statistic such as the mean. Unfortunately, imputing the mean will bias almost all scientific estimates. In general, all univariate imputation methods fail to retain multivariate associations in the data, and cannot account for MAR mechanisms. So, instead of imputing variables univariately, imputation should retain the multivariate structure of the data.

Capturing the multivariate structure of incomplete data is historically done in two steps: a modeling step and an imputation step. In the modeling step, a multivariate distribution of the full dataset would be defined, after which imputed values could be drawn from the joint model. In a joint model, one specifies $P(Y, X, R)$: the distribution of the incomplete data, complete covariates, and missingness indicator. Joint modeling imputation was originally developed for known distributions, such as multivariate normal data. With increasing data complexity and dataset size, specifying a joint model might not be feasible. The problem with joint modeling is that there might not exist a multivariate probability distribution, for example, with factor variables involved. Joint modeling is a direct but inflexible way to arrive at a joint distribution. Good thing we do not need this joint model explicitly.

Missing observations can also be filled in using several separate imputation models, instead of one joint distribution. In the 'fully conditional specification' framework (FCS, van Buuren et al., 2006), we do not specify an overall joint model, but conditional models per incomplete variable. Because of the variable-by-variable approach, this technique is also called 'sequential regression multivariate imputation' (SRMI, Raghunathan et al., 2001) or 'multivariate imputation by chained equations' (MICE, van Buuren & Groothuis-Oudshoorn, 1999). FCS does not assume any specific joint distribution, and is substantive model agnostic. While Rubin (1987) "distinguished three tasks for creating imputations under an explicit model: the *modeling* task the *imputation* task and the *estimation* task" (van Buuren, 2007, p. 221), FSC does not require these explicit tasks. Instead, FCS only requires an *imputation model* to describe how synthetic values may be generated, without explicit specification of the joint model. This makes FCS flexible: just specify an imputation model per incomplete variable.

With FCS, we define an imputation model per incomplete variable. FCS is a flexible and complete data model agnostic way to arrive at a joint distribution via several imputation models. These imputation models consist of the functional form of the model and imputation model predictors. The functional form of the model determines the distribution of the imputed

values. For example, Likert scale items (e.g., trust in the government) can be imputed using a semi parametric imputation method taht retains the range and distribution of the observed data (e.g., 'predictive mean matching', or PMM). The second ingredient in an imputation model are the imputation model predictors. Imputation model predictors are other variables in the data that we use to amend the missing data problem. For example, we can include variables with a high linear association with this incomplete variable, or variables that are related to the missingness indicator of this incomplete variable (such as demographic data). The relations implied by the imputation models can correct for MAR missingness mechanisms, to yield an unbiased estimate of the quantity of scientific interest.

In short, we have learned that imputation has the ability to produce unbiased estimates. The next question to consider is whether these estimates are properly uncertain too (i.e., confidence-valid). Well, that depends. Imputation does not necessarily yield confidence valid inferences. Only imputation processes that correctly propagate uncertainty do. All techniques that rely on a single replacement value risk under-representing uncertainty in the inferences, because the uncertainty due to the non-response is not taken into account. That's where multiple imputation (MI) comes in.

Why multiply impute? (or: several 'wrongs' *can* make a 'right')

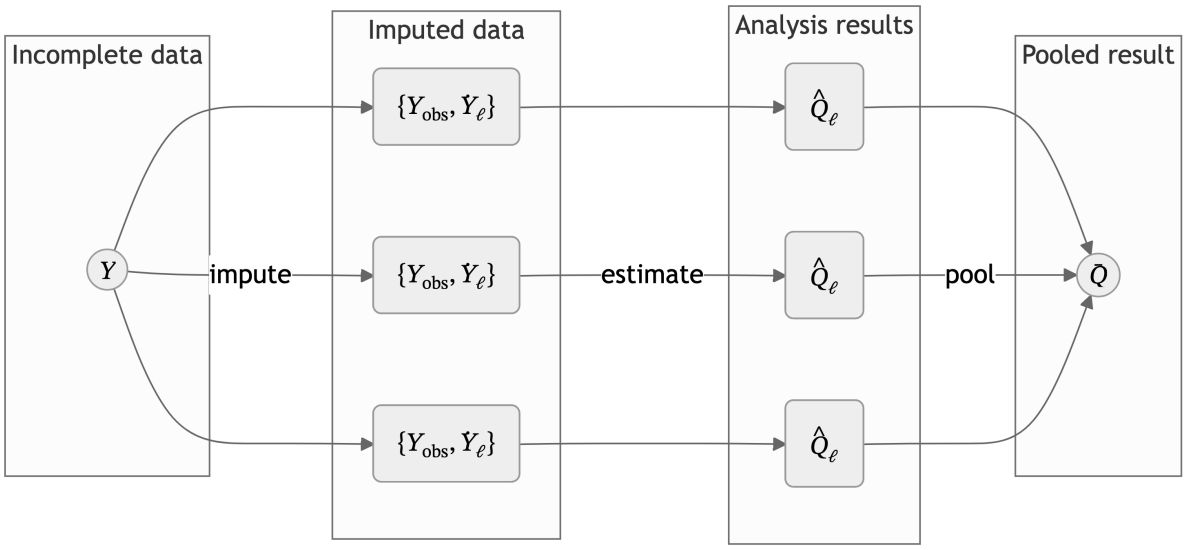
The uncertainty due to non-response can be captured using the multiple imputation framework. MI is the practice of generating several complet*ed* datasets by replacing each missing value with a synthetic substitute multiple times. The completed datasets can be analyzed as if the data were complete to begin with. On each completed dataset, we obtain estimates of the analysis of scientific interest. These estimates per imputation are then combined to obtain one pooled estimate. See Figure 4 for a graphical presentation of these steps.

In Figure 4, the incomplete data Y is partially observed, consisting of observed and missing data, $Y = \{Y_{\text{obs}}, Y_{\text{mis}}\}$. The missing data Y_{mis} in Y is imputed m times to obtain m completed datasets, $\{Y_{\text{obs}}, \dot{Y}_\ell\}$, where $\ell = 1, 2, \ldots, m$. On each imputation, the quantity of scientific interest Q is estimated, yielding m analysis results $\hat{Q}_\ell$. These results are pooled across imputations to obtain a single pooled result $\bar{Q}$.

Conceptualized by Rubin (1976), MI posits that repeated stochastic draws per per missing datapoint create a distribution of plausible values: the posterior distribution of the missing data conditional on the observed data and the missingness, $P(Y_{\text{mis}}|Y_{\text{obs}}, R)$. This posterior distribution reflects the uncertainty due to non-response. By drawing imputations from the posterior distribution of the missing data, we obtain several completed versions of the data that *might have* been.

From the probability distribution for each missing value, we draw multiple values as imputations. If we would use infinitely many values as imputations, we would obtain the entire probability distribution. In practice, however, we use a limited number of imputations. With that, we *approximate* the probability distributions of missing values. The more imputations are

Figure 4: Multiple imputation scheme ($m = 3$)



Note. Y = incomplete data, ℓ = imputation number, $\{Y_{\text{obs}}, \dot{Y}_\ell\}$ = imputed data per imputation, $\hat{Q}_\ell$ = estimated quantity of scientific interest per imputation, $\bar{Q}$ = pooled estimate.

used, the better we can represent the uncertainty in the data due to missingness. This is expressed in the 'simulation error'. The variance of the pooled estimate incorporates not just the sampling variance (within each imputation) but also added variance due to non-response (between imputation variance and simulation error). The variability between imputations reflects uncertainty due to the missingness.

MI is a convenient method for obtaining confidence-valid statistical inferences from incomplete data. There are many implementations of MI available in statistical software packages. In this dissertation, we will focus on the `mice` package in R. By default, `mice` runs variable-by-variable (FCS) multiple ($m = 5$) imputation. `mice` enables researchers with missing data problems to correct for differences due to missingness and maintain the appropriate level of uncertainty. Under the right conditions, this yields statistical inferences that are both unbiased and confidence valid. But how can we know when this is the case? That's the topic of this dissertation.

1.2 Research design

What is the research problem? (or: why extend the status quo)

In this dissertation, I investigate how imputation methodology can be evaluated. The status quo for evaluating imputation methodology may not always be sufficient. Please allow me to

illustrate this by presenting a bit of a caricature:

- Q: How do methodologists know whether an imputation approach 'works'?
  - A: They obtain estimates of its performance via simulation studies (and–if available–formal properties via analytical derivations).
- Q: How do imputers know whether the imputation workflow was successful on their particular dataset?
  - A: Our current answer is: well, they do not know for sure.

This highlights a fundamental issue that underlies all missing data methodology. We cannot assess our missing data treatments without the one thing we do not have: the missing data itself. Since missing data are–by definition–missing, there is no way to know for sure that our imputation workflow yielded appropriate imputations. Only when the true, but unobserved data is somehow recovered, we could check the effectiveness of the missing data treatment. In practice, with empirical incomplete data, this is rarely achieved. In simulation studies, however, we can create our own 'true' data and introduce artificial missingness. This allows us to experimentally test how well a missing data method handles the missingness. Unfortunately, this too comes with downsides. That is why I study which tools are required for evaluating the quality of imputation processes in applied use as well as in simulations.

To understand the status quo, let us consider a typical life cycle for imputation methodology. First, an imputation method is theorized, and–if possible–its analytical properties are derived. Then, the method is implemented in code as a 'proof of concept'. With the method implemented in a script or software, we can run a simulation study. Simulation studies provide experimental evidence of the method's properties under a certain set of conditions. Ideally, the method is then put to scrutiny by peers in the scientific community, who may run additional simulations to replicate or challenge its reported performance. Finally, the method is applied in practice. End users run the imputation procedure on real-world data and, implicitly or explicitly, evaluate its appropriateness in that setting. In this life cycle of imputation methodology, we can differentiate three types of evaluation:

1. Analytical evaluation—using analytical derivations to evaluate the formal statistical properties of the method.
2. Experimental evaluation—using simulation studies to evaluate the empirical performance (estimates of the method's attributes).
3. Experiential evaluation—using real-world data to evaluate the operational utility (the method's usefulness in practice).

The first and preferred option is analytic evaluation. If we can derive closed-form results for bias, variance, and confidence validity of an estimator, we obtain deductive truths: under stated assumptions, the method has proven properties. However, this route is often impossible or unfeasible, for example when no known joint model exists for a fully conditional specification (FCS) method. Without analytical proof, we cannot definitively conclude that an imputation

method is an unbiased and confidence valid estimator of a population quantity. In such cases, we fall back on experimental evaluation.

Simulation studies are the default mode of evaluation for imputation methodology. In a simulation study, we can assess imputation workflows under varying missing data conditions. Because we introduce the missingness in the simulation design ourselves, there is a 'comparative truth' (e.g., population parameters, or the generated complete data). We can then evaluate the results of our imputation process against the complete data to assess imputation accuracy. But better yet, if we have a specific analysis model and estimand in mind, we can compute performance metrics such as bias and confidence interval coverage. These performance measures provide evidence about whether the estimation process (the combination of imputation and inference) is approximately unbiased and confidence valid under the simulated conditions. Note that this requires a specific analysis model: "A missing-value treatment cannot be properly evaluated apart from the model estimation or testing procedure in which it is embedded" (Schafer & Graham, 2002, p. 149). And for FCS imputation specifically, the use of imputation models "creates new responsibilities for substantiating its correctness for a given statistical analysis" (van Buuren, 2007, p. 222). The quality of an imputation model is thus best assessed in pair with the complete data model. In practice, however, simulation studies cannot cover all potential use cases. Not all possible complete data models can be known or available to the simulator who designs the evaluation proocedure. This makes simulation studies no guarantee for practical performance. An additional problem is that the design and documentation of simulation studies for imputation methodology are not standardized, so it's difficult to compare different methods across studies. In short, simulation studies may be the standard, but they are not standardized or sufficient for evaluating imputation methodology.

The third kind of evaluation is experiential. Experiential evaluation focuses on operational utility in applied use, on a case-by-case basis with real data. Whereas we could construct the 'truth' in simulation studies, in practice we cannot. We therefore have to rely on assumptoins and theoretical justifications, combined with indirect evidence about imputation model performance. Some assumptions are impossible to definitively determine (e.g., MNAR mechanisms), while some we can partially evaluate by looking at the data. We can, for example, inspect the imputation algorithm output for signs of non-convergence, or compare the distributions of the imputed data against the observed. Diagnosing algorithmic non-convergence, however, is a challenging task (see, e.g., Gelman & Shalizi, 2013; van Buuren, 2018). And there are few diagnostic tools for assessing imputation model fit, especially for large datasets (Abayomi et al., 2008; Bondarenko & Raghunathan, 2016; He et al., 2010). Therefore, it remains difficult to aquire proof of imputation success in practice.

Even if we known from simulation studies that an imputation method yields unbiased and confidence valid inferences, this only holds under certain conditions. And these conditions cannot be definitively met in practice. For FCS imputation specifically, we would need correct assumptions about the missingness mechanism, 'congeniality' between imputation and analysis models, and adequate algorithmic behavior. When these conditions fail (e.g., under MNAR

missingness, incomplete imputation models, or non-convergence in the algorithm), our inferences may not be unbiased and/or confidence valid. However, there is not a systematic way to flag potential risks. Users of imputation methods often do not know whether they have correctly applied the procedure.

In this dissertation, I focus on experimental and experiential evaluation of imputation methodology. The studies I present will follow this distinction into the two phases in the life cycle of FCS imputation methods: first a simulation study, and then applied use. For both phases, I will investigate which existing evaluation tools are already avialable, and whether all necessary tools are present yet in the imputer's toolbox. The central question then becomes: what is needed for the experimental and experiential evaluation of imputation methodology? To address this question, I will borrow some terminology from the computer science discipline. Specifically, I will use the 'system life cycle' framework to provide a unified perspective on how imputations may be evaluated.

How can we conceptualize evaluation? (or: imputation as a sociotechical system)

In this section we will learn how imputation methodology can be evaluated as a 'system'. The system life cycle framework offers terminology and tools for systems engineering: to build, develop and evaluate systems. From this perspective, imputation workflows are 'sociotechnical systems' specifically. A sociotechnical system is a collection of interrelated components (software, hardware, and people) that delivers services to users (Sommerville, 2015). In contrast to technical computer-based systems, sociotechnical systems contain both technical components as well as human operators. Imputation workflows, therefore, are sociotechical systems. They contain not just software (e.g., `mice`), but also the imputer who runs the imputations.

The inclusion of a 'human in the loop', has three noteworthy consequences. Firstly, sociotechnical systems have non-deterministic outputs, resulting from human operators who may differ in the choices they make. Secondly, they have emergent properties of the system as a whole, which only becomes apparent when integrating the separate components. And thirdly, the success criteria of sociotechnical systems are subjective, depending on the context of evaluation: "it is practically impossible to have a complete understanding, in advance, of their behavior" (Sommerville, 2015, p. 558). So, the success of sociotechnical systems as a whole relies on the joint functioning of its components, plus the context of its application. We therefore cannot judge a sociotechnical system by its parts, or in isolation. Evaluating sociotechnical systems thus requires assessment of the entire system in use.

What a full system evaluation looks like in practice, depends on the system properties. Still, irrespective of the system specifics, there are certain standards for evaluating systems. The 'International Organization for Standardization' (ISO) and 'International Electrotechnical Commission' (IEC) collectively publish 'Information technology' vocabularies, which include definitions of evaluation practices (ISO & IEC, 2015). A set of two of these practices are relevant here: verification and validation (V&V). Roughly stated, verification asks 'was the system built

well?', whereas validation asks 'was the right system built?'. These two terms are further detailed by the 'Institute of Electrical and Electronics Engineers' (IEEE), which routinely publishes versions of the 'Standard for System, Software, and Hardware Verification and Validation' (IEEE Standard for System, Software, and Hardware Verification and Validation, 2025). We will explore and operationalize the verification and validation standards for imputation methodology.

System evaluation through verification is about whether the system conforms to its specifications. The technical definition of verification is "confirmation, through the provision of objective evidence, that specified requirements have been fulfilled" (Systems and Software Engineering — Vocabulary, 2017, def. 3.4538). In other words, does the system produce the expected output? We can verify a system's output if we have a clearly defined input–output relation. For a given input, we then already know what the correct output should be, and we can review whether the system reproduces that output. A typical form of verification are unit tests in software. Unit tests check if individual function in software packages return the correct output when a certain known input is supplied. Verification is typically carried out before or independent of real-world use. Note, therefore, that "a system could be verified to meet the stated requirements, yet be unsuitable for operation by the actual users" (Systems and Software Engineering — Vocabulary, 2017, def. 3.4538). For evaluation of systems in actual use, we need validation.

Validation concerns whether the system is suitable for applied use. Validation is "the set of activities ensuring and gaining confidence that a system is able to accomplish its intended use, goals and objectives" (Systems and Software Engineering — Vocabulary, 2017, def. 3.4500). Validation evaluates how well a system satisfies the user needs. Typical validation processes consists of use cases and usability testing, to evaluate whether users have all they need to use the system. Validation combines technical inspection with human judgment and domain knowledge. That's why validation is a crucial element in evaluating sociotechnical systems, such as imputation processes. In the next paragraphs, I will operationalize the standards of verification and validation for evaluating imputation methodology specifically. I define two evaluation approaches, that together provide a toolbox for V&V of imputation workflows: computational evaluation and graphical evaluation.

Computational evaluation is a largely quantitative evaluation approach. It covers the use of metrics, tests, and simulations to evaluate properties of a system. The practice of computational evaluation combines *effectiveness* (i.e., did the process yield the correct result, e.g., accuracy, precision, recall, etc.), *system quality* (e.g., speed, coverage of all possible results, etc.) and, importantly, *user utility* (e.g., happiness, productivity, etc.) (Manning et al., 2008). Computational evaluation can therefore contribute both to verification as well as validation of systems. A type of computational evaluation for verification, for example, is the use of simulation studies to quantify the performance of statistical methods. For validation, computational evaluation can be useful, for example, to quantify a certain method's fault rate in applied use. The term 'computational evaluation' is not specifically defined for the field of statistics, but it could be useful. I want to plant a flag: how can we define computational evaluation in the context of imputation methodology?

Since 'computational evaluation' is not defined for statistics, there is no direct definition to use to define computational evaluation for imputation methodology. Computational evaluation stems originally from information retrieval research (see e.g. "Evaluation Measures (Information Retrieval)," 2025). One definition of computational evaluation is "to determine the quality of solutions attainable" (Dudek, 2004) Another definition of computational evaluation is "the process of ensuring an algorithmic solution is a good one: that it is fit for purpose" (Curzon et al., 2014). A working definition for imputation methodology is: the systematic assessment of algorithmic behavior and data-level outputs of imputation procedures. In this dissertation I will use computational evaluation as a suite of tools to investigate the domains of algorithmic stability, data plausibility, and user burden, next to inferential validity. These aspects are often already taken into account by imputers, but I try to systematize and operationalize these aspects that are often informal or *ad hoc*. Please note that computational evaluation does not replace statistical evaluation; it extends the scope of the evaluations. Computational evaluation encompasses all systematic, computer-based evaluation tools, including inferential estimates and statistical performance measures such as bias and confidence validity. Computational evaluation goes beyond statistical evaluation. Even if we know the process yields correct inferences, we can evaluate the 'fit' of the solution. For example, a certain model may be statistically valid, but take too long to converge for real-time use in a production pipeline or patient consult. Computational evaluation will then tell us that this statistically valid method is not fit for purpose.

Graphical evaluation is the second approach I will employ. This is a largely qualitative type of evaluation. Graphical evaluation stands in a long tradition within the scientific discipline of statistics. Many data analysis workflows encourage visual inspection prior to and after modeling. For example, statistical assumptions are often checked by means of diagnostic visualization. This practice is diagnostic because the purpose is to identify patterns and problems, such as mismatches between the data and model assumptions. Graphical evaluation helps users understand the system's behavior and judge whether it's acceptable for their needs. As a working definition, I will consider graphical evaluation to be the use of plots and figures to communicate system behavior to end users. For imputation methodology, graphical evaluation is mostly applicable as validation tool. Graphical evaluation can aid imputers in assessing user utility, such as usability of the system, appropriateness of the methods, and plausibility of the imputations. To show how computational and graphical evaluation can be useful in practice, the next section outlines a typical imputation workflow with `mice`.

What is involved in a `mice` workflow? (or: human in the loop)

The R package `mice` enables researchers with missing data problems to correct for differences due to missingness and maintain the appropriate level of uncertainty. Figure 5 shows an imputation workflow with `mice`. The flowchart shows four steps in the imputation workflow where human judgement is encouraged: inspecting the missingness, defining imputation models, inspecting algorithmic convergence, and inspecting imputed values. Not depicted in the flowchart are steps

that require external stakeholders (e.g., involving subject-matter experts). There are some decisions that you, as imputer, implicitly or explicitly make. The imputation function `mice()` will happily produce imputations with its defaults, but not necessarily "good" imputations.

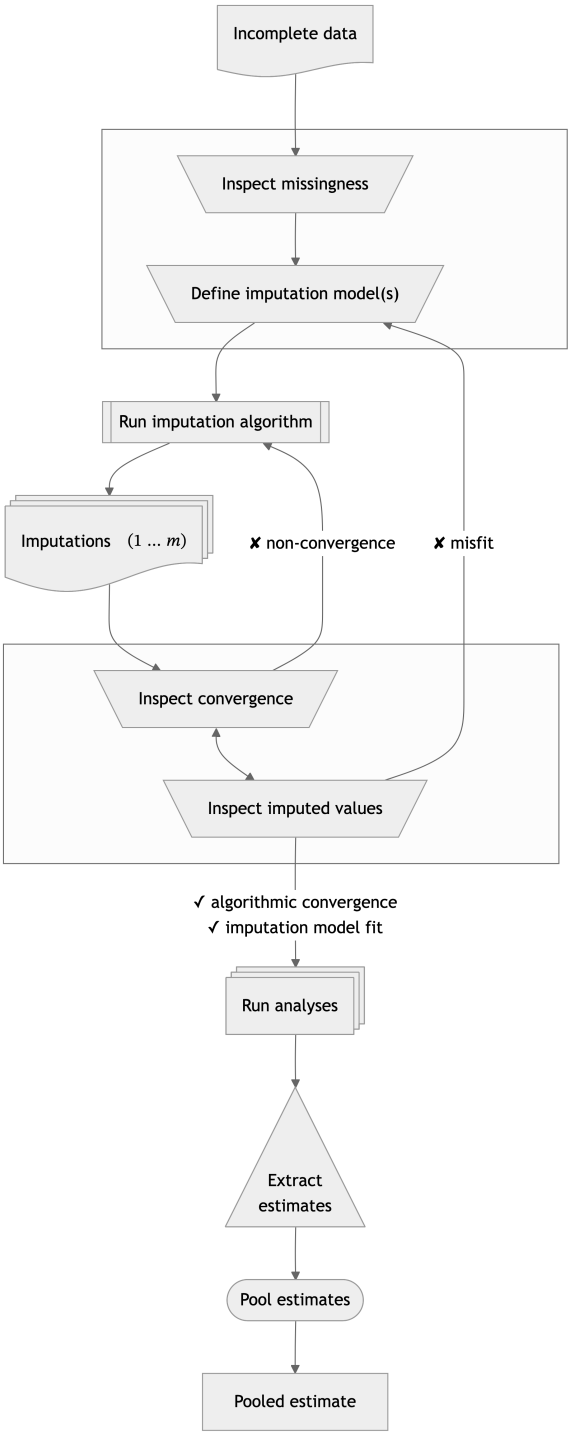
A data analyst faced with incomplete data should not start imputing right away. They should first ask themselves: what kind of missingness am I dealing with? Inspect the patterns of missingness and consider which missingness mechanism(s) might be at play. A missing data pattern plot tells you where missingness occurs (which variables, which cases) and which variables tend to be missing together. For definitive conclusions about the missing data mechanism, however, we need external information.

Imputation with `mice` assumes ignorable missingness. MCAR and MAR are modeling assumptions, not empirically verifiable facts. In practice, it it impossible to determinately differentiate between MCAR and MNAR. But specific MAR mechanisms may become apparent through computational and graphical evaluation (e.g., by calculating or plotting associations between observed data and missingness indicators). If MNAR is suspect, use dedicated tools and/or design sensitivity analyses to assess how robust your conclusions are. If MAR mechanisms show up, include the relevant relations into the imputation models.

Imputation models consist of imputation methods and imputation model predictors. For each incomplete variable, `mice` asks you to specify an imputation method. This is a choice of the functional form of the imputation model (e.g., logistic regression for dichotomous variables). How do we know which functional form to choose? The simplest option is to accept `mice`'s defaults, which are motivated by simulation studies and pragmatism. For example, the default method for continuous variables is the semi-parametric method `pmm`: predictive mean matching. With `pmm`, observed data entries are used as donors when imputing missing entries, conveniently yielding imputed values that lie within the range of the observed data. Diverting from these defaults might be useful in case of big, wide, or sparse data. That is, some methods can be computationally inefficient with big data (large n, large p), mathematically undefined with wide data ($n < p$), or yield suboptimal imputations because of a lack of donors in sparse data ranges. Moreover, if the column type in the incomplete data is not correctly specified, `mice` will assign incorrect imputation methods. Therefore, inspecting the distribution of each incomplete variable is advised. You can use bar charts, histograms, and density plots of the observed part of the variable to assess the distribution of the incomplete variables and determine imputation methods. Visual inspection can also be used to check whether there is skew or spikes (e.g., structural zeros) that require transformations. In short, the imputation method determines the distribution of the imputed values, which should match the distribution of the observed values to at least some extent.

The next major decision concerns the predictor matrix: which variables should be used to impute which? Some common sources of input are the associations assumed by the analysis model, associations in the observed data, associations with missingness indicators, and external input and design information. If you already know your primary analysis model (e.g., multivariate regression), it is good practice to include all analysis model predictors and outcomes in the imputation models. Using the associations implied by the analysis model may safeguard against

Figure 5: Flowchart of multiple imputation with `mice`



Note. Processes that require human intervention are depicted with trapezoids in the flowchart. Notably, these steps occur in pairs: before imputation, the user can inspect the missingness and define imputation models; after imputation, the user can evaluate the output of the imputation algorithm.

uncongeniality (Meng, 1994). Roughly speaking, the concept of congeniality refers to how well the imputation model and complete data model 'fit together', and could stem from the same data-generating process. When an analysis model variable is omitted as a predictor from the imputation model, we are implicitly assuming that its association with the imputed variable is zero (Rubin, 1996). If we constrain these parameters to zero in the imputations, we are systematically suppressing relations in the imputed data (i.e., biasing the regression coefficients in the analysis model towards zero). We should therefore always use the complete data model—if available—when building imputation models. If there is already one specific data analysis model for the effect of scientific interest that you have in mind, please using consider substantive model compatible fully conditional specification (SMC-FCS, Bartlett et al., 2015). SMC-FCS is optimized for known analysis model models: we assume a complete data model and derive the imputations from there. Imputation with `mice`, however, is complete data 'agnostic'. We do not need to know the model of scientific interest upfront. Instead, we can rely on other sources of information for choosing imputation model predictors.

Deciding which associations to include in the imputation models can be a data-driven process. A data-driven approach would fit many modern data science pipelines, where the imputed data may be used for several exploratory and/or unsupervised analyses. But also when the analysis model is known, it is advised to inspect associations in the incomplete data when choosing imputation model predictors. You may include a broader set of predictors than strictly needed for the complete data model by adopting 'auxiliary variables' (i.e., extra imputation model predictors that are not part of the analysis model or variables of interest). For example, we can include predictors that are strongly associated with the incomplete variable: ("include an auxiliary variable if its correlation with the main variable is 0.5," Nguyen et al., 2021, p. 4). Not just associations with the incomplete variable itself may be used to select imputation model predictors. We can also use the association of the missingness indicator with other variables in the data. Predictors that explain missingness can help you approximate MAR or correct for design-related patterns. Other design-related patterns require expert knowledge e.g., about branching or which external information should be included to mitigate MNAR. Choosing imputation model predictors encodes your beliefs about what information is relevant for reconstructing the incomplete data and capturing missingness mechanisms.

After specifying the imputation model methods and predictors, there are still some optional parameters to consider changing in the imputation process. For example, the number of imputations (argument `m` in the `mice()` function) and number of iterations in the imputation algorithm (argument `maxit`) may affect the quality of the imputations and resulting statistical inferences. For guidance on these parameters, see e.g. van Buuren (2018) and the `mice` documentation. We can now run the imputation algorithm and evaluate the output.

After a successful `mice` run, you need to ask whether the imputation process behaved adequately. Some post hoc evaluation tools include algorithm-level diagnostics (e.g., traceplots of key parameters over iterations to assess algorithmic convergence), visual inspection of the distribution of the imputed values (e.g., to identify mismatch between observed and imputed data distributions), and sensitivity analyses (e.g., via over-imputation, which temporarily

treats some observed values as missing to compare imputations to the true values). These evaluations are neither fully automated nor purely objective. They involve building and reading plots, interpreting diagnostics, and deciding what degree of misfit is tolerable for your research aims. Without graphical or computational diagnostic for non-convergence, users might analyze imputed values of non-converged chain. Another question to consider is: do the imputed values lie within the range of the observed values? For statistical validity this is not needed. Imputation theory does not require the imputed values to be plausible. Rubin (1976, 1987) developed the methodology to get correct population-level inferences from incomplete samples, not at the individual case-level within the sample. Bluntly stated: imputation theorists don't care for the imputed values themselves, as long as they result in statistically valid inferences. But end-users (e.g. clinicians) often do care for plausibility. For example, imputing negative values for age will yield 'unrealistic' imputed values.

Taken together, these decisions show that a `mice` workflow is not a push-button solution. It is a modeling exercise in which the user is involved in model building and evaluation. Imputation with `mice` is a sociotechnical process. While `mice` is a technical system (i.e., a piece of software with algorithms to generate imputations) applications of `mice` are sociotechnical. In practice, imputation workflows include `mice`, the imputation script, the analysis script, visualizations, and the researcher interpreting the output and deciding what to accept. Human input makes imputation workflows subjective to individual choices and preferences. Since "systems have properties that only become apparent when their components are integrated and operate together", we should evaluate the imputation workflow not as separate functions, but as a whole. For imputation methods this means that they should work in practice, not just in simulations. For such systems, analytic guarantees (e.g., asymptotic unbiasedness of MI) are not enough. We need computational and graphical checks on the whole workflow.

What to expect in this dissertation? (or: structure & scope)

In this dissertation, I investigate how imputation workflows with `mice` (van Buuren & Groothuis-Oudshoorn, 2011) can be evaluated. I frame imputation processes as sociotechnical systems, based on a system engineering conceptual framework. This allows us to apply the verification and validation (V&V) standards, as defined for software systems (IEEE Standard for System, Software, and Hardware Verification and Validation, 2025). I operationalize V&V through computational and graphical evaluation tools. Verification of imputation methodology mostly concerns computational evaluation tools, such as simulation. Validation requires both computational as well as graphical evaluation tools, to assess the operational utility of imputation workflows. Each of the three substantive research chapters in this dissertation covers one of these aspect: computation evaluation for verification, computational evaluation for validation, and graphical evaluation for validation, respectively.

Computational evaluation for verification

In Chapter 2, I investigate how we can standardize the verification of imputation methodology. I posit simulation studies as a computational evaluation tool to judge whether imputation strategies work experimentally. A question underlying this chapter is: how can computational evaluation criteria be systematically incorporated into simulation studies for imputation methodology? Other questions that come up are: how can we design simulation studies that are fair and comparable with one another, which simulation conditions to consider, which performance metrics to report, and how to document this in such a way that others can reproduce and reuse the evaluations? I address these questions based, in part, on a literature review I was involved in as a research assistant (Liu et al., 2023). This chapter focuses on methodologists (who design imputation methods and study their properties) rather then imputers (applied researchers who apply imputation models to specific datasets). Methodologists need comparable, reproducible evaluations of imputation methods across scenarios. Without a shared evaluation framework, claims about 'valid, unbiased estimates' might not translate to practice. I argue that imputation methodology should be subjected to a standardized evaluation framework for simulation studies.

Computational evaluation for validation

In Chapter 3, I study how one might diagnose algorithmic non-convergence in `mice` (van Buuren & Groothuis-Oudshoorn, 2011). This may be considered a type of computational evaluation for validation, since non-convergence diagnostics can flag problems in the imputation process to `mice` users. Non-convergence may happen because MICE (as implemented in `mice`) is an iterative algorithm.

Incomplete variables are imputed one-by-one, conditional on other variables. If these other variables are incomplete too, iteration is required. After the first cycle through the columns, we can get better imputations based on the now-updated imputation model predictors. In `mice`, this cycle is repeated several times until convergence is reached (by default five iterations). Because of its iterative nature, convergence *should* be a requirement for valid inference, or is it? This chapter investigates two questions: 1) how does convergence in iterative imputation affect inferential validity, and 2) how can non-convergence be diagnosed? These questions are answered via a simulation study.

Graphical evaluation for validation

The fourth chapter introduces the software `ggmice` (Oberman et al., 2025), an R package for building and evaluating imputation models. The `ggmice` package offers tools for the visualization of incomplete data, imputation models, and imputed data. `ggmice` supports `mice` workflows by making imputation modeling choices visible and, therefore, open to scrutiny. Compatibility with `ggplot2`'s 'grammar of graphics' (Wickham, 2016) makes `ggmice` graphs

extendable with the familiar tools of the `tidyverse` (Wickham et al., 2019), and adaptable for publication-ready visualizations.

Although `ggmice` has not undergone the traditional journal-based peer review, it has been vetted in practice. The package has been released on CRAN in 2022 (Oberman et al., 2025). CRAN downloads are reaching approximately 1.000x per month at the time of writing. The accompanying Zenodo publication (Oberman, 2022) has been cited in seven scientific works. Open source code and issue tracking are available in a repository on the programming platform GitHub (github.com/amices/ggmice). There have been multiple external 'Issues' and 'Pull Requests' on GitHub, indicating that other researchers and developers have actively reviewed and improved the codebase. In short, there is active use of and involvement in `ggmice`, as indicated by regular downloads from CRAN, citations to the Zenodo record in the scientific literature, and community feedback via platforms such as GitHub and StackOverflow.

`ggmice` is developed according to 'open source' and FAIR principles. The package is implemented in the open source language R, distributed via CRAN and GitHub, and archived on Zenodo. Its evolution is driven by user feedback (e.g., GitHub issues, StackOverflow questions, conference discussions, teaching) and collaboration (e.g., code contributions, statistical consulting with external partners).

Other design principles are grounded in `ggmice`'s compatibility with the popular R packages `ggplot2` and `mice`. `ggplot2` offers flexible data visualizations based on the 'grammar of graphics' framework. `mice` offers flexible imputation workflows for inference with incomplete data. Together, these packages inform the design and functionalities of `ggmice`.

`ggmice` produces `ggplot` objects, leveraging the 'grammar of graphics' to create layered, easily adjustable plots. The figure that appears is the output of a series of design decisions. The grammar of graphics treats plots as structured ways of reasoning from data (Wilkinson, 2012). Layering decomposes graphics into a small set of elements. For example, variables are translated into visual properties (axes, color, etc.) via 'aesthetic mapping', and geometric objects determine the shape of the graph itself. A graphic then is a combination of several elements, and layering allows for building complex figures. The user can add or change layers without breaking the entire figure, making the graphics transparent and manipulable.

`mice` imputation models are substantive model agnostic: variable-by-variable definition of imputation models. `ggmice` supports this. Because `mice` is substantive-model agnostic, `ggmice` offers tools to inspect whether the imputation model fits the observed data, instead of imposing particular analysis models. Assumptions: imputation model should fit the observed part, imputation model should be congenial `ggmice` provides tools to see observed and imputation side-by-side, adhering to advice to evaluate the distribution of observed and imputed data (Nguyen et al., 2017).

`ggmice` offers a unified toolbox for the development and evaluation of imputation models. The `ggmice` package aims to solve the following problems:

- No direct graphical comparisons between incomplete and imputed data. Visualizations with `ggmice` can be made to inspect marginal and joint distributions of incomplete variables side-to-side to compare incomplete and imputed data. Without `ggmice`, it is cumbersome to see the data before and after imputation in comparable plots. `ggmice` offers functions that visualize incomplete data distributions and their completed counterparts.
- No previously existing methods for visualizing `mice` imputation models. The structure of `mice` imputation models (i.e., predictor matrix and methods vector) are often printed to the R console as text. Complex imputation models are easier to review through visualization. `ggmice` provides imputation model plots that make complex imputation specifications easier to review and discuss.
- Limited chart types and design options for imputed data. Existing plotting tools for `mids` objects not easily editable or modifiable with different chart types. Visualizations with `ggmice` can be extended and adapted with `tidyverse` functions to create publication-ready graphs of imputed data (reflecting the uncertainty due to missingness).

The `ggmice` package fills a hiatus in the software landscape, with other packages generally supporting either `mice` or `ggplot2`, but not both. Table 1 provides an overview of R packages that offer visualization tools for incomplete and imputed data. In short: with `mice` you cannot plot incomplete data distributions, all other packages lack support for `mice`-imputed data, and visualization of `mice` imputation models did not exist at all prior to `ggmice`.

In particular, visualization of `mice` imputation models did not exist prior to `ggmice`. And side-by-side visualizations of incomplete and imputed data was not supported in a unified graphical framework (nl., `naniar`'s (Tierney & Cook, 2023) `ggplot` graphs with missing data, versus `mice`'s `lattice` (Sarkar, 2008) plots with imputed data).

Table 1: Differential functionality of `ggmice` versus other R packages

	ggmice	mice	naniar	VIM
Incomplete data (e.g., `data.frame` object)				
– missing data patterns	+	+	±	+
– joint & marginal distributions	+	−	+	+
Imputation model building				
– associations in observed data	+	−	−	−
– associations with missingness indicators	+	+	−	−
– conditional distribution with missingness indicator	+	−	−	−
– imputation models	+	−	−	−
Imputed data (e.g., `mids` object)				
– trace plot of the imputation algorithm	+	+	−	−
– joint & marginal distributions	+	+	−	−
User utility				
– extendability (i.e., non-deterministic graph types)	+	−	±	−

Table 1: Differential functionality of `ggmice` versus other R packages

	ggmice	mice	naniar	VIM
− editability (i.e., design adaptable for publication)	+	−	+	−
− popularity (i.e., monthly downloads)	−	+	±	±

Note. This is an incomplete overview of the R package's functionalities. The purpose of this table is to display *differences* in functionality based on the features offered by `ggmice`. I intend to show `ggmice`'s added value compared to existing methodology, not a universal comparison. Symbols denote high/well supported (+), medium/moderate support (±), and low/not supported (−).

`ggmice` does not pretend that visualization yields objective truth. A good graphical fit between observed and imputed data (e.g. overlapping scatterplots) is not a guarantee of statistical validity or correct missingness assumptions. Visual diagnostics can only suggest plausibility or highlight obvious misfits (imputation models should fit the observed part of the data). They cannot determinately identify the missing-data mechanism. The interpretation remains subjective (e.g. convergence, MAR assumption, etc.). Visualization is not a replacement for, but support to statistical and computational evaluation. In short, `ggmice` is an attempt to make imputation model assumptions and consequences as transparent as possible.

Scope

In this dissertation, I will focus on missing data problems with item non-response and ignorable missingness mechanisms. You might expect me to go into other types of missingness (e.g., unit non-response, and MNAR mechanisms), but these are outside the scope of this dissertation. I will also limit the applications in this dissertation to the general linear models framework for statistical inference. This excludes other types of substantive models and analysis frameworks (e.g., deep learning methods, and causal inference). The imputation method of interest in this disseration is FCS in the flavor of MICE. However, some results may generalize to other iterative (FCS) imputation algorithms.

2 Chapter 2: evaluation

Published as Oberman & Vink (2024).

[TODO: add credit statement, merge formatting]

Received: 31 March 2022 | Revised: 1 February 2023 | Accepted: 8 February 2023

DOI: 10.1002/bimj.202200107

Biometrical Journal

REVIEW ARTICLE

Toward a standardized evaluation of imputation methodology

Hanne I. Oberman | **Gerko Vink**

Departement of Methodology & Statistics, Utrecht, The Netherlands

Correspondence
Hanne I. Oberman, Department of Methodology, and Statistics, Padualaan 14, 3584 CH Utrecht, The Netherlands.
Email: h.i.oberman@uu.nl

This article has earned an open data badge "**Reproducible Research**" for making publicly available the code necessary to reproduce the reported results. The results reported in this article could fully be reproduced.

Abstract

Developing new imputation methodology has become a very active field. Unfortunately, there is no consensus on how to perform simulation studies to evaluate the properties of imputation methods. In part, this may be due to different aims between fields and studies. For example, when evaluating imputation techniques aimed at prediction, different aims may be formulated than when statistical inference is of interest. The lack of consensus may also stem from different personal preferences or scientific backgrounds. All in all, the lack of common ground in evaluating imputation methodology may lead to suboptimal use in practice. In this paper, we propose a move toward a standardized evaluation of imputation methodology. To demonstrate the need for standardization, we highlight a set of possible pitfalls that bring forth a chain of potential problems in the objective assessment of the performance of imputation routines. Additionally, we suggest a course of action for simulating and evaluating missing data problems. Our suggested course of action is by no means meant to serve as a complete cookbook, but rather meant to incite critical thinking and a move to objective and fair evaluations of imputation methodology. We invite the readers of this paper to contribute to the suggested course of action.

KEYWORDS

evaluation, imputation, missing data, simulation studies

1 | INTRODUCTION

Imputation is a state-of-the-art technique for drawing valid conclusions from incomplete data. The technique has earned a permanent spot in research and policy-making, demonstrated, for example, by the detailed manual created by the National Research Council (Little et al., 2012). Although top-down enforcement of valid ways to handle missing data is not yet very pronounced, an increasing amount of researchers and data scientists are embracing and developing imputation methodology. After all, the principle of imputation is very intuitive. The idea behind imputation is to impute (fill in) missing values to obtain a valid estimate of what could have been. The completed data that are thus obtained can be analyzed by standard techniques, which conveniently separates the missing data problem from the analysis.

For some data analyses, a single imputation may suffice, while for other data analyses, multiple imputations are needed. For example, in the case of inferential analyses, such multiple imputations (Rubin, 1976) have proven to be a valuable technique for obtaining valid inferences on incomplete data sets. The resulting multiple inferences on multiple completed data sets can be combined into a single inference using Rubin's rules (Rubin, 1987, p. 76). When generating predicted values is the goal of the data analysis, a single imputation may already generate sufficient predicted values (Nijman et al., 2020).

In all scenarios, it can be said that the quality of the solution obtained by imputation depends on the statistical properties of the incomplete data and the degree to which the imputation procedure is able to capture these properties when modeling missing values. In general, it holds that modeling missing data become more challenging when the amount of missingness increases. However, when (strong) relations in the data are present, the observed parts can hold great predictive power for the models that estimate the missingness. In that case, imputation may be substantially more efficient than the ubiquitous complete case analysis.

When evaluating the statistical properties—and thereby the practical applicability–of imputation methodology, researchers most often make use of simulation studies. Such studies are typically composed of an analysis pipeline in which data are subsequently generated, made incomplete, and imputed repeatedly under different simulation conditions. A set of evaluation criteria is then postulated to evaluate the performance of one or more missing data methods. Recent attention on best practices for method evaluation by means of simulation (Morris et al., 2019) and Pawel et al. (2022)'s advice to preregister simulation protocols will guide methodologists in conceptualizing their simulation workflow. However, there is no consensus on how to design, execute, and report simulation studies aimed to evaluate imputation methodology. Without a "gold standard" for evaluation, the validity and comparability of simulation setups may differ tremendously from one developer to another. This brings forth a chain of potential problems in the objective assessment of imputation method performance within and across studies, which could lead to suboptimal use of imputation in practice.

The purpose of this paper is threefold: First, we raise some concerns with respect to evaluating imputation methodology. These concerns stem from careful consideration with fellow "imputers" and from encounters as reviewers for statistical journals. Second, we provide imputation methodologists with a suggested course of action when using simulation studies to evaluate imputation techniques for missing data problems. This suggested approach should identify common ground, but is in no way intended as an absolute solution. This identifies the third purpose of our paper: discussion. We hope to elicit critical thinking regarding the problems at hand. We are all convinced that our methodology has some merit. But for sake of progress, it would be much more advantageous if the aim of our evaluations would go beyond *proving the point* and would legitimately consider the statistical properties.

2 | WHY SOME EVALUATIONS SHOULD NOT BE TRUSTED

The ideal evaluation of imputation methodology may differ between studies. A simulation study developed for the comparison of several imputation methods will typically have a different design than one aimed at establishing the inferential validity of a single (novel) method. How the evaluated methods are used in practice thereby governs the simulation procedure. Since the simulation aims and setup are intertwined with the choice of the evaluated imputation methods, one should always assess the suitability of the chosen evaluation metrics with the imputation methods' purpose in mind.

We limit the scope of this paper to comparative simulation studies in the context of statistical inference. With that, we exclude the evaluation of imputation methodology for detailed aspects of causal inference and prediction. The type of simulation study under consideration in this paper requires a form of "comparative truth" to assess the inferential validity of imputation methods. This is in contrast to simulation studies aimed at comparing the predictive performance of imputation and prediction method pairs. Such a simulation design typically does not start out from a "ground truth" (e.g., a complete data set in which missingness is subsequently induced by the simulator). Rather, the imputation and prediction method pairs are evaluated on their ability to yield high predictive accuracy in one or more incomplete benchmark data sets (Liu et al., 2021). In these designs, only the comparative performance of methods may be established. We do not recommend this approach if the inferential validity of the imputations is of interest. For an overview of missingness in the prediction context, see, for example, Wood et al. (2015) and Sperrin et al. (2020). Missing data within a causal inference framework have been described by, for example, Moreno-Betancur et al. (2018) and Mohan and Pearl (2021).

Within the scope of comparative simulation studies aimed at statistical inference, we observe several problems. To demonstrate the broad impact of these problems, we recognize the following four distinct categories: problems with simulation design, problems with data generation, problems with missingness generation, and problems with performance evaluation. We further detail the impact each of these problems may have on the validity of evaluations.

2.1 | Simulation design

Before setting up a simulation study, the simulator should clearly define and describe the scope of their evaluations. If the simulation parameters are unclear, any methodological problems with simulation study design may be obfuscated by the cognitive problem of generalization (Greenland, 2017). For example, misinterpretation and (unintended) questionable research practices can prompt extrapolation beyond the scope of the simulations (Greenland, 2017; Pawel et al., 2022). Imputation methods that seem to work well in simulations may then not be suitable in practice, when applied to incomplete empirical data.

One potential reason for discrepancies between imputation method performance in simulations versus applications is the use of sampling variance in the simulation design. For example, when dealing with infinite populations, the conventional pooling rules proposed by Rubin (1987, pp. 76–77) apply. When dealing with a finite incomplete population, some adjustment is needed for pooling rules to yield correct inferences. In such cases, valid inferences can be obtained by excluding the sampling variance components from the calculation of the total variance about $(Q - \bar{Q})$ (Raghunathan et al., 2003; Vink & Van Buuren, 2014), where Q would be the estimand and $\bar{Q} = \sum_{l=1}^{m} \hat{Q}/m$ is the average over the m imputed estimates (cf. Rubin, 1987, p. 76-77). Vink and Van Buuren (2014) propose that the total variance $T = B + B/m$ would then solely rely on the between imputation variance $B = \sum_{l=1}^{m}(\hat{Q}_l - \bar{Q})'(\hat{Q}_l - \bar{Q})/m - 1$ and demonstrate that the degrees of freedom would then equal $m - 1$.

We can make use of this unique real-world data property when simulating the performance of incomplete data methodology. When we take a single drawn complete data set as our comparative truth, just inducing simulated missingness would suffice, and the necessary Monte Carlo variance for our evaluations would stem from the sampled missingness. The above-detailed pooling rules can be used to obtain valid inferences in those simulations. A detailed overview of simulation strategies that adopt this approach, as well as conventional simulation strategies can be found in Vink (2022).

We would like to note that it is not necessary to exclude sampling variance from missing data simulations, as the additional variance induced by the sampling step would be ignorable. In practice, however, we have found it to be convenient to allow for a single data set to serve as comparative truth in all simulation repetitions. Especially for data-driven simulation strategies with complex data transformations, it can be challenging to derive the true parametric references to evaluate against. Evaluating against a single generated set would then avoid many procedural problems and computational challenges. In practice, there are also other benefits to this simulation approach as it is computationally convenient, requires less data archiving, and may potentially sharpen comparisons between imputation methods, thereby clarifying inconclusive results on method imputation performance. After all, we are interested in solving for the missingness and can do without the ignorable noise induced by the sampling mechanism for evaluation in such studies.

A well-designed simulation does not guarantee fair evaluations. Estimands and other simulation targets should be clearly and unambiguously defined in the context of the study aim (Petersen & Van der Laan, 2014). The study aim and the required level of precision may also inform the number of simulation repetitions (e.g., as determined from a maximum tolerable level of uncertainty in terms of a performance measure's Monte Carlo error; see Morris et al., 2019). The simulation design should ideally be fully factorial, that is, varying each simulation condition against all other conditions. There may, for example, be interactions between the source and amount of missingness in the incomplete data and the efficacy of imputation methods—that is, the validity of the assumed missingness mechanism becomes increasingly important with higher missingness proportions (Schouten & Vink, 2021). If there is more than one imputation method under evaluation, the simulation design should apply each method to every incomplete data set. Applying all methods to the same incomplete data set is computationally convenient, and minimizes unnecessary variation, which makes for fairer comparisons.

2.2 | Data generation

To evaluate the ability of an imputation routine to handle missingness, a form of ground truth has to be established. Those who perform simulation studies are in the luxury position to establish the truth beforehand by choosing a data-generating mechanism. Data-generating mechanisms define how a complete dataset is obtained at the start of each simulation repetition. We highlight two general data generation approaches: (i) model-based simulation, in which data are drawn from a known statistical model or probability distribution, such as the multivariate normal distribution; and (ii) design-based simulation, where data are sampled from a sufficiently large observed set, such as official registers. Model-based data-

generating mechanisms have advantages in flexibility and precision, because data are generated from a known statistical model and the true theoretical parameters can be derived. A design-based approach is often used in situations where a probability distribution is not available, or where real-life data structures are of interest. Of course, the simulator should choose their data-generating mechanism in line with the study scope. Other methods to simulate data may be more suitable, for example, synthetization of the actual data-generating process (Li et al., 2022; Volker & Vink, 2021).

The challenge with model-based data-generating mechanisms is that a method's performance on simulated data may not translate to empirical data. Real-life data may not follow a known theoretical distribution, so there is no guarantee that simulation results are generalizable. Moreover, data are often generated such that the problem being studied is most pronounced, for example, with consistently high correlations between groups of variables. This results in simulated data that contain such valuable information structures that no matter what type of missingness would subsequently be induced, the observed parts of the data will still hold much (if not all) of the information about the missing part. Unsurprisingly, the performance of any imputation method will then be evaluated as good.

Another threat to the generalizability of model-based simulations is the use of a single model for both data generation and imputation. If data are generated following a model that is also used for imputing the data, the imputation approach will be deemed good (or better than other methods) purely due to the evaluated conditions being in favor of the problem that is studied. Other potentially unfair comparative advantages in favor of a certain imputation method may occur due to characteristics of the generated data, such as the number of observations, the number of variables, the measurement level, and the coherence between variables. In contrast to design-based studies, such characteristics are not always explicit simulation conditions, which may give a false sense of objectivity.

An obvious problem with design-based simulation is that obtaining a large data set without missing entries can be challenging. Most real-world data contains at least some missing entries, for which the true underlying missing data model is—by definition—unknown. Therefore, the simulator needs to deal with missingness in *some* way before incomplete empirical data can serve as comparative truth in the simulations. It might seem like an intuitive solution to only draw complete cases from the large data set, which would indeed yield complete samples. However, there may be inherent differences between cases with and cases without any missing values, due to the unknown missing data model. Only sampling complete cases from the data set may thus result in samples that fail to capture all relevant real-world conditions, which, in turn, refutes the main reason for using design-based simulation. Another way to deal with missingness in a design-based simulation is to impute the incomplete data set once, to obtain a single completed data set to draw samples from. Unfortunately, this practice may favor the imputation method that was used in this initial imputation step throughout any further evaluations. Just to be clear: leaving the missingness as-is and inducing additional missing values to impute is no option here, because we would not have a real and unbiased comparative truth.

2.3 | Missingness generation

Often, reports of simulation studies remain vague about the missingness conditions under investigation and, even worse, some authors only report something like:

> We generated missing data following a missing at random missingness mechanism.

This should be considered unacceptable, as claims about the validity of the imputation inference heavily depend on the simulated missingness conditions, such as missingness mechanisms and missingness patterns. Missingness mechanisms describe the relationship between missing entries and observed data values, whereas missingness patterns concern the location of missing entries across incomplete data (Little & Rubin, 2020, p. 8). Under this definition, there is a row-wise element to the missingness pattern describing which variables are jointly observed, and a column-wise element encompassing the amount of missingness in the data.

Even the terminology on missingness generation can be confusing. Does a missingness proportion of 50% mean that half of the entries in an incomplete data set are missing, or that half of the rows have at least one missing entry? In this paper, we will henceforth refer to the latter as the proportion of incomplete cases, and keep the term "missingness proportion" restricted to the variable-by-variable interpretation. The distinction between the two concepts, however, is not always clear in the literature, and convoluting the terms may lead to incorrect generalizations, because they rarely mean the same (e.g., a proportion of 50% incomplete cases in bivariate data could translate to a missingness proportion of 25% in both variables, or one completely observed variable and one with 50% missingness). Simulators should therefore

be explicit in their description of the missing data-generating model and should make sure that the model fits the aim of their simulations.

Extrapolating simulation results from a specific missing data-generating model to more intricate (empirical) missingness could lead to suboptimal advice in practice. For example, if a simulator induces missingness in the outcome variable of their analysis model exclusively, they may inadvertently induce missingness according to the special case described by Van Buuren (2018, §2.7). Under this specific missingness model, list-wise deletion may outperform any imputation method. Such a conclusion would, unfortunately, only translate to data that adhere to the same special case, while biasing inferences in other cases. Another example of potential side effects of missingness generation may occur when multivariate missingness is generated using step-wise univariate missingness induction. The resulting incomplete data may then not have the desired statistical properties or unintentional nonignorable missingness may be generated (Robins & Gill, 1997; Schouten et al., 2018). This complicates any definitive conclusions about imputation method performance in relation to the data generation parameters.

Missingness should be induced according to several sets of missing data conditions. We encourage simulators to consider different missingness patterns and mechanisms. Missingness mechanisms were first defined in Rubin's seminal work (Rubin, 1976). For more recent considerations, see, for example, Doretti et al. (2018), Little and Rubin (2020), Moreno-Betancur et al. (2018), Mohan and Pearl (2021), Mealli and Rubin (2015), Schomaker (2021), Seaman et al. (2013), and Schouten and Vink (2021). Although not every missingness mechanism is realistically assumed in practice, they can all offer valuable insights as simulation condition. A disconnect between induced missingness, real missingness, and assumed missingness may result in simulation studies that are not as informative as they could be.

Truly random missingness across all data entries ("missing completely at random" or MCAR under Rubin's (1976) definition) may be considered as a necessary simulation condition for the evaluation of imputation procedures, because the statistical properties of the observed data given the missing data are known. Any imputation routine that cannot at least mimic the performance of the observed data inference should be deemed inefficient in the scope of the simulation. If an imputation method is not able to solve the problem (i.e., yield valid inference) under MCAR, the statistical properties of the procedure are not universally sound. Sadly, the straightforward case of MCAR is often neglected in simulation studies, whereas it offers an informative reference condition in many simulation setups.

Besides MCAR missingness, observed-data-dependent missingness mechanisms—such as missing at random (MAR) missingness—are paramount in statistical inference investigations. A straightforward technique for inducing univariate MAR missingness is described in Van Buuren (2018, §3.2.4), whereas generalizations to multivariate MAR missingness can be found in Schouten et al. (2018). If the missingness is to be induced in longitudinal data, generating the missingness through autoregressive MAR models can be useful (see, e.g., Shara et al., 2015, models 2 and 3). It is advisable to investigate varying shapes of MAR missingness to achieve a more realistic indication of the robustness of the imputation performance across the range of random missingness. The effects of different types of MAR mechanisms are described in Schouten and Vink (2021).

Inducing an MAR mechanism presents the simulator with a choice, namely, the type or functional form of the missingness model. Given the simulated data distributions, one random missingness model may be far more disastrous to the observed information than another model (Schouten & Vink, 2021). This may influence the performance of some (but not necessarily all) imputation routines. For example, inference from hot-deck techniques such as predictive mean matching (Little, 1988; Rubin, 1986) may be more severely impacted by large amounts of one-tailed missingness than inference from parametric techniques. It would be a shame to overlook such results due to the focus on a single functional form of the missingness-generating model.

Although MAR missingness is often considered as a simulation condition, the problem of spurious MAR is generally overlooked. With MAR missingness mechanisms, observed relations in the data are used to induce missingness during simulation (e.g., weight is made incomplete based on observed gender to mimic a situation wherein one gender is less likely to disclose their weight). These relations in the observed data may, however, be weak or nonexistent. If MAR is induced based on weaker relations in the true data, claims for a method's applicability to situations where the missingness is random become less valid. The most extreme example would be when MAR is induced from data without multivariate relations. The inferential implications of the missingness would then mimic those of MCAR, even though the missingness is generated randomly. Figure 1 demonstrates this: Both univariate MAR mechanisms in Figure 1 are induced based on the right tail, but only the mechanisms for which the *variable to ampute* and the *missingness covariate* have a positive correlation will yield valid MAR mechanisms. In the case of zero correlation between the data columns, iterative univariate non-MCAR missingness schemes may induce invalid missingness mechanisms that mimic MCAR. Table 1 demonstrates this and highlights the importance of including complete case analysis as an evaluation analysis to allow for identifying

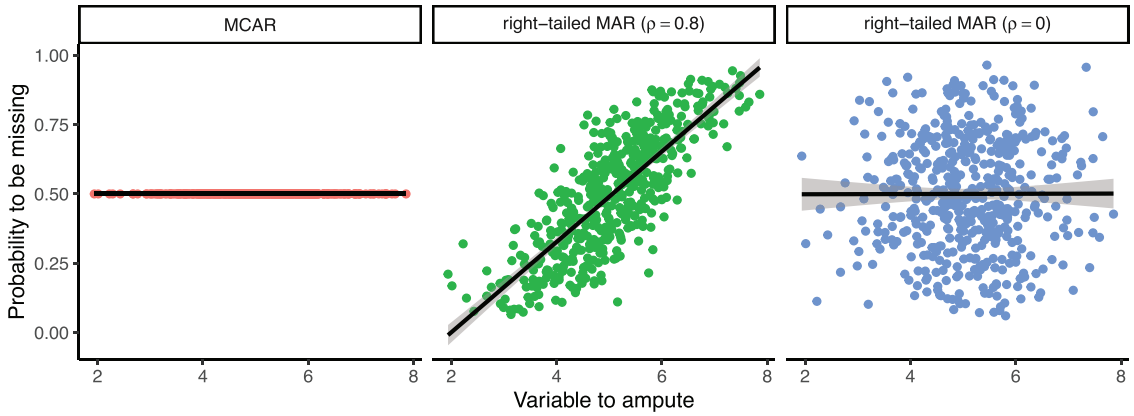


FIGURE 1 Three different realizations of missingness generation for MCAR and right-tailed MAR missingness mechanisms for different levels of correlation ρ between the *variable to ampute* and the *missingness covariate* that guide the probability to be missing. Note that the conditional probability of right-tailed MAR ($\rho = 0$) mimics that of MCAR because the variable to ampute and the missingness covariate have $\rho = 0$.

TABLE 1 Inferences obtained by complete case analysis (CCA) and multiple imputation over 1000 simulations. Displayed are the bias of the mean, coverage rate (cov) of the corresponding 95% confidence interval, and the confidence interval width (ciw). Imputations are generated by Bayesian linear regression imputation.

		CCA			Imputation		
Mechanism		**Bias**	**Cov**	**Ciw**	**Bias**	**Cov**	**Ciw**
[$\rho = 0.8$]	mcar	0.000	0.940	0.178	−0.000	0.941	0.160
[$\rho = 0.8$]	mar	0.335	0.000	0.169	0.004	0.954	0.190
[$\rho = 0.0$]	mar	0.001	0.973	0.180	0.011	0.939	0.311

such invalidly generated missingness mechanisms in simulation. In fact, this would amount to one of the special cases described by Van Buuren (2018) under which complete case analysis would be more efficient than imputation: the missingness does not depend on the incomplete variable. This property might be useful in practice, but considering it as a condition to evaluate the performance under MAR missingness is pointless.

A nonignorable or MNAR mechanism—where the unobserved values themselves may also play a role in the probability to be missing—might fall outside many studies' scope, yet could yield informative insights for daily practice. Even if an imputation method is specifically developed with ignorable missingness in mind, chances are that the method will be applied to nonignorable empirical missingness at some point in time. After all, for every MNAR mechanism, there is an MAR mechanism with equal fit and one cannot definitively verify that empirical missingness is random (Molenberghs et al., 2008). Therefore, it can be argued that MNAR is the more likely mechanism for real-life missingness scenarios. It may be wise to include MNAR missingness in the simulation study just in case: A method that performs well under MAR and some cases of MNAR may be preferable over a method that only works under MAR, and not under MNAR. Such "late-phase" methodological investigations (see Heinze et al., 2022) can provide an understanding of an imputation method's robustness. Alternatively, a sensitivity analysis may provide an indication of the validity of the obtained inference, given that the assumed missingness mechanism is suspected to be invalid (see, e.g., Molenberghs et al., 2014, part 5).

In addition to varying the missingness mechanisms, each simulated mechanism should be combined with different missingness patterns. Remember that missingness is only ignorable under MAR when the parameter of the data is distinct and *a-priori* independent from the parameter of the missing data process (Little & Rubin, 2020, Corollary 6.1A). Under MAR missingness, we assume that we may use the observed data to make inferences about the joint (observed and unobserved) data. The dependency of the procedure on the assumption under which we obtain inference is only influenced by the amount of missingness. If there is no missingness—or if there is no data, for that matter—the inference does not depend on the assumption. Alternatively, the validity of assumptions becomes increasingly important when the missingness increases. Since we control the MAR mechanism, the assumption under which we may solve the missing data problem should hold and it is only fair to assess performance under stringent missingness conditions. We therefore propose to evaluate imputation methodology under several missingness proportions to emulate a realistic range in sever-

ity of the missingness problem. Depending on how the missingness mechanism interacts with the simulated data, higher missingness proportions may yield biased or invalid completed data inferences. The missingness proportions should thus be considered carefully.

The above emphasizes the need for both thorough evaluation and thorough documentation of the intended and used missingness mechanisms. Simulators must make clear which mechanisms are of interest. To be able to verify these mechanisms, simulators must also be transparent about the process that generated the missingness. For example, by sharing the code or detailing the process that generated the missing values. Alternatively, missingness generation devices, such as the `mice::ampute()` function, can be used to generate valid, pattern-based missingness in any structured data set (Schouten et al., 2018; Van Buuren & Groothuis-Oudshoorn, 2011). Finally, simulators must include simulation conditions and evaluation parameters that make it possible to identify the proper generation of missing values. This would prevent that errors in the missingness generation would go unnoticed.

2.4 | Performance evaluation

The evaluation criteria used to assess imputation performance vary from one simulator to another. This is not surprising as people from different fields could have a different focus on the problem at hand. There are, however, some overarching issues when assessing imputation method performance. In the first place, there are pitfalls in the evaluation of the estimates that are obtained by fitting the analysis model after imputation. For example, focusing on one performance measure over another may yield different conclusions about imputation efficacy. In the second place, diagnostic evaluation of the generated imputations is often left out of simulation study results. Identifying problems with the imputation-generating process (e.g., an iterative imputation algorithm) may offer explanations for underperformance of imputation methods. Such evaluations, however, are typically omitted and valuable insights into the imputation method(s) may then be overlooked.

At minimum, imputation method performance should be quantified using appropriate measures. It depends on the specifics of each simulation study which performance measures would be most suitable. If the goal is inference—not prediction—the uncertainty about estimates needs to be properly quantified. To capture this uncertainty, the standard errors of the estimates should be correctly calculated, which requires multiple imputation. The aim of multiple imputation is not to reproduce the data, but to allow for obtaining valid inference given that the data are incomplete. This means that, given the framework provided by Rubin (1987), and for any $(Q - \bar{Q})$, statistical properties such as bias, confidence intervals, and the coverage rate of the confidence intervals should always be studied. We therefore recommend evaluating the following points:

(i) The methods should preferably be unbiased. Note that the way bias is calculated should be carefully chosen and described, because this can greatly influence the interpretation of the results. A negligible absolute bias, for example, may already yield an almost infinite relative bias if the true value of the estimand is zero. In most cases, unbiased estimation may be expected under an MCAR missingness mechanism, which can be easily verified by including MCAR missingness in the scope of missingness investigations.

(ii) The intervals around estimates should have a valid coverage of the population (i.e., true) value. Coverage of a 95% interval should in theory be $\geq$ 95%, where a coverage rate of 95% would be most efficient (Neyman, 1934, p. 589). Undercoverage indicates that the procedure is not confidence valid and may lead to invalid inference. Undercoverage may occur when the estimation procedure is either biased, too liberal leading to intervals that are too narrow, or both. Overcoverage, though technically not confidence invalid, would indicate that efficiency could still be gained. In such cases, a narrower interval exists for which the method would still yield confidence valid result.

(iii) The width of the confidence or credible interval may convey statistical efficiency, which should be considered to compare imputation methods. Wider intervals are associated with more uncertainty, whereas a more narrow interval that is still properly covered indicates a sharper inference. However, inference from a wider interval that is properly covered is to be considered more valid than a more narrow interval that is not properly covered anymore. That said, with valid nominal coverage, the method that yields the narrowest interval would be most efficient.

(iv) Resemblance to the true data values may be quantified using the root mean squared error (RMSE). We do not generally recommend the use of the RMSE as evaluation criterion, because this metric does not account for the inherent uncertainty of the missing values and it may inflate the type I error rate of statistical inferences (Chapter 2.6 Van Buuren, 2018). However, if a study is aimed at obtaining predictions, and inferential validity is not of interest, the

RMSE of the prediction errors (residuals) can be compared between methods. Then, the RMSE may yield valuable information about the methods' comparative efficiency in terms of accuracy and precision.

After establishing the statistical validity of an imputation routine on the level of the estimand, it remains paramount to also study the imputation-generating process. Omitting such investigations may yield suboptimal results. One simple example arises often in practice: generating implausible imputed values. Even though the estimation on the analysis level may be justified, some methods can yield imputations that may seem completely invalid to applied researchers. For example, one could very accurately estimate average human height by filling in negative values and values that are unrealistically large. While the obtained inference could still be valid under such imputations, the plausibility of the imputed values given the observed data should be under scrutiny. Under many circumstances, imputation methods may be realistically expected to preserve both marginal and conditional distributions with respect to the comparative truth. Imputation methods that fail to do so should not be considered general-purpose methods. Many techniques can yield valid inferences, but techniques that sample realistic or plausible values may be preferable in practice.

The evaluation of simulated results goes far beyond plausibility and distributional characteristics. The quality and validity of an imputation task will also rely on the relation between the missing data models, imputation models, and analysis models. When a joint model exists whose conditionals would include the imputation and analysis model, then the imputation model and analysis are said to be congenial (Bartlett et al., 2015; Meng, 1994). The congeniality of all imputation models should be assessed, but methods to assess the suitability of imputation models and diagnose misfit often rely on visual inspection of the imputations (see, e.g., Abayomi et al., 2008; Bondarenko & Raghunathan, 2016). This makes its assessment a challenging and time-consuming endeavor in many simulation studies.

When algorithms are used, the simulator should study the validity of the algorithmic process. Many contemporary imputation techniques rely on iterative algorithms, such as the Gibbs sampler, to generate imputations. As with any iterative algorithm—but especially with imputation algorithms that are critically considered to be possibly incompatible Gibbs samplers (PIGS, Li et al., 2012)—algorithmic convergence should be carefully evaluated. Oberman et al. (2021) demonstrate that statistical validity can happen before algorithmic convergence is reached, but this is not a guarantee. Unfortunately, there is no universal quantitative method to diagnose nonconvergence in iterative imputation algorithms (Oberman et al., 2021; Zhu & Raghunathan, 2015) and the alternative (visual inspection of the imputation algorithm; Van Buuren, 2018) is neither efficient nor fail-proof. As a result, imputation algorithms may be terminated before reaching a stable state, which could yield suboptimal imputations and underestimated performance of the method. Problems with producing stable imputations are not exclusive to iterative imputation algorithms. Any method may run into failures of the imputation-generating process and subsequently lack results, for example, due to overparameterization errors.

Every imputation workflow should therefore contain an evaluation of the obtained imputations. Even though inspecting each imputation may be labor-intensive due to the number of imputations generated in a simulation study, we highly recommend simulators consider the following aspects:

(i) The absence of nonconvergence in the imputation-generating process is a minimum requirement for any imputation method. If nonconvergence is suspected, the inference resulting from the imputations might be invalid. However, preliminary work suggests that iterative imputation algorithms could achieve inferential validity before reaching a stable state (Oberman et al., 2021).

(ii) The fit of the imputation model may be verified with the help of a posterior predictive check (Nguyen et al., 2017; Zhao, 2022). A straightforward posterior predictive check for imputation methodology is the multiple overimputation of observed data values (Cai et al., 2022). If the statistical properties of the overimputed values are equivalent to those of the observed data values, one could infer that the imputation model fits the observed part of the incomplete data reasonably well. Then, by extension, one could assume that the imputation model might be able to produce good imputations for the missing part of the data too. This overimputation procedure is straightforward with the `overimpute()` function in `Amelia II` (Honaker et al., 2011) and the `where` argument in `mice` (Cai et al., 2022; Van Buuren & Groothuis-Oudshoorn, 2011).

(iii) The distributional characteristics of the imputations should be inspected for anomalies. The distribution of the incomplete data may differ greatly from the observed data. Under anything but the MCAR assumption, this can be expected. When evaluating imputations, the distributional shapes should be checked and diagnostic evaluations should be performed (see Abayomi et al., 2008, for a detailed overview of diagnostic evaluation for multivariate imputations). When anomalies are found, and if the imputation method is valid, there should be an explanation, especially in the controlled environment of a properly executed simulation study.

(iv) Finally, the plausibility of the imputed values may be evaluated. Plausible imputations–imputations that could be real values if they had been observed—are not a necessary condition for obtaining valid inference. However, in practice, especially when the imputer and the analyst are different persons, plausible imputations may be a desired property. One would prefer an imputation technique to yield both valid inference and plausible imputations. It should be studied if an imputation method is prone to deliver such impractical results, and if so, under what conditions. When evaluating imputation routines, the evaluator should mention whether the routine is prone to deliver implausible values. We must note, however, that the aim of imputation should be valid inference in the first place, and plausibility in the second place.

There are parallels between evaluating the performance and evaluating the missingness. Just like using MCAR as a condition to benchmark the validity of the missing data solution, we can use the simulation's complete data (i.e., without missing values) as an upper limit for our performance evaluation. In addition, the simulator should also perform complete case analysis, that is, analysis on the incomplete data without solving for the missingness. We know the theoretical properties of complete case analysis, which makes the ad hoc technique useful as a lower limit for evaluating imputation performance. Complete case analysis may therefore serve as a benchmark method against which imputation performance should be evaluated. Realistically, the simulator would expect the imputation solution to perform not worse than the complete case analysis and, preferably, mimic the complete data results. Moreover, pairing complete case analysis with MCAR and MAR mechanisms allows the simulator to evaluate the impact of the missing data generation process under the chosen analysis model and to identify the unintentional generation of accidental MNAR.

Last, when the simulator is on the verge of drawing conclusions about the performance of the imputation methodology, the performance should be carefully qualified. Comparing the performance of an imputation routine, given a population (or true) parameter allows for quantitative evaluation. Yet, in order to pose qualitative statements about the performance on simulated conditions, comparative methodology is required. For example, when claiming that imputation performance is unacceptable when deviations from normality become rather stringent, such performance is highly dependent on the simulation conditions that are used. For a well-balanced judgment about the severity of the performance drop, comparative simulations with, for example, nonparametric models should be executed. A method may perform badly, but if it still outperforms every other approach, it may yet be of great practical relevance.

3 | SUGGESTED COURSE OF ACTION

We encourage simulators to carefully consider and document their choices in the evaluation of imputation methodology. Simulation studies should not only be well executed, but also properly reported. For example, the inconsistent display of simulation conditions may impact the objectivity of metaevaluations over imputation methods, as one method's performance may appear to be favorable because of less stringent simulation conditions. This may ultimately lead to statisticians recommending a less efficient method to applied researchers, thereby limiting the efficiency of the imputation approach and unnecessarily lowering the statistical power. Simulators should therefore be explicit in the descriptions of their evaluations. However, deciding what and how to report may be challenging. The simulation design could be presented textually, in a flowchart or as a block of pseudocode, whereas missingness mechanisms could be written as a function of the data or displayed graphically. Ideally, the evaluations should be supplemented by an online repository with all of the data and code required to reproduce the simulation results. To aid simulators in reporting and move toward standardization in evaluation, we provide a draft version for reporting guidelines in Appendix A.1 (also available from www.gerkovink.com/evaluation). We invite the readers of this paper to contribute to its development.

We aim to elicit critical thinking about incomplete data simulation and to establish a common ground for the evaluation of imputation routines. Such a common ground would be the basis of a standardized evaluation. This would allow for fairer and more efficient comparisons between imputation techniques. Ultimately, it would be desirable to evaluate every imputation routine against the same standardized set in order to quantify the statistical properties across imputation routines. If properly executed, such evaluations would allow for careful matching of imputation methodologies to new missing data problems.

ACKNOWLEDGMENTS

We thank the Amices team for the fruitful discussions and highly value the comments and suggestions from the reviewers and the editors.

CONFLICT OF INTEREST STATEMENT

The authors have declared no conflict of interest.

DATA AVAILABILITY STATEMENT

This review article only contains simulated data, which is openly available from https://github.com/gerkovink/evaluation

OPEN RESEARCH BADGES

This article has earned an Open Data badge for making publicly available the digitally-shareable data necessary to reproduce the reported results. The data is available in the Supporting Information section.

This article has earned an open data badge "**Reproducible Research**" for making publicly available the code necessary to reproduce the reported results. The results reported in this article could fully be reproduced.

ORCID

Hanne I. Oberman https://orcid.org/0000-0003-3276-2141
Gerko Vink https://orcid.org/0000-0001-9767-1924

REFERENCES

Abayomi, K., Gelman, A., & Levy, M. (2008). Diagnostics for multivariate imputations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, *57*(3), 273–291.

Barnard, J., & Rubin, D. B. (1999). Miscellanea. small-sample degrees of freedom with multiple imputation. *Biometrika*, *86*(4), 948–955.

Bartlett, J. W., Seaman, S. R., White, I. R., Carpenter, J. R., & Initiative*, A. D. N. (2015). Multiple imputation of covariates by fully conditional specification: Accommodating the substantive model. *Statistical Methods in Medical Research*, *24*(4), 462–487.

Bondarenko, I., & Raghunathan, T. (2016). Graphical and numerical diagnostic tools to assess suitability of multiple imputations and imputation models. *Statistics in Medicine*, *35*(17), 3007–3020.

Cai, M., Van Buuren, S., & Vink, G. (2022). Graphical and numerical diagnostic tools to assess multiple imputation models by posterior predictive checking. *arXiv preprint. arXiv:2208.12929*. https://arxiv.org/abs/2208.12929

Doretti, M., Geneletti, S., & Stanghellini, E. (2018). Missing data: A unified taxonomy guided by conditional independence. *International Statistical Review*, *86*(2), 189–204.

Greenland, S. (2017). Invited commentary: The need for cognitive science in methodology. *American Journal of Epidemiology*, *186*(6), 639–645.

Heinze, G., Boulesteix, A.-L., Kammer, M., Morris, T. P., & White, I. R. (2022). Phases of methodological research in biostatistics - Building the evidence base for new methods. https://arxiv.org/abs/2209.13358

Honaker, J., King, G., & Blackwell, M. (2011). Amelia II: A program for missing data. *Journal of Statistical Software*, *45*(7), 1–47. https://www.jstatsoft.org/v45/i07/

Li, F., Yu, Y., & Rubin, D. B. (2012). *Imputing missing data by fully conditional models: Some cautionary examples and guidelines*. Duke University Department of Statistical Science Discussion Paper, *1124*.

Li, H., Rosete, S., Coyle, J., Phillips, R. V., Hejazi, N. S., Malenica, I., Arnold, B. F., Benjamin-Chung, J., Mertens, A., & Colford Jr, J. M. (2022). Evaluating the robustness of targeted maximum likelihood estimators via realistic simulations in nutrition intervention trials. *Statistics in Medicine*, *41*(12), 2132–2165.

Little, & Rubin (2020). *Statistical analysis with missing data* (3rd ed.). Wiley Series in Probability and Statistics. Wiley.

Little, R. J. (1988). Missing-data adjustments in large surveys. *Journal of Business & Economic Statistics*, *6*(3), 287–296.

Little, R. J., D'Agostino, R., Cohen, M. L., Dickersin, K., Emerson, S. S., Farrar, J. T., Frangakis, C., Hogan, J. W., Molenberghs, G., Murphy, S. A., Neaton, J. D., Rotnitzky, A., Scharfstein, D., Shih, W. J., Siegel, J. P., & Stern, H. (2012). The prevention and treatment of missing data in clinical trials. *New England Journal of Medicine*, *367*(14), 1355–1360.

Liu, D., Oberman, H. I., Muñoz, J., Hoogland, J., & Debray, T. P. A. (2021). Quality control, data cleaning, imputation. *arXiv preprint arXiv:2110.15877*.

Mealli, F., & Rubin, D. B. (2015). Clarifying missing at random and related definitions, and implications when coupled with exchangeability. *Biometrika*, *102*(4), 995–1000.

Meng, X.-L. (1994). Multiple-imputation inferences with uncongenial sources of input. *Statistical Science*, *9*(4), 538–558.

Mohan, K., & Pearl, J. (2021). Graphical models for processing missing data. *Journal of the American Statistical Association*, *116*(534), 1023–1037.

Molenberghs, G., Beunckens, C., Sotto, C., & Kenward, M. G. (2008). Every missingness not at random model has a missingness at random counterpart with equal fit. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, *70*(2), 371–388.

Molenberghs, G., Fitzmaurice, G., Kenward, M. G., Tsiatis, A., & Verbeke, G. (2014). *Handbook of missing data methodology*. CRC Press.

Moreno-Betancur, M., Lee, K. J., Leacy, F. P., White, I. R., Simpson, J. A., & Carlin, J. B. (2018). Canonical causal diagrams to guide the treatment of missing data in epidemiologic studies. *American Journal of Epidemiology*, *187*(12), 2705–2715.

Morris, T. P., White, I. R., & Crowther, M. J. (2019). Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, *38*(11), 2074–2102.

Neyman, J. (1934). On the two different aspects of the representative method: the method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, *97*(4), 558–625.

Nguyen, C. D., Carlin, J. B., & Lee, K. J. (2017). Model checking in multiple imputation: An overview and case study. *Emerging Themes in Epidemiology*, *14*(1), 8.

Nijman, S. W. J., Hoogland, J., Groenhof, T. K. J., Brandjes, M., Jacobs, J. J. L., Bots, M. L., Asselbergs, F. W., Moons, K. G. M., & Debray, T. P. A. (2020). Real-time imputation of missing predictor values in clinical practice. *European Heart Journal - Digital Health*, *2*, 154.

Oberman, H. I., van Buuren, S., & Vink, G. (2021). Missing the point: Non-convergence in iterative imputation algorithms. *arXiv preprint arXiv:2110.11951*.

Pawel, S., Kook, L., & Reeve, K. (2022). Pitfalls and potentials in simulation studies. *arXiv preprint arXiv:2203.13076*.

Petersen, M. L., & Van der Laan, M. J. (2014). Causal models and learning from data: Integrating causal modeling and statistical estimation. *Epidemiology*, *25*(3), 418–426.

Raghunathan, T. E., Reiter, J. P., & Rubin, D. B. (2003). Multiple imputation for statistical disclosure limitation. *Journal of Official Statistics*, *19*(1), 1.

Reiter, J. P. (2003). Inference for partially synthetic, public use microdata sets. *Survey Methodology*, *29*(2), 181–188.

Robins, J. M., & Gill, R. D. (1997). Non-response models for the analysis of non-monotone ignorable missing data. *Statistics in Medicine*, *16*(1), 39–56.

Rubin, D. B. (1976). Inference and missing data. *Biometrika*, *63*(3), 581–592.

Rubin, D. B. (1986). Statistical matching using file concatenation with adjusted weights and multiple imputations. *Journal of Business & Economic Statistics*, *4*(1), 87–94.

Rubin, D. B. (1987). *Multiple imputation for nonresponse in surveys*. Wiley Series in Probability and Mathematical Statistics Applied Probability and Statistics. Wiley.

Schomaker, M. (2021). Regression and causality. *arXiv preprint arXiv:2006.11754*.

Schouten, R. M., Lugtig, P., & Vink, G. (2018). Generating missing values for simulation purposes: A multivariate amputation procedure. *Journal of Statistical Computation and Simulation*, *88*(15), 2909–2930.

Schouten, R. M., & Vink, G. (2021). The dance of the mechanisms: How observed information influences the validity of missingness assumptions. *Sociological Methods & Research*, *50*(3), 1243–1258.

Seaman, S., Galati, J., Jackson, D., & Carlin, J. (2013). What is meant by "missing at random"? *Statistical Science*, *28*(2), 257–268.

Shara, N., Yassin, S. A., Valaitis, E., Wang, H., Howard, B. V., Wang, W., Lee, E. T., & Umans, J. G. (2015). Randomly and non-randomly missing renal function data in the strong heart study: A comparison of imputation methods. *PLoS One*, *10*(9), e0138923.

Sperrin, M., Martin, G. P., Sisk, R., & Peek, N. (2020). Missing data should be handled differently for prediction than for description or causal explanation. *Journal of Clinical Epidemiology*, *125*, 183–187.

Van Buuren, S. (2018). *Flexible imputation of missing data*. Chapman and Hall/CRC.

Van Buuren, S., & Groothuis-Oudshoorn, K. (2011). Mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, *45*(1), 1–67.

Vink, G. (2022). Strategies for simulating missingness. https://www.gerkovink.com/simulate. *GitHub repository*. https://doi.org/10.5281/zenodo.7467995

Vink, G., & Van Buuren, S. (2014). Pooling multiple imputations when the sample happens to be the population. *arXiv preprint arXiv:1409.8542*.

Volker, T. B., & Vink, G. (2021). Anonymiced shareable data: Using mice to create and analyze multiply imputed synthetic datasets. *Psych*, *3*(4), 703–716.

Wood, A. M., Royston, P., & White, I. R. (2015). The estimation and use of predictions for the assessment of model performance using large samples with multiply imputed data. *Biometrical Journal Biometrische Zeitschrift*, *57*(4), 614–632.

Zhao, Y. (2022). Diagnostic checking of multiple imputation models. *AStA advances in statistical analysis*, *106*(2), 271–286.

Zhu, J., & Raghunathan, T. E. (2015). Convergence properties of a sequential regression multiple imputation algorithm. *Journal of the American Statistical Association*, *110*(511), 1112–1124.

SUPPORTING INFORMATION

Additional supporting information can be found online in the Supporting Information section at the end of this article.

How to cite this article: Oberman, H. I., & Vink, G. (2024). Toward a standardized evaluation of imputation methodology. *Biometrical Journal*, *66,* 2200107. https://doi.org/10.1002/bimj.202200107

APPENDIX

A.1 | Reporting guidelines

Table A1 provides a checklist for reporting on imputation methodology evaluations. We have structured the checklist according to simulation study reporting format proposed by Morris et al. (2019).

TABLE A1 Checklist for reporting on imputation methodology evaluations.

☐	**Aims**
	e.g., simulation scope
	☐ simulation design (incl. pseudocode or flow diagram)
	☐ required level of precision (incl. number of simulation repetitions)
☐	**Data-generating mechanisms**
	e.g., data generation and missingness generation
	☐ data source (incl. model-based or design-based, sampling variance)
	☐ data characteristics (incl. multivariate relations and data structures such as clustering)
	☐ missingness mechanisms (incl. type or functional form of the missing data model)
	☐ missingness patterns (incl. missingness proportion)
☐	**Estimands**
	e.g., complete data target
☐	**Methods**
	e.g., missing data methods, analytic method and inference pooling rules
	☐ imputation methods (incl. parameters such as the number of imputations)
	☐ estimation method (incl. reference methods such as complete case analysis)
	☐ methods used to construct standard errors and confidence intervals
	– Such as rules by Barnard and Rubin (1999); Rubin (1987); Reiter (2003), etc.
☐	**Performance measures**
	e.g., evaluation of estimates and imputed values
	☐ statistical properties (incl. comparative performance, if applicable)
	☐ validity of imputations (incl. imputation model fit and distributional characteristics)
	– Does the imputation model fit well to the observed values? (e.g., posterior predictive check)
	– Are there unrealistic values in the data? (e.g., negative where positive expected)
	– Are there implausible combinations in the data? (e.g., pregnant grandfathers)

3 Chapter 3: convergence

Pre-printed as Oberman et al. (2021).

[TODO: update version, add credit statement, merge formatting]

Missing the point

Non-convergence in iterative imputation

H. I. Oberman

Iterative imputation has become the de facto standard to accommodate for the ubiquitous problem of missing data. While it is widely accepted that this technique can yield valid inferences, these inferences all rely on algorithmic convergence. Our study provides insight into identifying non-convergence in iterative imputation algorithms. We show that these algorithms can yield correct outcomes even when a converged state has not yet formally been reached. In the cases considered, inferential validity is achieved after five to ten iterations, much earlier than indicated by diagnostic methods. We conclude that it never hurts to iterate longer, but such calculations hardly bring added value.

Introduction

Multiple imputation has become the de facto standard to accommodate missing data in social scientific research. The technique enables researchers to draw valid inferences from incomplete data (Rubin 1976). Multiple imputation separates the missing data problem from the scientific problem. First, the missing values are imputed (i.e., filled in) using an imputation algorithm. Once the data are completed, the analysis of scientific interest can be performed on the completed data. This process is repeated several times to obtain a distribution of plausible values, which reflects the uncertainty in the data due to missingness. The analysis of scientific interest thus yields one estimate for each imputed dataset. Afterwards, the estimates from the different imputations are pooled into a single estimate. This pooled estimate is unbiased and confidence-valid if–and only if–both the missing data problem and the scientific problem are appropriately considered.

To obtain valid scientific estimates from incomplete data, both the missing data problem and the scientific problem should be appropriately considered. The popular imputation algorithm 'Multivariate Imputation by Chained Equations' (MICE) has proven to be a powerful tool to draw valid inferences from incomplete data under many circumstances (van Buuren and Groothuis-Oudshoorn 2011; van Buuren 2018). The algorithm is named after its iterative nature: missing data are imputed on a variable-by-variable basis, using a sequence of univariate imputation models. The validity of this iterative process naturally depends on the convergence

of the imputation algorithm. Yet, there has not been a systematic study on how to evaluate the convergence of iterative imputation algorithms.

Determining whether an algorithm has converged is not trivial, especially in the context of iterative imputation. There is no scientific consensus on how to evaluate the convergence of imputation algorithms (Zhu and Raghunathan 2015; Takahashi 2017). Moreover, the behavior of such algorithms under certain default imputation models (e.g., 'predictive mean matching') is an entirely open question (Murray 2018). Therefore, algorithmic convergence should be monitored carefully—although this is not straightforward. Since the aim of iterative imputation is to converge to a distribution and not to a single point, the algorithm produces some fluctuations even after it has converged. Because of this property, it may be more desirable to focus on *non*-convergence. A widely accepted practice is visual inspection of the algorithm (Raghunathan and Bondarenko 2007; Abayomi et al. 2008; van Buuren 2018). Diagnosing non-convergence through visual inspection may, however, be undesirable: 1) it may be challenging to the untrained eye, 2) only severely pathological cases of non-convergence may be diagnosed, and 3) there is not an objective measure that quantifies convergence (van Buuren 2018). Additionally, "inspection of such plots is a notoriously unreliable method of assessing convergence and in addition is unwieldy when monitoring a large number of quantities of interest, such as can arise in complicated hierarchical models" (Gelman et al. 2013, 2013, p. 285). Therefore, a quantitative diagnostic method to assess convergence would be preferred. Fortunately, there are non-convergence identifiers for *other* iterative algorithms, but the validity of these identifiers has not been evaluated on imputation algorithms.

In this paper we study different methods for assessing non-convergence in iterative imputation algorithms. In this study, we explore different methods for identifying non-convergence in the iterative imputation algorithm MICE. We evaluate whether these methods are able to cover the extent of the non-convergence, and we also investigate the relation between non-convergence and the validity of the inferences. We define several diagnostics and evaluate how these diagnostics could be appropriate for iterative imputation applications. We then address the impact of inducing non-convergence in iterative imputation algorithms through model-based simulation in `R` (R Core Team 2025). For reasons of brevity, we only focus on the iterative imputation algorithm implemented in the popular `mice` package in `R` (van Buuren and Groothuis-Oudshoorn 2011). The aim of the simulation study is to determine whether unbiased, confidence-valid inferences may be obtained if the algorithm has not (yet) converged. And additionally, to evaluate the behavior and performance of several diagnostic methods to identify non-convergence. With that, we formulate an informed advice on when it is safe to conclude that the algorithm is converged *enough* for valid inferences. We translate the results of the study into guidelines for applied researchers, which may facilitate drawing valid inferences from incomplete data.

Background

Notation

Let $\mathbf{Y}$ denote an $n \times p$ matrix containing the data values on p variables for all n units in a sample. The data value of unit i ($i = 1, 2, ..., n$) on variable Y_j ($j = 1, 2, ..., p$) may be either observed or missing. The collection of observed data values in $\mathbf{Y}$ is denoted by $\mathbf{Y}_{\text{obs}}$; the missing part of $\mathbf{Y}$ is referred to as $\mathbf{Y}_{\text{mis}}$. We define the proportion of incomplete cases, π_{inc}, as the number of units with at least one missing value divided by the total number of units n in dataset $\mathbf{Y}$.

Figure Figure 1 provides an overview of the steps involved with MI. The missing part $\mathbf{Y}_{\text{mis}}$ of an incomplete dataset is imputed m times. This creates m sets of imputed data $\dot{\mathbf{Y}}_{\text{imp},\ell}$, where $\ell = 1, 2, ..., m$. The imputed data is then combined with the observed data $\mathbf{Y}_{\text{obs}}$ to create m completed datasets. On each of these datasets, the analysis of scientific interest is performed to estimate Q: the quantity of scientific interest (e.g., a regression coefficient). Since Q is estimated on each completed dataset, m separate $\hat{Q}_\ell$-values are obtained. Finally, the $\hat{Q}_\ell$-values are combined into a single pooled estimate $\bar{Q}$. The premise of multiple imputation is that $\bar{Q}$ is an unbiased and confidence-valid estimate of the true—but unobserved—scientific estimand Q (Rubin 1996).

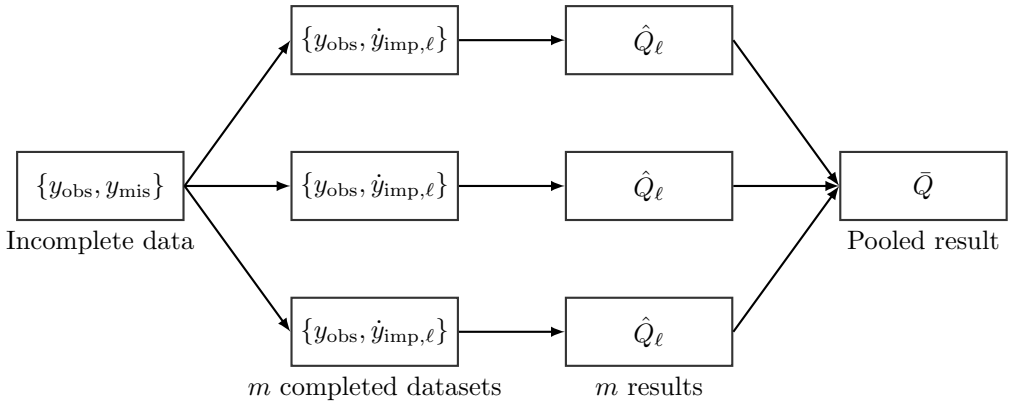


Figure 1: Scheme of the main steps in multiple imputation—from an incomplete dataset, to $m = 3$ multiply imputed datasets, to $m = 3$ estimated quantities of scientific interest $\hat{Q}$, to a single pooled estimate $\bar{Q}$.

A popular method to obtain imputations is to use the 'Multiple Imputation by Chained Equations' algorithm, shorthand 'MICE' (van Buuren and Groothuis-Oudshoorn 2011). With MICE, imputed values are drawn from the posterior predictive distribution of the missing values on a variable-by-variable basis. Algorithm 1 displays the iterative nature of the MICE algorithm.

Algorithm 1: MICE algorithm

Initialization phase ($t = 0$)

1. Specify an imputation model for each variable Y_j, where $j = 1, ..., p$.
2. Repeat over variables ($j = 1, ..., p$).
    2. Repeat over imputations ($\ell = 1, ..., m$).
        a. Fill in starting imputations ($\dot{Y}_j^0$) by random draws from the observed part of each variable (Y_j^{obs}).
3. End repeat j.

Iteration phase ($t = 1, ..., \tau$)

4. Repeat over iterations ($t = 1, ..., \tau$).
    a. Repeat over variables ($j = 1, ..., p$).
    I. Define $\dot{\mathbf{Y}}_{-j}^t$ as the currently complete data *except* variable Y_j at iteration t, where $\dot{\mathbf{Y}}_{-j}^t = (\dot{Y}_1^t, ..., \dot{Y}_{j-1}^t, \dot{Y}_{j+1}^{t-1}, ..., \dot{Y}_p^{t-1})$.
    II. Draw imputation model parameters $\dot{\phi}_j^t$ from the posterior predictive distribution of the observed data, where $\dot{\phi}_j^t \sim P(\phi_j^t | Y_j^{\text{obs}}, \dot{Y}_{-j}^t, R)$.
    III. Draw imputations $\dot{Y}_j^t$, where $\dot{Y}_j^t \sim P(Y_j^{\text{mis}} | Y_j^{\text{obs}}, \dot{Y}_{-j}^t, R, \dot{\phi}_j^t)$.
    b. End repeat j.
5. End repeat t.

The algorithm is initialized by filling in m sets of starting values, randomly drawn from the observed data. The algorithm then iterates over the p variables, generating m sets of imputed values for each variable j. After one full run through the data, the imputed values are 'updated' in the next iteration. This process should be repeated until convergence is reached.

With iterative imputation, plausible values are sampled at every iteration t of the imputation algorithm. But only the imputed values drawn at the final iteration τ (where $t = 1, 2, ..., \tau$) are used for further analysis. The state-space of the algorithm at a certain iteration t may be summarized by a scalar summary θ. A typical θ to track over iterations is the 'chain mean': the average of the imputed values for a variable Y_j at iteration t. The collection of θ-values between $t = 1$ (the state-space of the algorithm after initialization) and $t = \tau$ (the state-space for the imputed values) will be referred to as an 'imputation chain'.

Convergence of MICE

Iterative imputation algorithms such as MICE often are special cases of Markov chain Monte Carlo (MCMC) methods. In MCMC methods, convergence is not from a scalar to a point, but from one distribution to another. The values generated by the algorithm will vary even after convergence (Gelman et al. 2013). The most converged state that such an algorithm may reach is *approximate* convergence. Since MCMC algorithms do not reach a unique point at which convergence is established, diagnostic methods may only identify signs of *non*-convergence (Hoff 2009).

Currently, the recommended practice for evaluating the convergence of the MICE algorithm is through visual inspection (Raghunathan and Bondarenko 2007; van Buuren 2018). After running the imputation algorithm for a certain number of iterations, researchers are encouraged to produce traceplots. In a traceplot, a scalar summary of the state-space of the algorithm θ is plotted against the iteration number. Non-convergence is diagnosed if the chains show definite trends, or if the imputation chains are not freely intermingled with one another (van Buuren 2018, § 6.5.2). These guidelines are in line with the two requirements for convergence in MCMC algorithms: stationarity and mixing (Gelman et al. 2013). Stationarity is characterized by the absence of trending across iterations. Mixing implies that the chains intermingle nicely. If one of the two requirements is not met, we speak of non-convergence. Without mixing, imputation chains may be 'stuck' at a local optimum instead of sampling imputed values from the entire predictive posterior distribution of the missing values. The distribution of imputed values then differs across imputations. This may cause under-estimation of the variance between chains, which results in spurious, invalid inferences. Without stationarity, there is trending within imputation chains. Trending implies that further iterations would yield a systematically lower or higher set of imputations. Iterative imputation algorithms that have not (yet) reached stationarity, may thus yield biased estimates.

Non-convergence diagnostics

There are many diagnostic tools to identify non-convergence in iterative (MCMC) algorithms (Brooks and and Gelman 1998; El Adlouni et al. 2006). Non-convergence is diagnosed using an identifier and a parameter. The parameter can be any statistic that we track across iterations, for example the average imputed value per imputation (i.e., chain means). The identifier is a calculation of some sort that quantifies non-convergence in the scalar. We consider only two of them that may be appropriate for imputation algorithms—one to monitor signs of non-mixing, and one for non-stationarity. As recommended by e.g. Cowles and Carlin (1996) we will use the potential scale reduction factor $\widehat{R}$ to evaluate mixing ('Gelman-Rubin statistic,' Gelman and Rubin 1992), and autocorrelation to diagnose trending (*AC*, Schafer 1997; Gelman et al. 2013). With a recently proposed adaptation, $\widehat{R}$ may also serve to diagnose non-stationarity (Vehtari et al. 2021). We will evaluate the appropriateness of $\widehat{R}$ and *AC*. Other methods are outside the scope of this study, e.g. because they assume that values within chains represent

independent samples, whereas the MICE algorithm only uses the final iteration to produce imputations.

Potential scale reduction factor

In 2021, Vehtari et al. published an updated version of the potential scale reduction factor $\widehat{R}$, originally coined by Gelman and Rubin in 1992. This adapted version aims to detect non-mixing even in the tails of distributions, and non-stationarity over iterationsuses. To achieve this, the 2021 version uses three transformations on the scalar summary θ before computing $\widehat{R}$-values: rank-normalization, folding, and localization. To define $\widehat{R}$, we largely follow Vehtari et al.'s formulation (2019, p. 5), with a few exceptions. Let m be the total number of chains, τ the number of iterations per chain (where $\tau \geq 2$), and θ the scalar summary of interest. For each chain ($\ell = 1, 2, \ldots, m$), we estimate the variance of θ, and average these to obtain within-chain variance W.

$$W = \frac{1}{m} \sum_{\ell=1}^{m} s_\ell^2, \text{ where } s_\ell^2 = \frac{1}{\tau - 1} \sum_{t=1}^{\tau} \left(\theta^{(t\ell)} - \bar{\theta}^{(\cdot\ell)}\right)^2.$$

We then estimate between-chain variance B (note that we diverge from the typical notation in MI, where B denotes the variance between the estimated quantities of scientific interest $\hat{Q}_\ell$ cf. Rubin). B is defined as the variance of the collection of average θs per chain:

$$B = \frac{\tau}{m - 1} \sum_{\ell=1}^{m} \left(\bar{\theta}^{(\cdot\ell)} - \bar{\theta}^{(\cdot\cdot)}\right)^2, \text{ where } \bar{\theta}^{(\cdot\ell)} = \frac{1}{\tau} \sum_{t=1}^{\tau} \theta^{(t\ell)}, \; \bar{\theta}^{(\cdot\cdot)} = \frac{1}{m} \sum_{\ell=1}^{m} \bar{\theta}^{(\cdot\ell)}.$$

From the between- and within-chain variances we compute a weighted average, $\widehat{\text{var}}^+$, which over-estimates the total variance of θ. $\widehat{R}$ is then obtained as a ratio between this total variance and the within-chain variance:

$$\widehat{R} = \sqrt{\frac{\widehat{\text{var}}^+(\theta|y)}{W}}, \text{ where } \widehat{\text{var}}^+(\theta|y) = \frac{\tau - 1}{\tau} W + \frac{1}{\tau} B.$$

We can interpret $\widehat{R}$ as potential scale reduction factor since it indicates by how much the variance of θ could be shrunken down if an infinite number of iterations per chain would be run (Gelman and Rubin 1992). The assumption underlying this interpretation is that chains are 'over-dispersed' at $t = 1$, and reach convergence as $\tau \to \infty$. Over-dispersion implies that the initial values of the chains are 'far away' from the target distribution and each other. When the sampled values in each chain are independent of the chain's initial value, the mixing

component of convergence is satisfied. The variance between chains, B, is then equivalent to the variance within chains, W, and $\widehat{R}$-values will be close to one. High $\widehat{R}$-values thus indicate non-convergence.

Autocorrelation

Autocorrelation is defined as the correlation between two subsequent θ-values within the same chain (Lynch 2007, 147). In this study, we only consider AC at lag 1, i.e., the correlation between the t^{th} and $(t+1)^{th}$ iteration of the same chain. Following the same notation as for $\widehat{R}$,

$$AC = \Big(\frac{\tau}{\tau - 1}\Big) \frac{\sum_{t=1}^{\tau-1}(\theta_t - \bar{\theta}^{(\cdot m)})(\theta_{t+1} - \bar{\theta}^{(\cdot m)})}{\sum_{t=1}^{\tau}(\theta_t - \bar{\theta}^{(\cdot m)})^2}.$$

We can interpret AC-values as a measure of non-stationarity. If there is dependence between subsequent θ-values in imputation chains, AC-values are non-zero. Positive AC-values occur when θ-values are recurring (i.e., high θ-values are followed by high θ-values, and low θ-values are followed by low θ-values). Recurrence within imputation chains may lead to trending. Negative AC-values occur when θ-values of subsequent iterations are less similar, or diverge from one another. Divergence within imputation poses no threat to the convergence of the algorithm—it may even speed up convergence. Complete stationarity is reached when $AC = 0$. As non-convergence diagnostic, our interest is in positive AC-values.

Thresholds

Upon approximate convergence, the imputation chains intermingle such that the only difference between the chains is caused by the randomness induced by the algorithm ($\widehat{R} \approx 1$), and there is little dependency between subsequent iterations of imputation chains ($AC \approx 0$). In practice, we diagnose non-convergence when approximate convergence is violated, i.e., when $\widehat{R}$ and AC exceed a certain threshold. The conventional thresholds to diagnose non-mixing are $\widehat{R} > 1.2$ (Gelman and Rubin 1992) or $\widehat{R} > 1.1$ (Gelman et al. 2013). Vehtari et al. (2021) proposed a much more stringent threshold of $\widehat{R} > 1.01$. The magnitude of AC-values may be evaluated statistically, using a Wald test with $AC = 0$ as null hypothesis (Box et al. 2015). AC-values that are significantly higher than zero indicate non-stationarity.

Empirical example

As empirical example we use the incomplete `boys` dataset from the `mice` package (van Buuren and Groothuis-Oudshoorn 2011), which contains health-related data for 748 Dutch boys. Say,

we're interested in the relation between the boys heights and their respective ages, we could use a linear regression model to predict `hgt` from `age`. However, as Figure 2 shows, the variable `hgt` is not completely observed. To be able to analyze these data, we need to solve the missing data problem first.

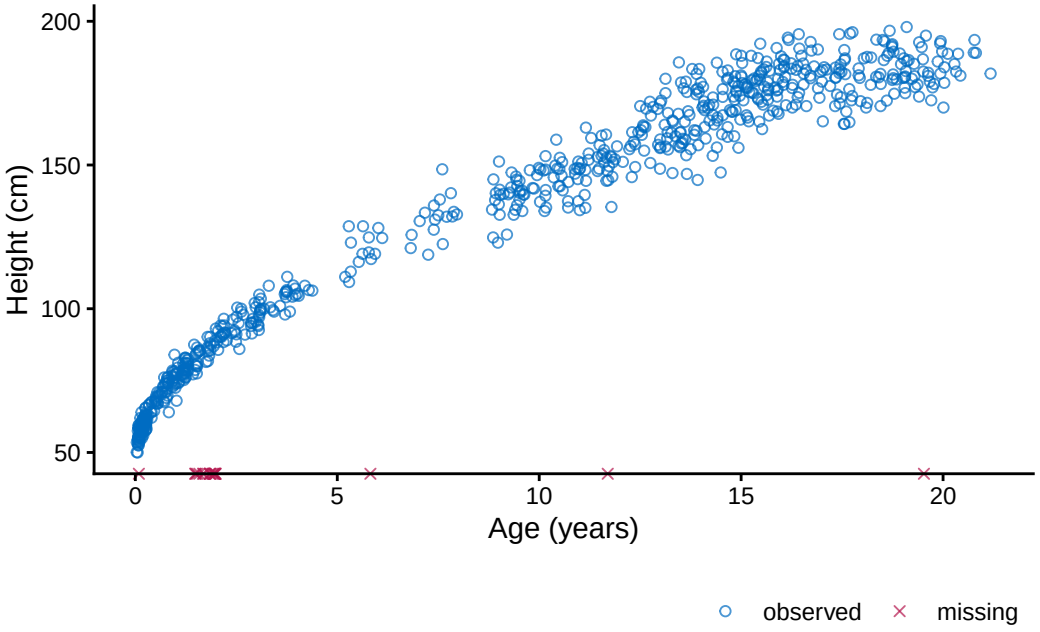


Figure 2

To illustrate what non-mixing and non-stationarity look like in iterative imputation algorithms, we reproduce an example from van Buuren (2018, § 6.5.2). The incomplete height variable can be imputed based on auxiliary variables, such as weight. A mis-specified imputation model can lead to non-convergence in the imputation algorithm. After imputation with `mice`, one would conventionally inspect the trace plots for signs of non-convergence. The figures show results of a `mice` run with two versions of the imputation models and 20 iterations, but otherwise default settings.

Figures 3 and 4 display two scenarios from the example. Figure 3 shows typical convergence of an iterative imputation algorithm. Figure 4 displays pathological non-convergence, induced by purposefully mis-specifying the imputation model. Each line portrays one imputation chain (i.e., the values of a scalar summary θ across iterations). The θ depicted here is the 'chain mean' of variable j, which is defined as the average of variable j in the m sets of imputations $\dot{\mathbf{Y}}_{\text{imp},\ell}$.

In the typical convergence scenario, the imputation chains intermingle nicely and there is little to no trending. From the traceplot of the chain means (Figure 3) it seems that mixing improves up-to 10 iterations, while trending is only apparent in the first three iterations.

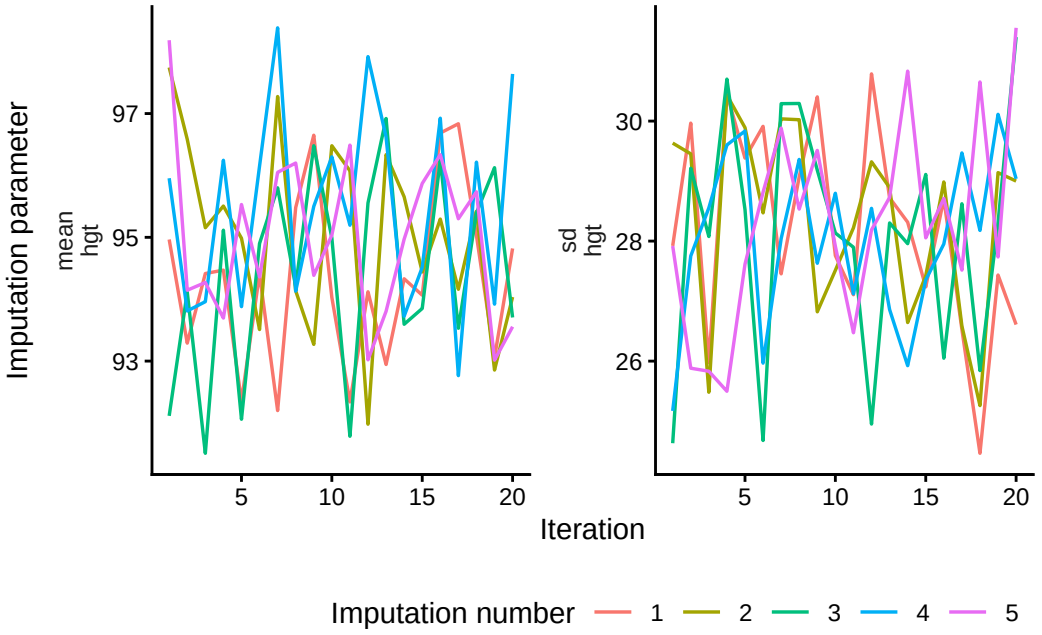


Figure 3: Traceplot with typical convergnce

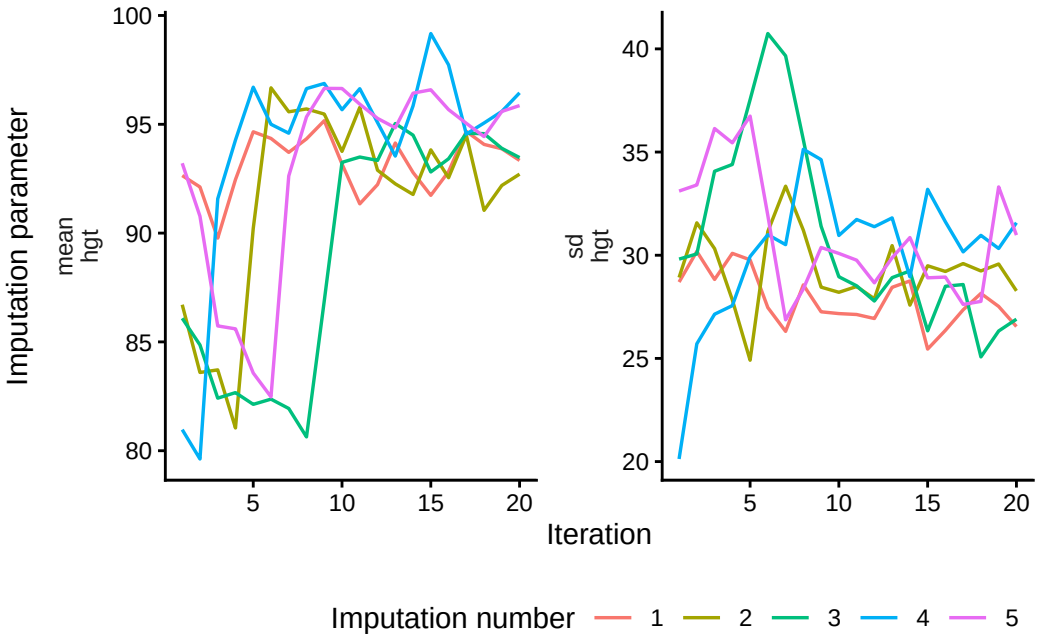


Figure 4: Traceplot with non-convergnce

In the non-convergence scenario, there is a lot of trending and some chains do not intermingle. Importantly, the chain means at the last iteration (the imputed values per imputation ℓ) are very different between the two scenarios. The algorithm with the mis-specified model yields imputed values that are on average a factor two larger than those of the typically converged algorithm. It is obvious that non-convergence in this example impacts the distribution of the imputed values per imputation $\dot{\mathbf{Y}}_{\text{imp},\ell}$. This effect may translate into the distribution of the m sets of completed data $\{\mathbf{Y}_{\text{obs}}, \dot{\mathbf{Y}}_{\text{imp},\ell}\}$, which consequently affects the estimated quantities of scientific interest $\hat{Q}_\ell$, and finally the pooled estimate $\bar{Q}$. $\bar{Q}$ is then a biased, invalid estimate of Q. Therefore, it is important to reach algorithmic convergence in iterative imputation algorithms.

Simulation set-up

We investigate non-convergence in iterative imputation through model-based simulation in R (version 4.5.0, R Core Team (2025)). We provide a summary of the simulation set-up in Algorithm 2. The complete script and technical details are available from github.com/hanneoberman/convergence. The number of simulation repetitions $n_{\text{sim}} = 2000$. Other simulation design aspects are described under the subsequent sections: aims, data-generating mechanism, estimands, methods, and performance measures (cf. Morris et al. 2019).

Algorithm 2: simulation set-up (pseudo-code)

```
for (each simulation run) {
  simulate complete data;
  for (each missingness condition) {
    create missingness;
    for (each imputation model iteration) {
      impute missingness;
      estimate quantities of scientific interest;
      apply performance measures to the estimates;
      compute non-convergence diagnostics
} } }
```

Aims

With this simulation, we assess the impact of non-convergence on the validity of scientific estimates obtained using the imputation package `{mice}` (van Buuren and Groothuis-Oudshoorn 2011). Inferential validity is reached when estimates are both unbiased and have nominal coverage across simulation repetitions ($n_{\text{sim}} = 2000$). To evaluate convergence, we terminate the imputation algorithm after a varying number of iterations ($n_{\text{it}} = 1, 2, ..., 100$, corresponding

Table 1: Missingness scenarios

Missingness conditions		
missing data mechanism		proportion of incomplete cases
	×	
MCAR, MAR		25%, 50%, 75%

to τ in each individual imputation run). We differentiate between six different missingness scenarios that are defined in the data generating mechanism (see Table 1).

Data generating mechanism

In each simulation repetition, we first generate a complete set of $n_{\text{obs}} = 200$ cases, representing person-data in a multiple linear regression problem. The predictor space consists of three multivariately normal random variables,

$$\begin{pmatrix} X_1 \\ X_2 \\ X_3 \end{pmatrix} \sim \mathcal{N} \left[\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & & \\ 0.5 & 1 & \\ 0.5 & 0.5 & 1 \end{pmatrix} \right].$$

The outcome variable Y is a linear combination of the three predictors, such that for each unit $i = 1, 2, ..., n_{\text{obs}}$,

$$Y_i = X_{1i} + X_{2i} + X_{3i} + \epsilon_i,$$

where $\epsilon \sim \mathcal{N}(0, 1)$. This results in a complete dataset of size $n_{\text{obs}} \times p$, with units $i = 1, 2, ..., n_{\text{obs}}$ and variables $j = Y, X_1, X_2, X_3$). Multivariate normal data are generated using the `mvtnorm` package (Genz and Bretz 2009).

Subsequently, the complete are 'amputed' (i.e., made incomplete) according to six missingness conditions. We use a 2×3 factorial design with two missing data mechanisms and three proportions of incomplete cases, see Table 1. We consider all possible multivariate patterns of missingness, as visualized in Figure 5.

Missing data mechanism

Missingness mechanisms refer to the probability of being missing for any given entry in a dataset. There are three distinct types of mechanisms, as defined by Rubin (1976). Roughly translated, the probability of being missing may be equal for all entries (MCAR; Missing Completely At Random), may depend on observed information (MAR; Missing At Random), or may depend on *un*observed information, making the missingness non-ignorable (MNAR; Missing Not At Random).

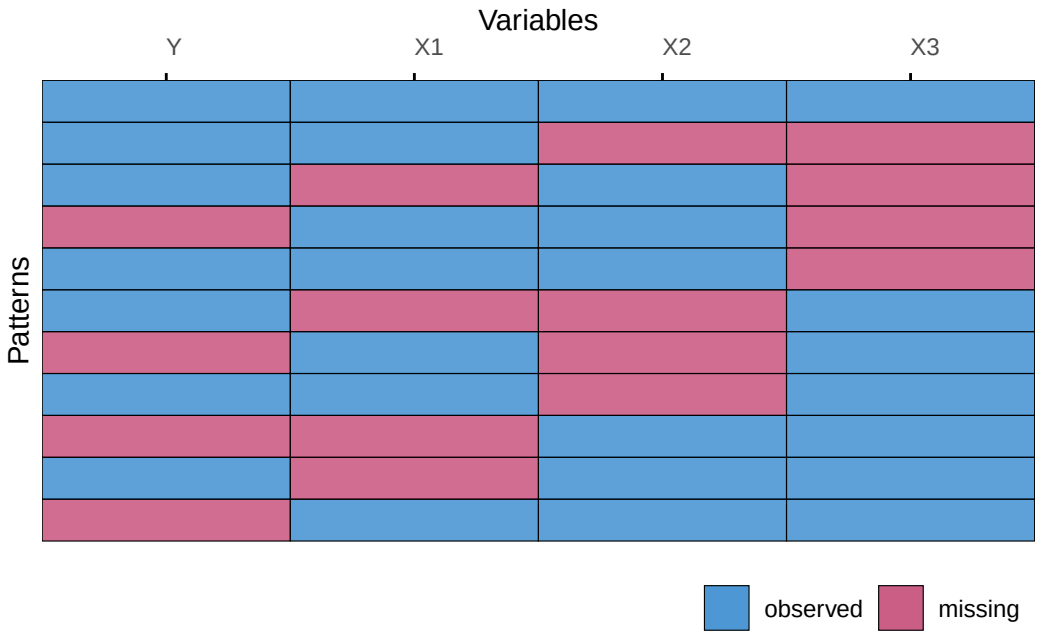


Figure 5

The missing data mechanisms under consideration are 'missing completely at random' (MCAR, Rubin 1987), where the probability to be missing is the same for all $n_{\text{obs}} \times p$ cells in y, and a right-tailed 'missing at random' (MAR) mechanism, where the probability to be missing is a function of the observed data, and higher values are more likely to be missing. The proportion of incomplete cases π_{inc} is set to 25%, 50%, and 75%. We use the 'mice' package (function `mice::ampute()`, van Buuren and Groothuis-Oudshoorn 2011) to obtain an incomplete dataset $\{\mathbf{Y}_{\text{obs}}, \mathbf{Y}_{\text{mis}}\}$ for every missingness condition.

Estimands

We impute the missing data five times ($m = 5$) using Bayesian linear regression imputation with `mice` (van Buuren and Groothuis-Oudshoorn 2011). Bayesian linear regression is a compatible imputation model for the set of variables at hand. Therefore, the imputation algorithm is guaranteed to converge (van Buuren 2007).

On each imputed dataset, we perform multiple linear regression as our analysis of scientific interest. Our estimands are the regression coefficients β,

$$\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3,$$

where $\hat{Y}$ is the predicted value of the outcome. We estimate these coefficients using the `lm()` function (R Core Team 2025) in each imputed dataset, and subsequently pool the estimates

Performance measures		
estimand		metric
	×	
β_0, β_1, β_2, β_3		bias, coverage rate, CI width

Non-convergence diagnostics				
identifier		parameter		variable
	×		×	
AC, $\widehat{R}$		chain means, chain variances		Y, X_1, X_2, X_3

across the five imputations using Rubin's rules (function `mice::pool()`, van Buuren and Groothuis-Oudshoorn 2011).

Performance measures

We evaluate the pooled estimates against their true population values using the performance measures bias, confidence interval width, and coverage rate, as recommended by van Buuren (2018, § 2.5.2). We calculate bias as $\bar{Q} - Q$. CR is defined as the proportion of simulation repetitions in which the 95% confidence interval (CI) around $\bar{Q}$ covers the true estimand Q. Finally, we inspect CI width (CIW): the difference between the lower and upper bound of the 95% confidence interval around $\bar{Q}$. CIW is of interest because it is a measure of efficiency. Under nominal coverage, short CIs are preferred, since wider CIs indicate lower statistical power.

Diagnostic methods

We use two non-convergence identifiers—autocorrelation and $\widehat{R}$—to diagnose non-convergence in the imputation algorithm. These two identifiers are applied to two scalar summaries of the state-space of the algorithm—chain means and chain variances. We compute the two identifiers on four different variables: the outcome variable Y, and the three predictors X_1, X_2, and X_3. In total, we apply the two identifiers on two parameters and four variables, resulting in 16 sets of identifier-parameter-variable pairs.

Simulation results

Our results show the non-convergence metrics and the performance of the MICE algorithm under different missingness conditions at a varying number of iterations. For reasons of brevity, we only discuss results for the worst-performing estimates in terms of bias and coverage, X_3. The full results are available in the online appendix.

Diagnostic methods

AC

Figures Figure 6 and Figure 7 show the autocorrelations in the chain means and chain variances, respectively.

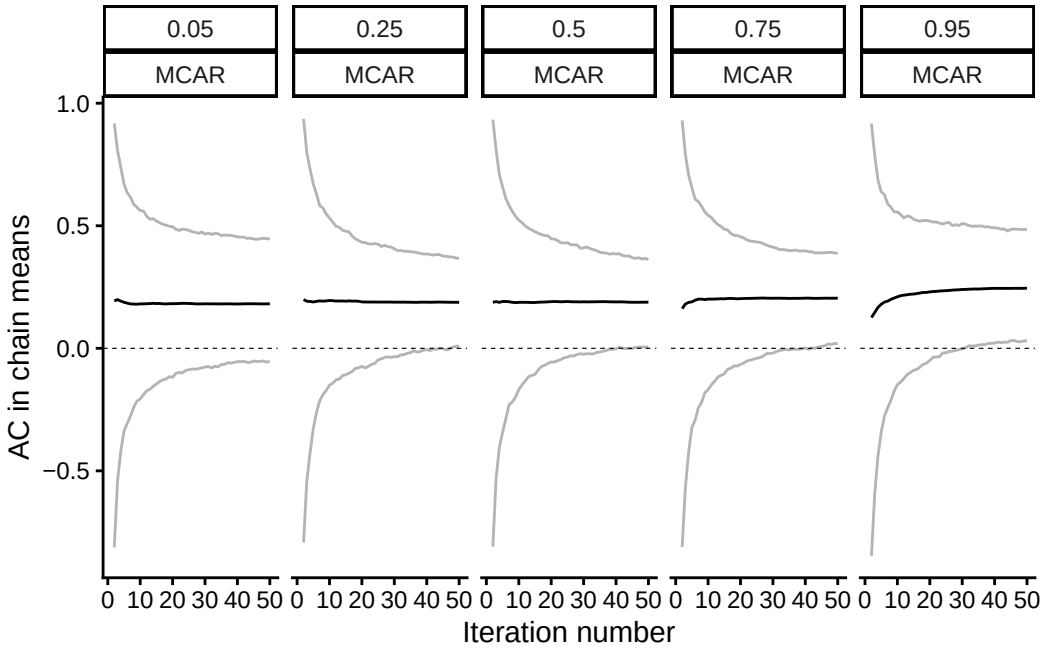


Figure 6: Autocorrelation in the chain means. The dashed line indicates perfect stationarity, $AC = 0$. The solid black line indicates the avarage autocorrelation accross simulation repetitions, with grey lines representing the empirical confidence interval.

Autocorrelation in the chain means decreases rapidly in the first few iterations (see Figure 6). The decrease is substantive until $n_{\text{it}} \geq 6$. This means that there is some initial trending within chains, but the average imputed value quickly reaches stationarity. These results hold irrespective of the missingness condition.

Autocorrelation in the chain variances show us something similar (see Figure 7). The number of iterations that is required to reach non-improving autocorrelations is somewhat more ambiguous than for chain means, but generally around $n_{\text{it}} \geq 10$. We do not observe a systematic difference between missingness conditions here either.

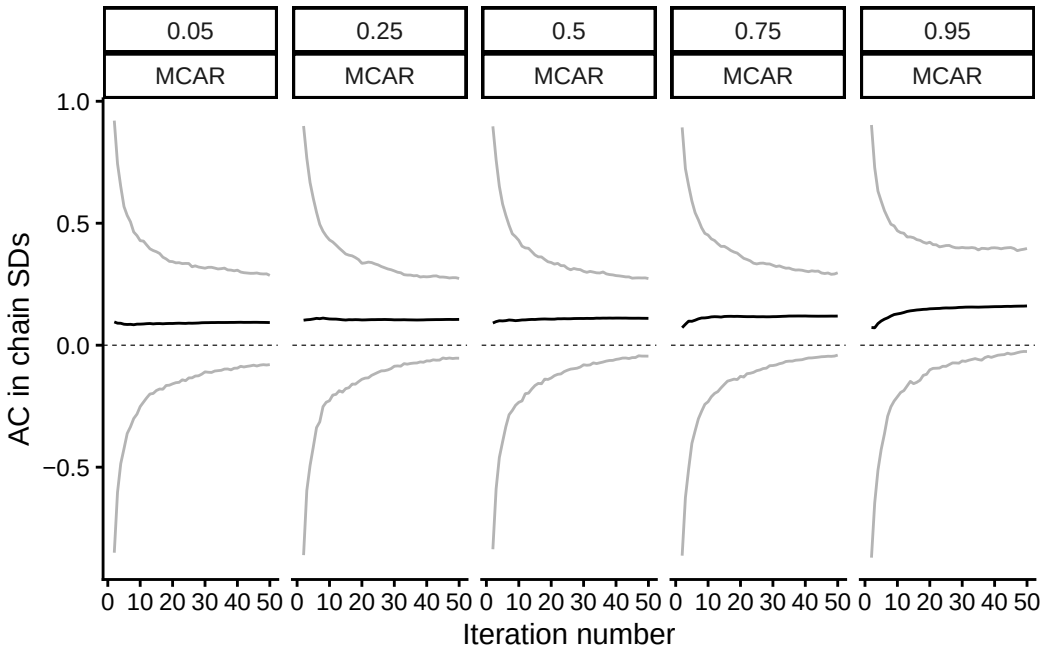


Figure 7: Autocorrelation in the chain variances. The dashed line indicates perfect stationarity, $AC = 0$. The solid black line indicates the avarage autocorrelation accross simulation repetitions, with grey lines representing the empirical confidence interval.

$\widehat{R}$

Figure Figure 8 and Figure 9 show the potential scale reduction factor in the chain means and chain variances respectively.

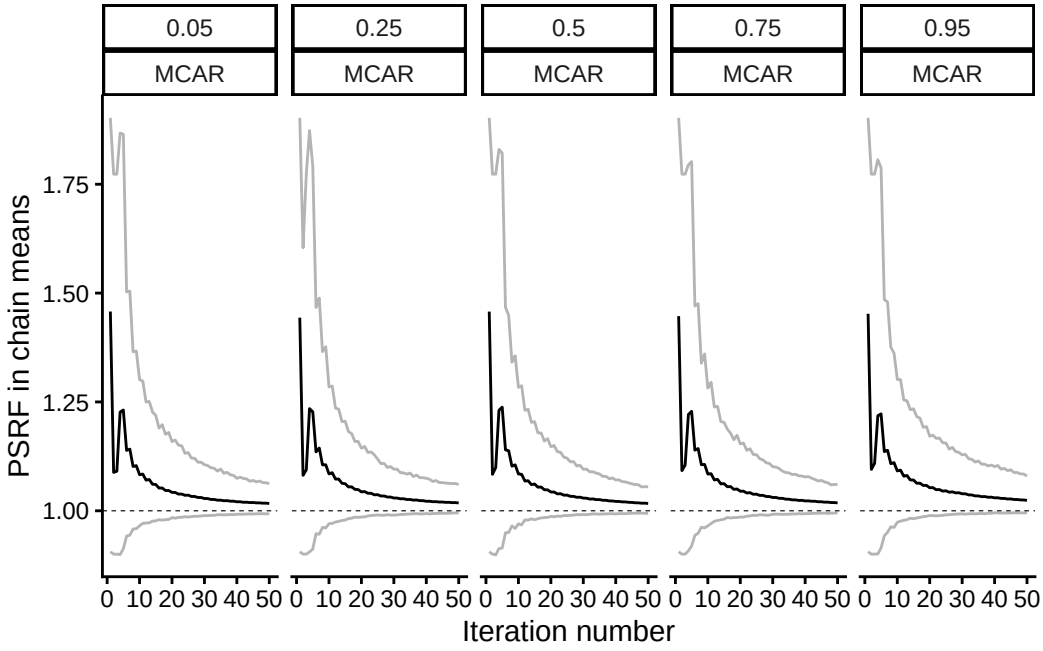


Figure 8: PSRF in the chain means. The dashed line indicates perfect mixing, $\widehat{R} = 0$. The solid black line indicates the avarage PSRF accross simulation repetitions, with grey lines representing the empirical confidence interval.

We observe that $\widehat{R}$-values of the chain means generally decrease as a function of the number of iterations (see Figure 8). An exception to this observation is a steep increase in iterations $3 \leq n_{\text{it}} \leq 5$. After the first couple of iterations, the mixing between chain means generally improves until $n_{\text{it}} \geq 30$ to 40. There is no apparent differentiation between the missingness conditions.

The mixing between chain variances mimics the mixing between chain means almost perfectly (see Figure 9). Irrespective of the missingness condition, the $\widehat{R}$-values taper off around $n_{\text{it}} \geq 30$.

Performance measures

In Figures 10 to 12 we show the performance measures: bias in the regression estimate, the empirical coverage rate of the regression estimate and the average confidence interval width of this estimate.

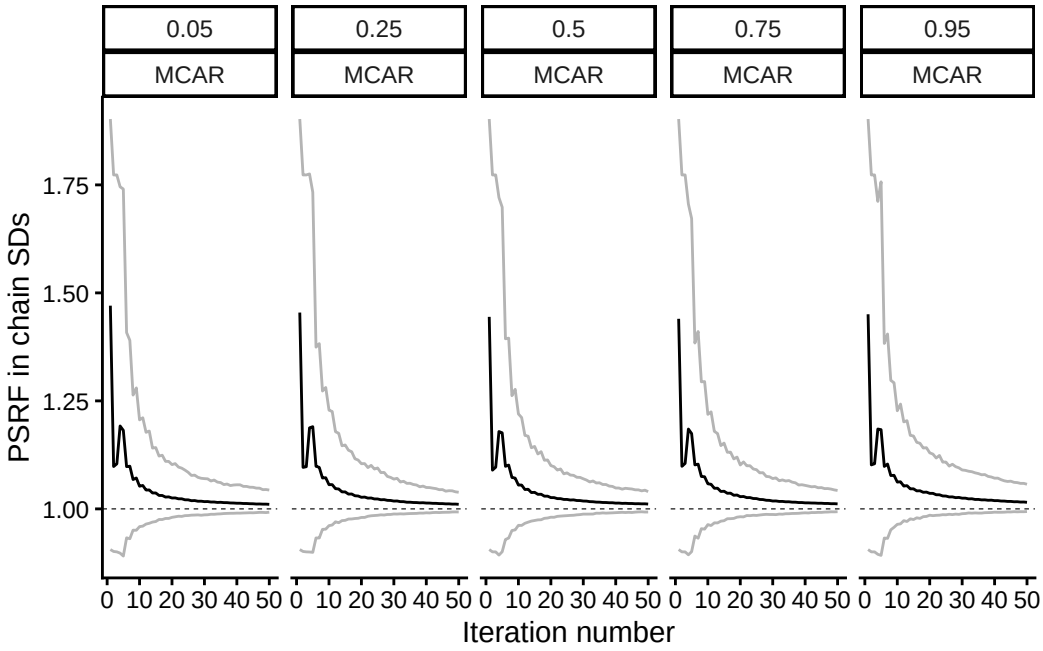


Figure 9: PSRF in the chain variances. The dashed line indicates perfect mixing, $\widehat{R} = 0$. The solid black line indicates the avarage PSRF accross simulation repetitions, with grey lines representing the empirical confidence interval.

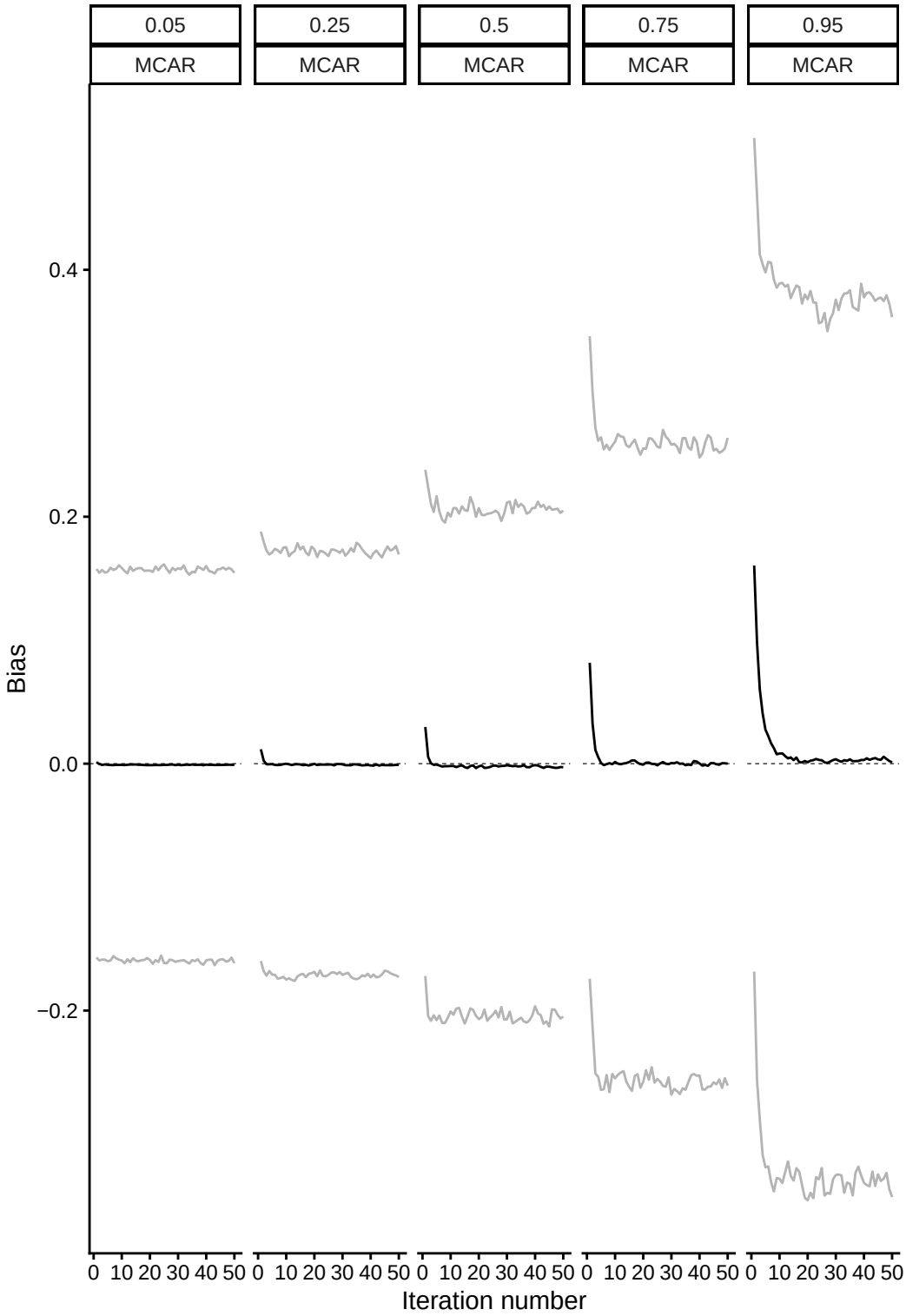


Figure 10: Bias in regression coefficient $\beta = 1$. The dashed line indicates no bias. The solid black line indicates the avarage bias accross simulation repetitions, with grey lines representing the empirical confidence interval.

We see that within a few iterations the bias in the regression estimate approaches zero (see Figure 10). When $n_{\text{it}} \geq 6$, even the worst-performing conditions (e.g., with a proportion of incomplete cases of 75%) produce stable, non-improving estimates.

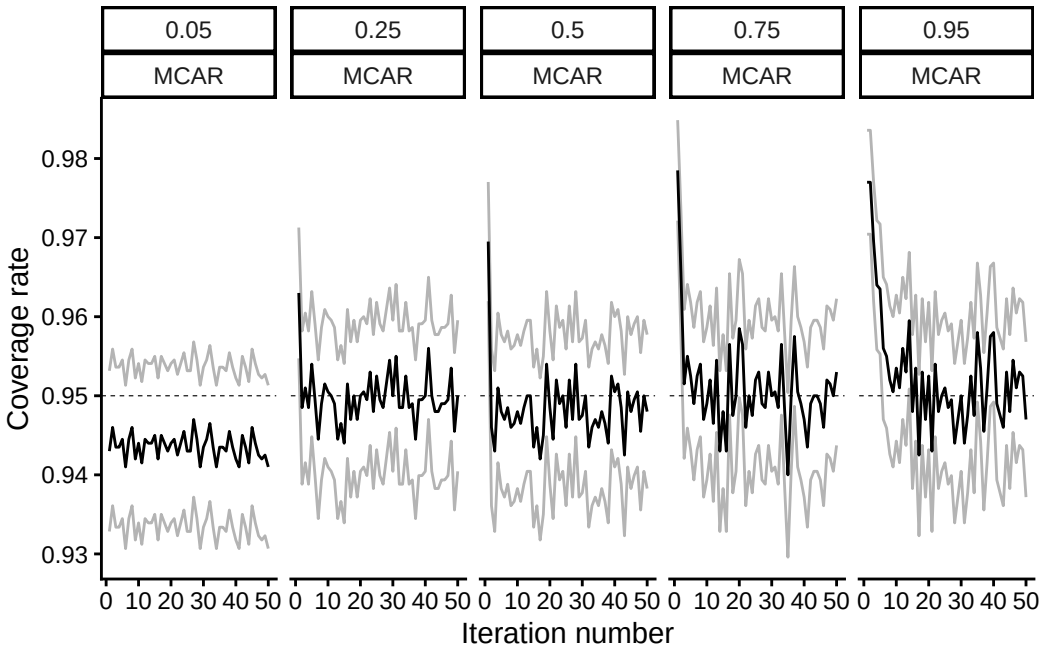


Figure 11: Coverage rate of regression coefficient $\beta = 1$. The dashed line indicates confidence-validity. The solid black line indicates the avarage coverage rate accross simulation repetitions, with grey lines representing the empirical confidence interval.

Nominal coverage is quickly reached (see Figure 11). After just three iterations, the coverage rates are non-improving in every missingness condition.

The average confidence interval width decreases quickly with every added iteration until a stable plateau is reached (see Figure 12). Depending on the proportion of incomplete cases this takes up-to $n_{\text{it}} \geq 9$.

Summary

Overall, the methods to diagnose non-convergence perform as expected: they indicate more signs of non-convergence in conditions with worse performance in terms of bias and confidence-validity. We observe approximately unbiased estimates after at most seven iterations (for any π_{inc} considered). This implies that the algorithm produces stable, non-improving estimates when $\tau \geq 7$. The number of iterations necessary to obtain approximately unbiased, confidence-valid estimates corresponds to a $\widehat{R}$ threshold of ≈ 1.2.

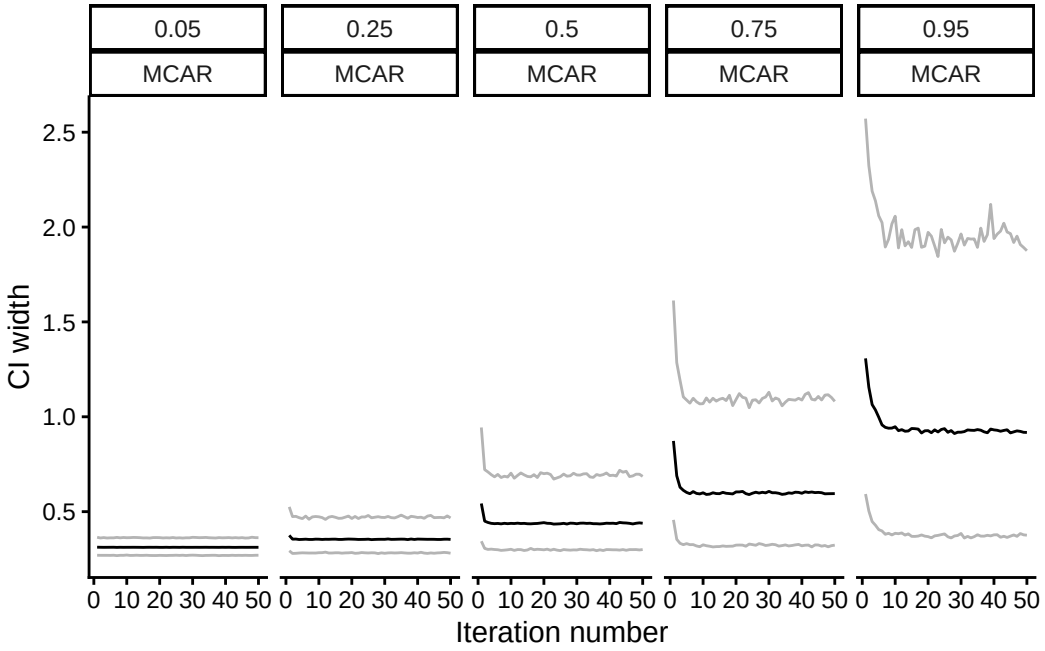


Figure 12: Confidence interval width of regression coefficient $\beta = 1$. The solid black line indicates the avarage CIW accross simulation repetitions, with grey lines representing the empirical confidence interval.

Discussion

Identifying non-convergence may be a crucial aspect of drawing valid statistical inferences from incomplete data. In this simulation study, we have shown that non-convergence in iterative imputation algorithms does indeed go hand in hand with biased, invalid estimates. Our study , however, found that inferential validity was achieved after five to ten iterations, much earlier than indicated by non-convergence identifiers. While convergence diagnostics kept improving substantially until $\tau \approx 20 - 30$, performance measures did not improve after $\tau = 9$. Of course, it never hurts to iterate longer, but such calculations hardly brought added value.

The main finding of this study is that valid inferences may be obtained much quicker than approximate algorithmic convergence is reached. Under the current specifications, approximately unbiased, confidence-valid estimates were obtained after a maximum of nine iterations. Continued iterations beyond $\tau = 9$ did not yield better estimates. In contrast, $\widehat{R}$-values decreased substantially until 20 to 30 iterations, which implies that mixing in the algorithm still improved with each additional iteration. AC-values only improved until about six iterations, suggesting minimal improvement in trending was obtained beyond $\tau = 6$ in the current simulation set-up.

The potential scale reduction factor metric assumes over-dispersed starting values of the iterative algorithm. The MICE algorithm does not start in an over-dispersed state. MICE does not rely on starting values for parameters at all: instead, the algorithm is initialized based on a 'zeroth' iteration, where missing values are filled in using a random draw from observed values. Under MCAR, this would typically not yield an over-dispersed state at all, depending on the parameter of interest: means and variances are not affected, regression coefficients and other multivariate parameters start biased downwards (because sampling starting values distorts multivariate structures in the data). Whether this accrues to over-dispersion is questionable. Under MAR (or MNAR), some over-dispersion might occur when the observed data and imputed data differ impactfully. The algorithm should 'escape' the initial state (based on the starting values drawn from observed data alone). Over iterations, all parameters of interest (e.g. means, variances, multivariate estimands) should converge towards a stable state. Moreover, the MICE algorithm is initialized with more information than a typical MCMC algorithm: parameter estimates depend not only on imputed data, but also on observed data that does not change over iterations. Therefore, the initial state of the algorithm is 'too good' to observe a relative decrease in $\widehat{R}$.

In short, the validity of iterative imputation stands or falls with algorithmic convergence—or so it's thought. We have shown that iterative imputation algorithms can yield correct outcomes even when a converged state has not yet formally been reached. Any further iterations just burn computational resources without improving the statistical inferences.

References

Abayomi, Kobi, Andrew Gelman, and Marc Levy. 2008. "Diagnostics for Multivariate Imputations." *Journal of the Royal Statistical Society. Series C (Applied Statistics)* 57 (3): 273–91. https://www.jstor.org/stable/20492604.

Box, George E. P., Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. 2015. *Time Series Analysis: Forecasting and Control*. John Wiley & Sons. https://books.google.com?id=rNt5CgAAQBAJ.

Brooks, Stephen P., and Andrew and Gelman. 1998. "General Methods for Monitoring Convergence of Iterative Simulations." *Journal of Computational and Graphical Statistics* 7 (4): 434–55. https://doi.org/10.1080/10618600.1998.10474787.

Buuren, S. van. 2018. *Flexible Imputation of Missing Data*. 2nd ed. CRC Press.

Buuren, Stef van. 2007. "Multiple Imputation of Discrete and Continuous Data by Fully Conditional Specification." *Statistical Methods in Medical Research* 16 (3): 219–42. https://doi.org/10.1177/0962280206074463.

Buuren, Stef van, and Karin Groothuis-Oudshoorn. 2011. "Mice: Multivariate Imputation by Chained Equations in R." *Journal of Statistical Software* 45 (1): 1–67. https://doi.org/10.18637/jss.v045.i03.

Cowles, Mary Kathryn, and Bradley P Carlin. 1996. "Markov Chain Monte Carlo Convergence Diagnostics: A Comparative Review." *Journal of the American Statistical Association* 91 (434): 883–904.

El Adlouni, Salaheddine, Anne-Catherine Favre, and Bernard Bobée. 2006. "Comparison of Methodologies to Assess the Convergence of Markov Chain Monte Carlo Methods." *Computational Statistics & Data Analysis* 50 (10): 2685–701. https://doi.org/10.1016/j.csda.2005.04.018.

Gelman, Andrew, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis*. CRC Press LLC.

Gelman, Andrew, and Donald B. Rubin. 1992. "Inference from Iterative Simulation Using Multiple Sequences." *Statistical Science* 7 (4): 457–72. https://doi.org/10.1214/ss/1177011136.

Genz, Alan, and Frank Bretz. 2009. *Computation of Multivariate Normal and t Probabilities*. Lecture Notes in Statistics. Springer-Verlag.

Hoff, Peter D. 2009. *A First Course in Bayesian Statistical Methods*. Springer Texts in Statistics. Springer New York. https://doi.org/10.1007/978-0-387-92407-6.

Lynch, Scott M. 2007. "Evaluating Markov Chain Monte Carlo Algorithms and Model Fit." In *Introduction to Applied Bayesian Statistics and Estimation for Social Scientists*, edited by Scott M. Lynch. Springer New York. https://doi.org/10.1007/978-0-387-71265-9_6.

Morris, Tim P., Ian R. White, and Michael J. Crowther. 2019. "Using Simulation Studies to Evaluate Statistical Methods." *Statistics in Medicine* 38 (11): 2074–102. https://doi.org/10.1002/sim.8086.

Murray, Jared S. 2018. "Multiple Imputation: A Review of Practical and Theoretical Findings." *Statistical Science* 33 (2): 142–59. https://doi.org/10.1214/18-STS644.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. Manual. R Foundation for Statistical Computing. https://www.R-project.org/.

Raghunathan, Trivellore, and Irina Bondarenko. 2007. *Diagnostics for Multiple Imputations*. SSRN Scholarly Paper ID 1031750. Social Science Research Network. https://papers.ssrn.com/abstract=1031750.

Rubin, Donald B. 1976. "Inference and Missing Data." *Biometrika* 63 (3): 581–92. https://doi.org/10.2307/2335739.

Rubin, Donald B. 1987. *Multiple Imputation for Nonresponse in Surveys*. Wiley Series in Probability and Mathematical Statistics Applied Probability and Statistics. Wiley.

Rubin, Donald B. 1996. "Multiple Imputation After 18+ Years." *Journal of the American Statistical Association* 91 (434): 473–89. https://doi.org/10.1080/01621459.1996.10476908.

Schafer, J. L. 1997. *Analysis of Incomplete Multivariate Data*. Chapman and Hall/CRC. https://doi.org/10.1201/9781439821862.

Takahashi, Masayoshi. 2017. "Statistical Inference in Missing Data by MCMC and Non-MCMC Multiple Imputation Algorithms: Assessing the Effects of Between-Imputation Iterations." *Data Science Journal* 16 (0, 0): 37. https://doi.org/10.5334/dsj-2017-037.

Vehtari, Aki, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. 2021. "Rank-Normalization, Folding, and Localization: An Improved R^ for Assessing Convergence of MCMC." *Bayesian Analysis* -1 (January): 1–38. https://doi.org/10.1214/20-BA1221.

Zhu, Jian, and Trivellore E. Raghunathan. 2015. "Convergence Properties of a Sequential

Regression Multiple Imputation Algorithm." *Journal of the American Statistical Association* 110 (511): 1112–24. https://doi.org/10.1080/01621459.2014.948117.

4 Chapter 4: ggmice

Published as Oberman (2022).

[TODO: add credit statement, merge formatting]

Get started with `ggmice`

The R package `ggmice` supports `mice` imputation workflows with visualizations. `ggmice` provides 'grammar of graphics' (`ggplot2`) functionality for exploring incomplete data, building imputation models, and evaluating imputations.

The core function, `ggmice()` , is designed to highlight missing and imputed observations, instead of omitting these from graphs. The resulting visualizations either contain observed and *missing* data, or observed and *imputed* data, allowing for direct graphical comparisons between incomplete and imputed data.

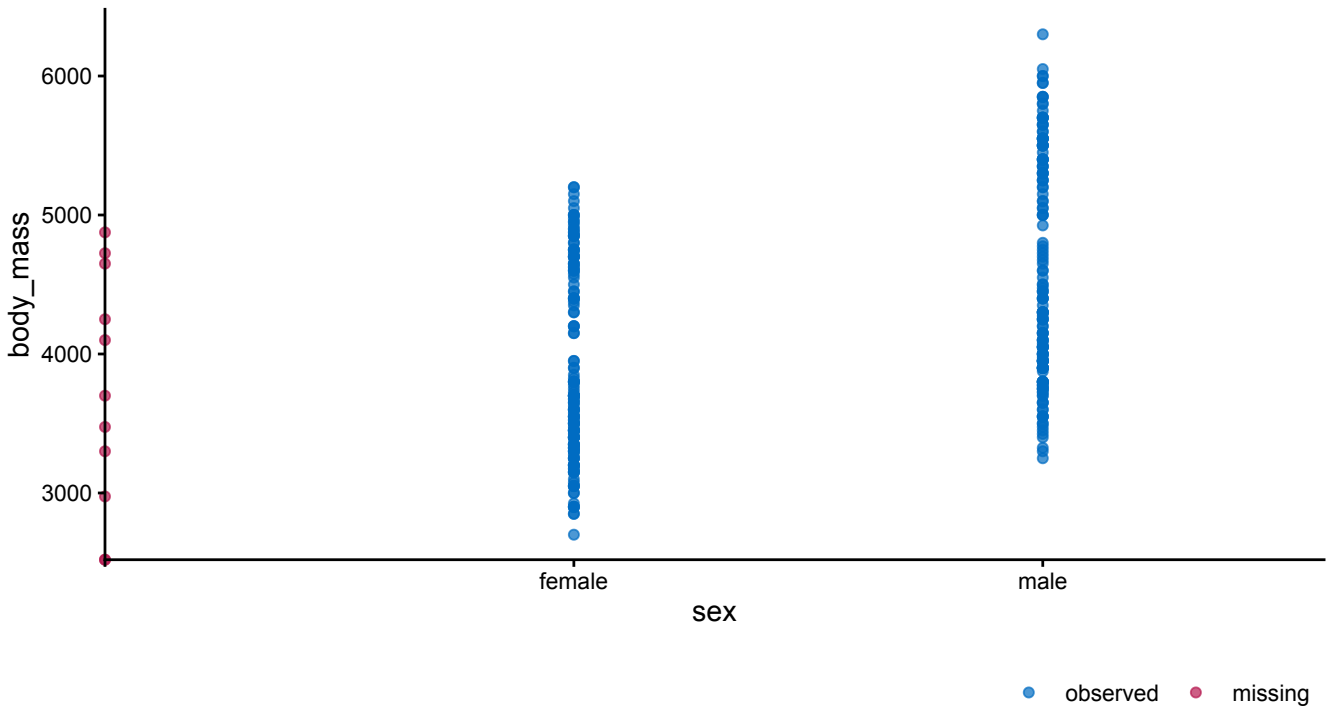
Minimal example

Set-up the environment in R with packages for imputation and visualization.

```
library(mice)
library(ggplot2)
library(ggmice)
```

Visualize some incomplete data with `ggmice()` . We use the `penguins` data from the base R `datasets` , with observational data on penguins ($n = 344$) with 8 variables (e.g., penguin species and body mass measurements).

```
ggmice(penguins, aes(sex, body_mass)) +
    geom_point()
```



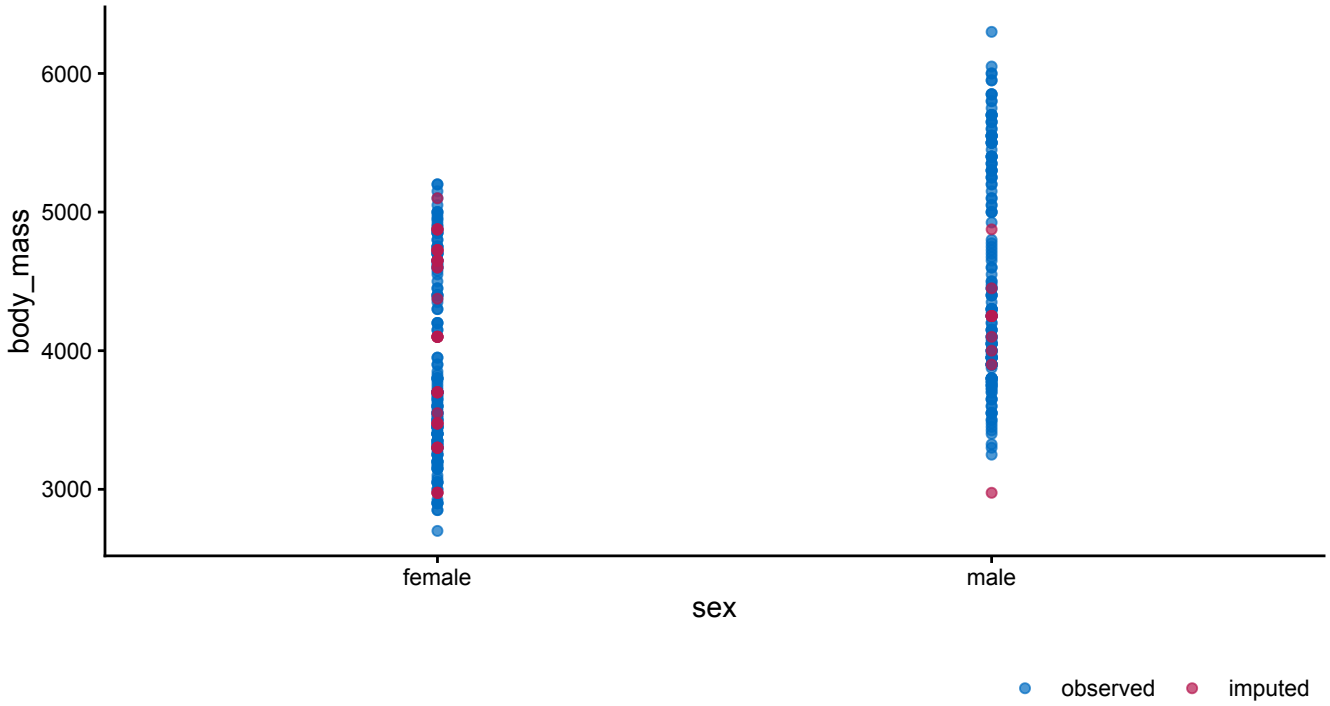
Visualization of the incomplete `penguins` dataset, displaying penguin's body mass measurements (in grams) by their assigned sex classification. Complete cases are displayed in blue and incomplete cases in red. The missing values are plotted on the axes lines, showing that some cases have observed data for body mass and missing sex classification, and at least one case has neither observed. The observed `body_mass` data of cases with missing `sex` are plotted on the vertical axis.

Impute the missing values in the `penguins` data set with `mice`.

```
imp <- mice(penguins, print = FALSE)
```

Visualize the imputed `penguins` data set with `ggmice`.

```
ggmice(imp, aes(sex, body_mass)) +
    geom_point()
```



Visualization of the `penguins` dataset, showing body mass measurements (in grams) and sex classification after multiply imputing the missing values with `mice`. Imputed values are displayed in red.

The `ggmice()` visualizations may be used to inspect the incomplete and imputed data, and to evaluate imputation model fit (e.g., whether the missing data have been imputed within the range of the observed data). Aside from the `ggmice()` function, the package contains several other visualization tools for imputation with `mice`. These other functions share the naming convention `plot_*()` and several function arguments (e.g., `vrb` to plot only a subset of variables).

In this vignette, you will learn how to create and interpret `ggmice` visualizations for each phase of your imputation workflow. But first, we need some background information about the `ggmice` package and its core function, `ggmice()`.

The `ggmice` package

The `ggmice` package unifies the visualization of incomplete and imputed data, and offers tools for building and evaluating imputation models in R (R Core Team 2025). `ggmice` is an extension package to the popular R packages `mice` (van Buuren and Groothuis-Oudshoorn 2011) and `ggplot2` (Wickham 2016).

`mice` has become standard software for handling the ubiquitous problem of incomplete data. With `mice`, missing data points are 'imputed' (i.e., filled in) to obtain several completed data sets. Filling in the missing data multiple times allows for a valid representation of the uncertainty due to missingness. `ggmice` supports `mice` workflows with graphical evaluation tools.

The `ggmice` package also extends `ggplot2`, by offering functionality for the visualization of missing and imputed values. The functions in `ggmice` adhere to `ggplot2` 's 'grammar of graphics' philosophy. The resulting plots are standard `ggplot` objects, which means you can add layers, labels, themes, facets, and transformations as usual. Moreover, these editable `ggplot` objects are easily adapted into publication-quality graphics.

The `ggmice()` function

The core function in the `ggmice` package is `ggmice()`. `ggmice()` is a `ggplot2::ggplot()` wrapper function, which plots either missing or imputed values.

- When the `data` argument is supplied with an incomplete dataset (a `data.frame` object), `ggmice()` visualizes observed and *missing* data.
- When the `data` argument is supplied with multiply imputed datasets (a `mids` object), `ggmice()` visualizes observed and *imputed* data.

The `ggmice()` function mimics how the `ggplot2` function `ggplot()` works. Both take a `data` argument and a `mapping` argument, and will return an object of class `ggplot`.

Using `ggmice()` looks equivalent to a `ggplot()` call:

```
ggplot(penguins, aes(x = body_mass))
ggmice(penguins, aes(x = body_mass))
```

The main difference between the two functions is that `ggmice()` includes some pre-processing steps for incomplete and imputed data. The functions can be used interchangeably, except for two key details:

1. The object supplied to the `data` argument in `ggmice()` may be either an incomplete dataset of class `data.frame`, or an imputation object of class `mice::mids`.
2. The `mapping` argument in `ggmice()` cannot be empty.

This is in contrast to the aesthetic mapping in `ggplot()`, which may also be provided in subsequent plotting layers. Because of the internal processing in `ggmice()`, the `mapping` argument is **required** for each `ggmice()` call. An `x` or `y` mapping (or both) has to be supplied for `ggmice()` to function. This aesthetic mapping can be provided with the `ggplot2` function `aes()` (or equivalents). Other mappings may be provided too, except for `colour`, which is already used to display observed versus missing or imputed data.

After creating a `ggplot` object, any desired plotting layers may be added (e.g., with the family of `ggplot2::geom_*` functions), or adjusted (e.g., with the `ggplot2::labs()` function). This makes `ggmice()` a versatile plotting function for incomplete and imputed data.

Incomplete data

If the object supplied to the `data` argument in the `ggmice()` function is a `data.frame`, the visualization will contain observed data in blue and missing data in red.

Because missing data points are by definition unobserved, their values cannot be plotted directly. While for categorical variables we can display the missing values as their own category, there is no straightforward way to show the missingness in continuous variables. Therefore, we typically omit incomplete observations from our graphs.

In bivariate graphs, however, we can display incomplete observations in variable pairs. When there is a missing datapoint on one variable and observed data on the other, there is no bivariate coordinate in the graph to display this incomplete observation. But we can still plot the information that we do have: the observed datapoint. These incomplete observations are plotted on the axes lines:

- Cases with observed `X` and missing `Y` are plotted on the horizontal axis.
- Cases with observed `Y` and missing `X` are plotted on the vertical axis.
- Cases with both `X` and `Y` missing are plotted on the intersection of the two axes, since no data is available.

These plotted values are observed for the variable belonging to that axis line, but not for the variable orthogonal to the axis line. This provides a visual cue that the missing data is distinct from the observed values, but still displays the observed value of the other variable. In short, `ggmice()` plots the observed values of incomplete cases on the axis line of the observed variable. This is in contrast to a regular `ggplot()` call with the same arguments, which would leave out all cases with missingness.

Imputed data

If the `data` argument in `ggmice()` is provided a `mice::mids` object, the resulting plot will contain observed data in blue and imputed data in red.

Experienced `mice` users may already be familiar with the `lattice` style plotting functions in `mice` for visualizing imputed data. These 'old friends' such as `mice::stripplot()` can be re-created with the `ggmice()` function, see the Old friends (https://amices.org/ggmice/articles/old_friends.html) vignette for advice.

So, with `ggmice()` we lose less information, and may gain valuable insight into the missingness in the data.

Imputation workflow

In this vignette, the `ggmice` functions are presented in the order of a typical imputation workflow, where the missingness is first investigated, then imputation models are built based on relations between variables, and finally the imputations are inspected visually.

Set-up

You can install the latest `ggmice` release from CRAN (https://CRAN.R-project.org/package=ggmice) with:

```
install.packages("ggmice")
```

The development version of the `ggmice` package can be installed from GitHub with:

```
# install.packages("devtools")
devtools::install_github("amices/ggmice")
```

After installing `ggmice`, you can load the package into your `R` workspace. It is highly recommended to load the `mice` and `ggplot2` packages as well. This vignette assumes that all three packages are loaded:

```
library(mice)
library(ggplot2)
library(ggmice)
```

We will use the `penguins` data for illustrations, available from the base R `datasets`. The `penguins` dataset contains incomplete observational data on penguin sightings ($n = 344$). Each row represents an individual penguin, and the nine variables describe its characteristics (e.g., species) and measurements (e.g., body weight in grams).

```
head(penguins)
#>   species    island bill_len bill_dep flipper_len body_mass
#> 1  Adelie Torgersen     39.1     18.7         181      3750
#> 2  Adelie Torgersen     39.5     17.4         186      3800
#> 3  Adelie Torgersen     40.3     18.0         195      3250
#> 4  Adelie Torgersen       NA       NA          NA        NA
#> 5  Adelie Torgersen     36.7     19.3         193      3450
#> 6  Adelie Torgersen     39.3     20.6         190      3650
#>      sex year
#> 1   male 2007
#> 2 female 2007
#> 3 female 2007
#> 4   <NA> 2007
#> 5 female 2007
#> 6   male 2007
```

In this vignette, we will treat the penguins data as our working example.

With that, we have the necessary packages (`mice`, `ggplot2`, and `ggmice`) and an incomplete dataset (`penguins`) to start the full imputation workflow. We will first use `ggmice` to explore marginal and joint distributions of incomplete variables. Next, we will construct and inspect imputation models based on relations between the variables. Finally, we will visualize the imputation algorithm output and assess whether the imputation models yield plausible values.

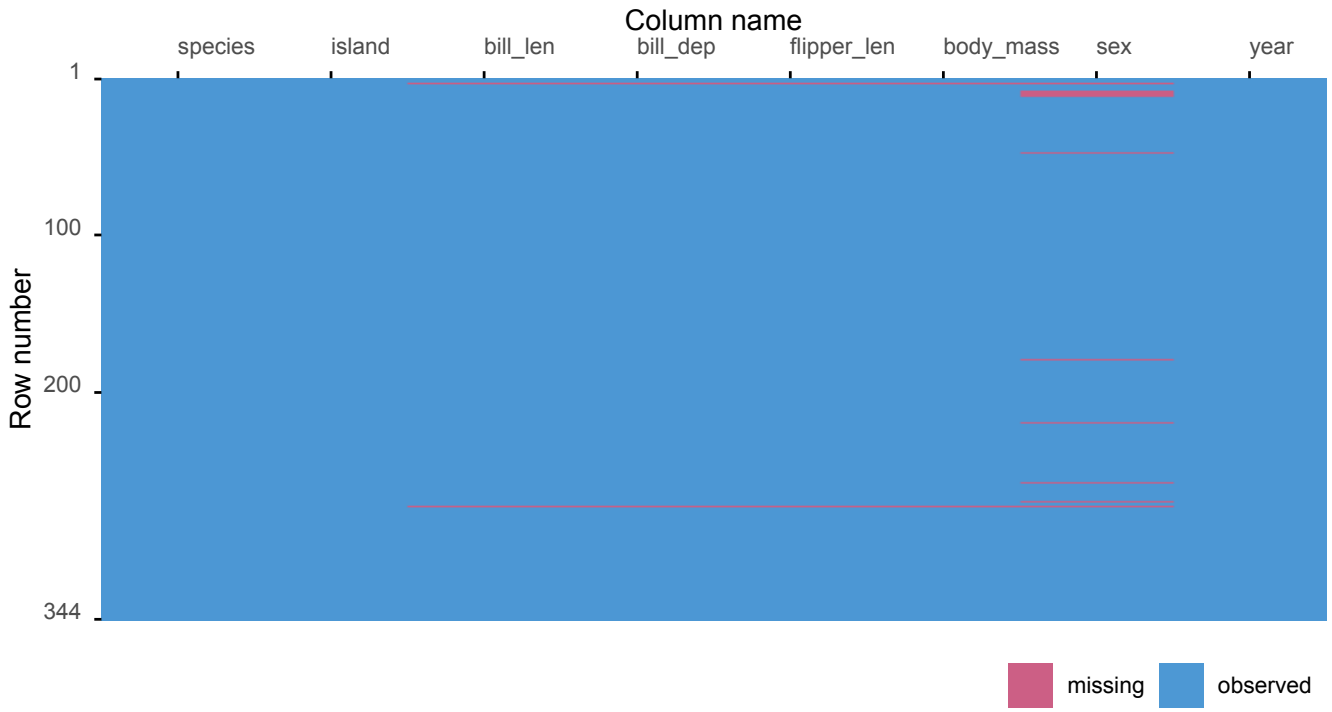
Exploring incomplete data

In a `mice` workflow, the missing data should first be inspected before determining an imputation strategy. The missing data pattern shows where in the incomplete data the missing values occur. Multivariate graphs may highlight the severity of the missing data problem in the variables that will be used in the eventual analysis model.

`plot_miss()`

The `plot_miss()` function facilitates the exploration of the location of the missingness in the data. The result is the graphical equivalent to the missingness matrix `is.na(penguins)`.

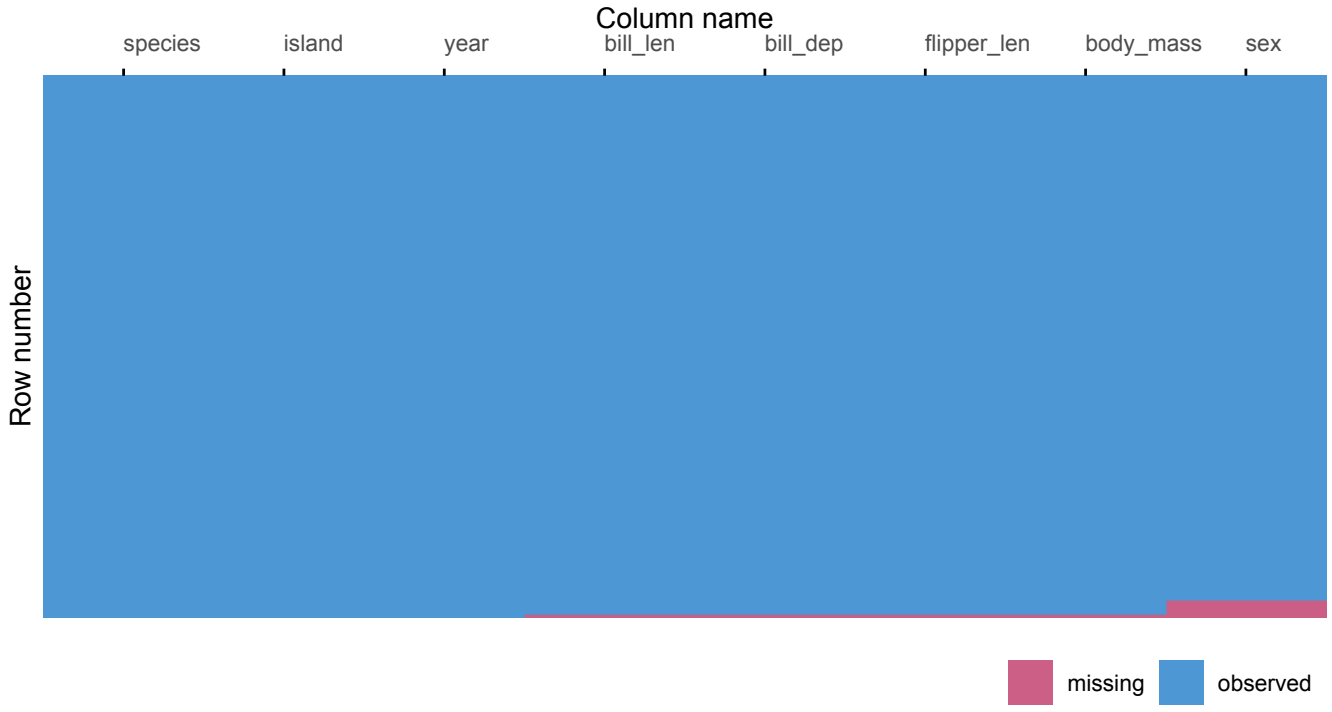
```
plot_miss(penguins)
```



Visualization of the missing data matrix.

The plot can be ordered by the missingness proportion. In the ordered graph, cases with more missing values are plotted last, matching the order of the missing data pattern plot (`plot_pattern()`).

```
plot_miss(penguins, ordered = TRUE)
```



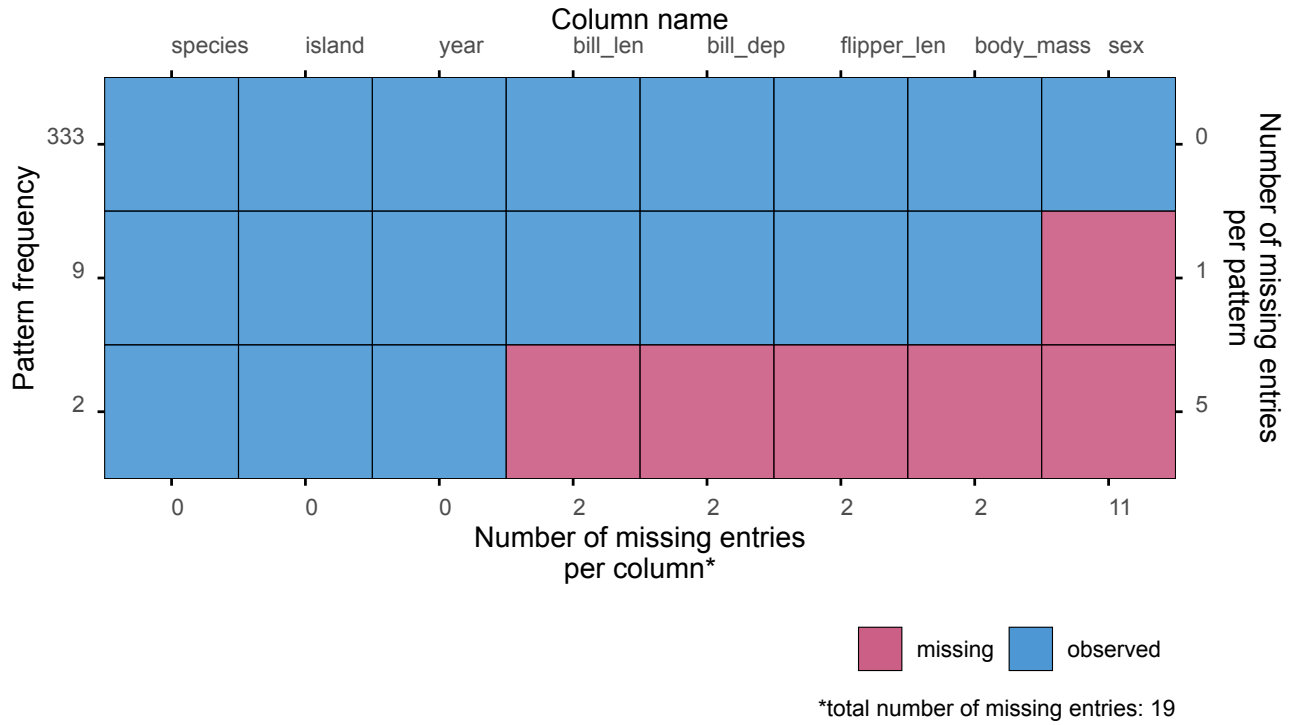
Visualization of the missing data matrix, ordered by the missingness proportion.

Other optional function arguments can be specified too (e.g., `rotate` to display the column names at a 90 degree angle). Please refer to the function documentation for details.

plot_pattern()

The `plot_pattern()` function displays the missing data pattern in an incomplete dataset, which should be supplied via the `data` argument.

```
plot_pattern(penguins)
```



Missing data pattern plot.

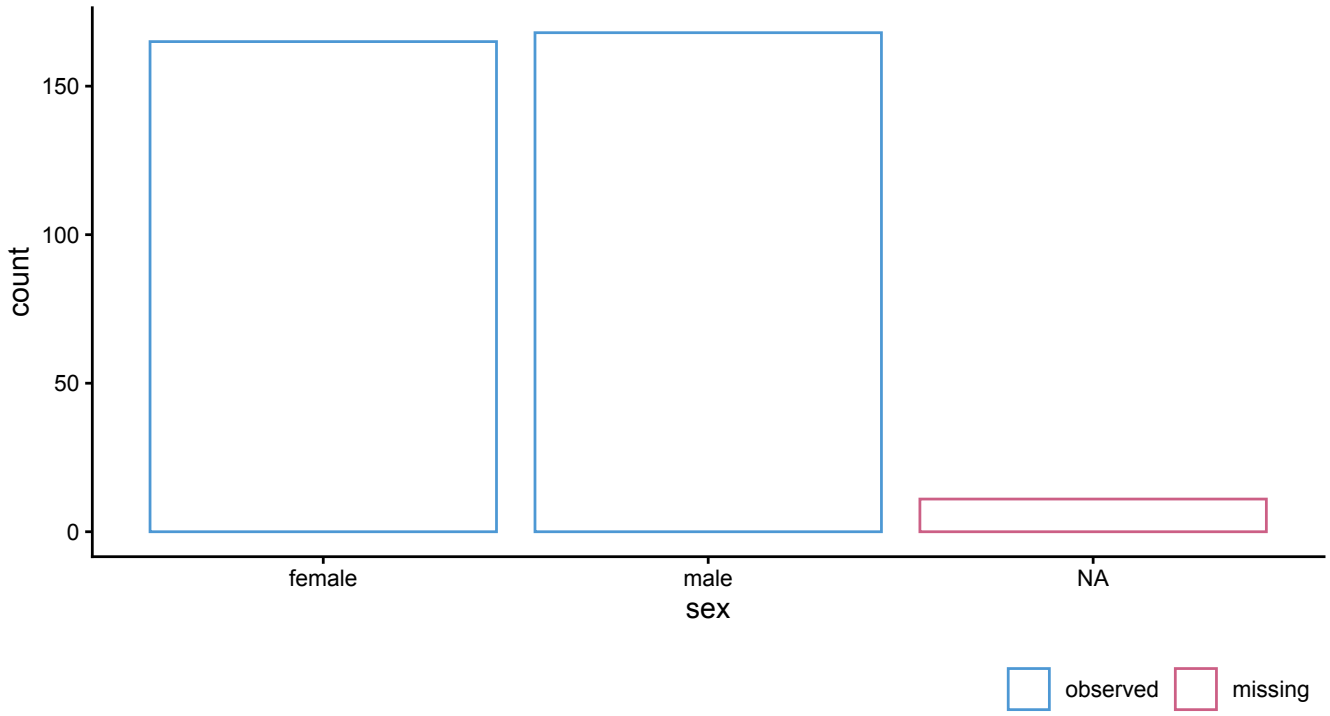
The `plot_pattern()` function has several optional arguments, such as `square`, which determines whether the missing data pattern plot has the default square or rectangular tiles. Please refer to the function documentation for details.

ggmice()

In the missing data exploration phase, the `ggmice()` function can be used to visualize the distributions of incomplete variables, associations with other variables, and with missingness indicators.

For example, we can plot the distribution of an incomplete categorical variable with:

```
ggmice(penguins, aes(x = sex)) +
  geom_bar(fill = " white")
```

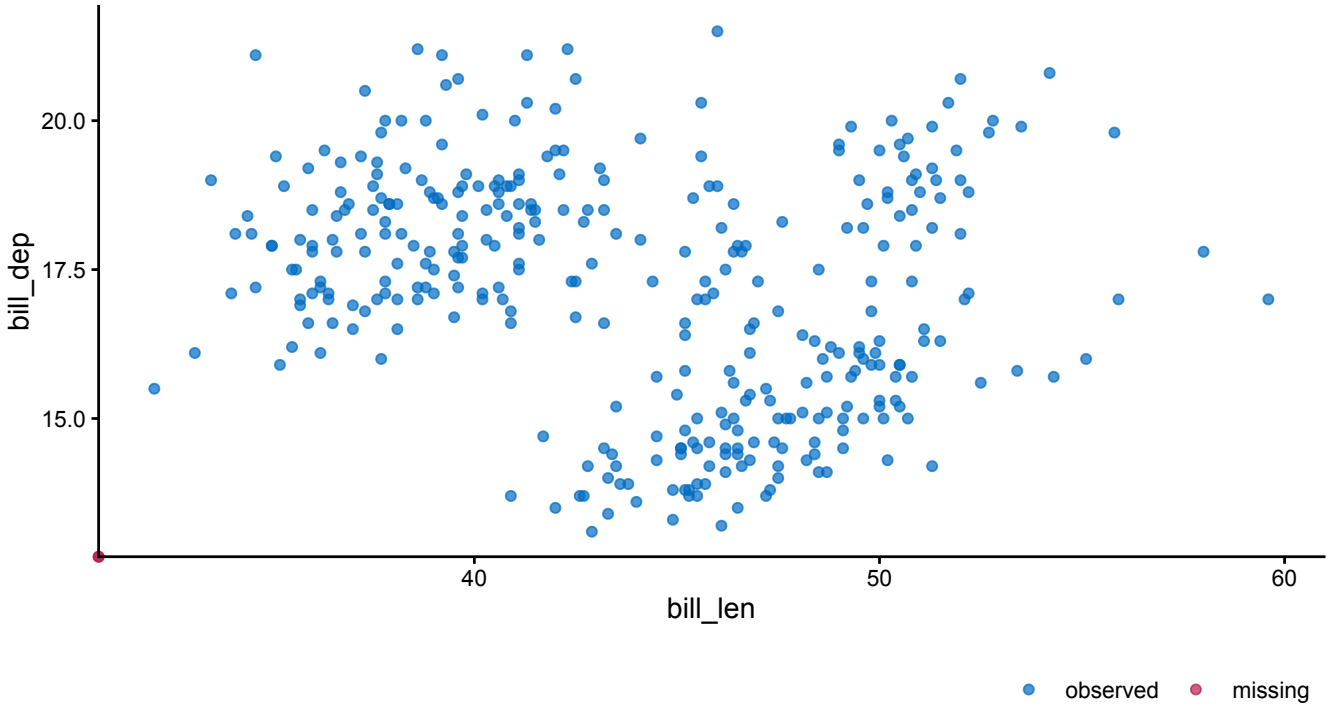


Bar graph showing the distribution of and missingess in the variable `sex`.

In this graph, the missing data is plotted as a separate category.

We can also plot bivariate distributions in the incomplete data. To create a scatterplot of two continuous incomplete variables we can use:

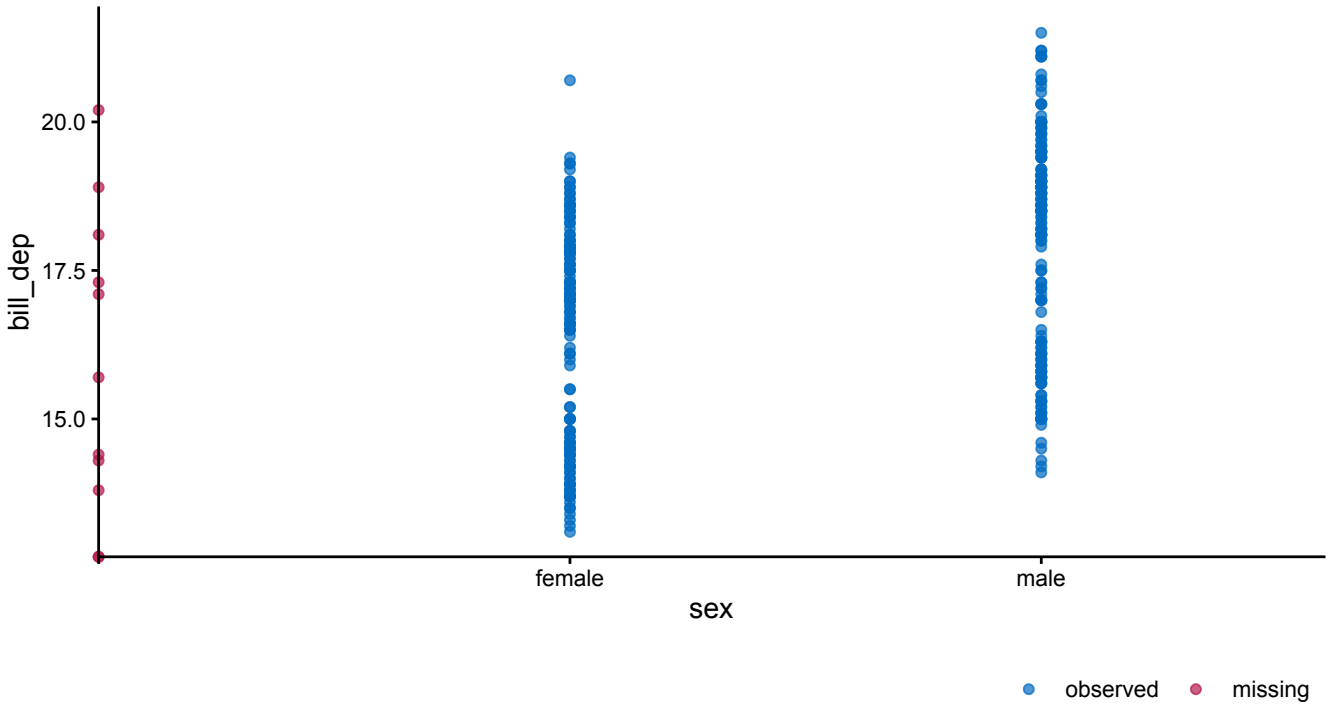
```
ggmice(penguins, aes(x = bill_len, y = bill_dep)) +
  geom_point()
```



Scatterplot of the `penguins` bill depth (in cm) by bill length (in cm), showing missing values on the axis lines. The red point at the intersection of the axes indicates that at least one penguin has neither bill depth nor bill length observed.

Another bivariate graph can be made with the incomplete continuous variable `bill_dep` against the incomplete categorical variable `sex` :

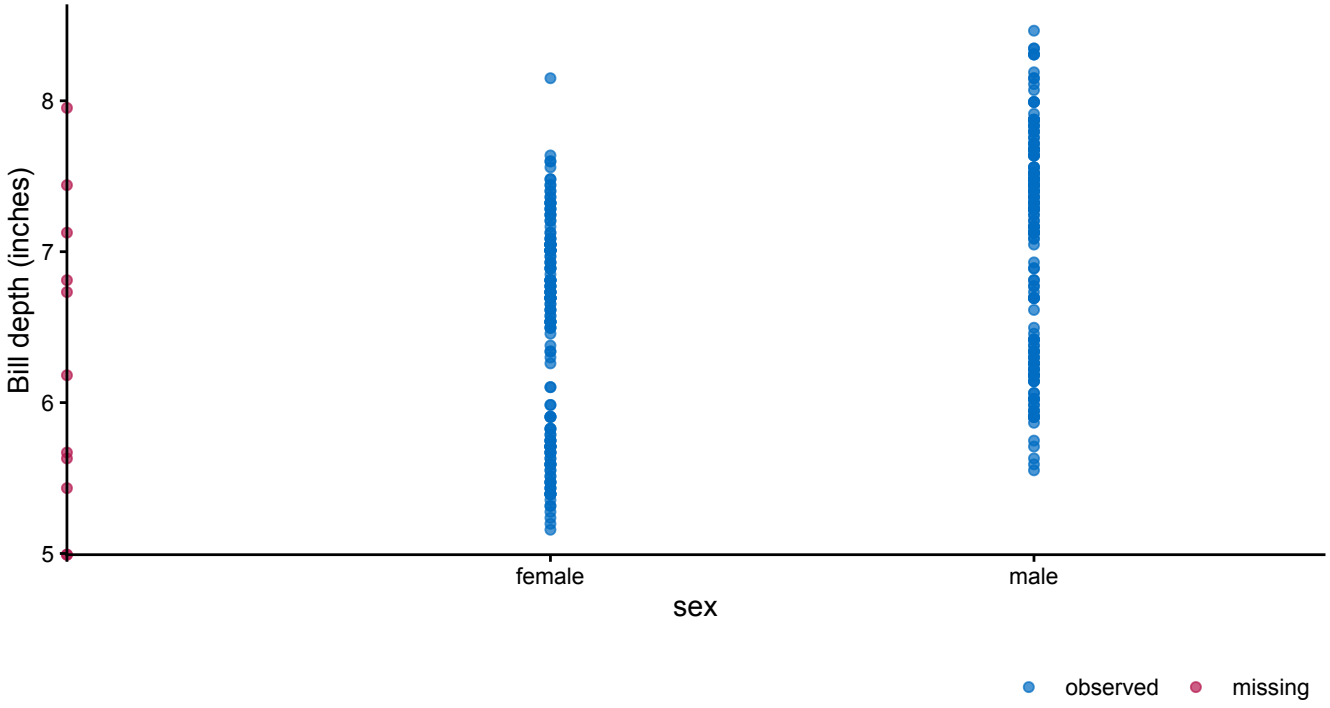
```
ggmice(penguins, aes(x = sex, y = bill_dep)) +
  geom_point()
```



Visualization of bill depth (in cm) by sex, showing missing values in red on the axis lines. Observed values for bill depth with missing data for sex are plotted on the vertical axis. The red point at the intersection of the axes indicates that at least one penguin has neither bill depth nor sex observed.

The 'grammar of graphics' makes it easy to adjust the plots programmatically. The `ggplot2` framework allows us to convert the plotted values of the variable `bill_dep` from centimeters to inches with:

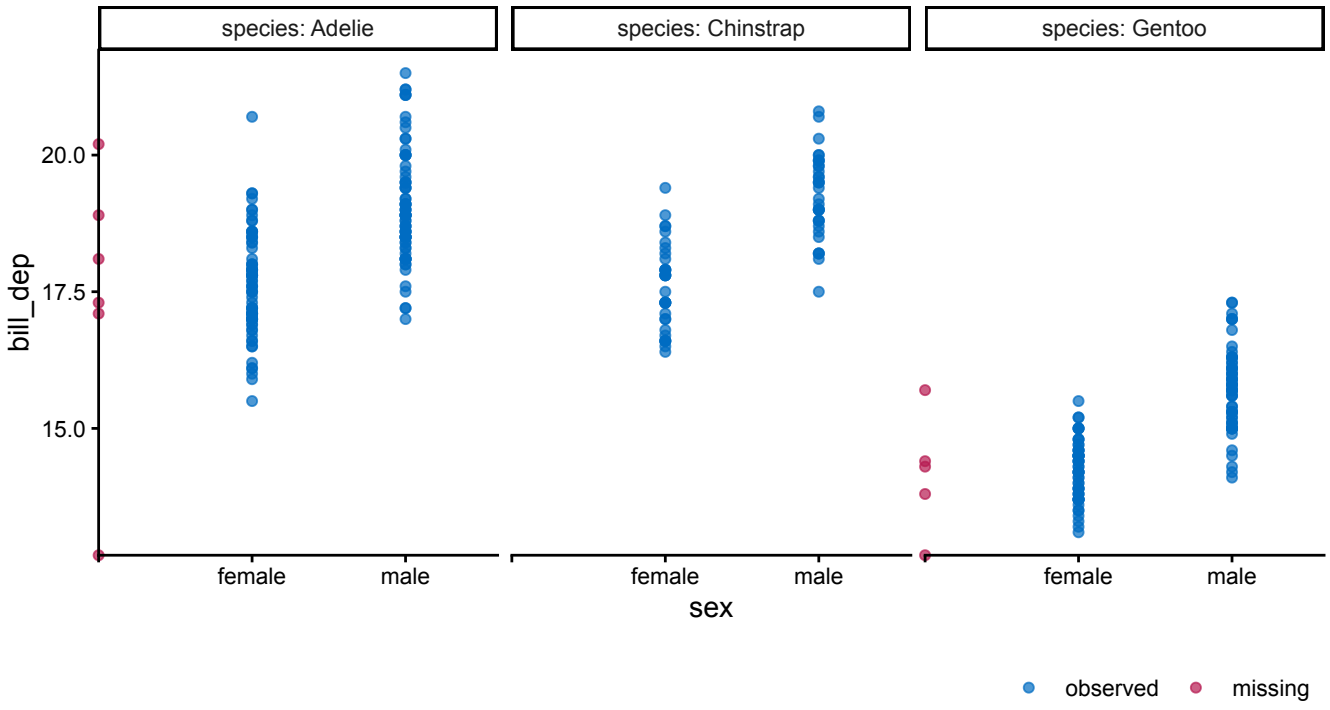
```
ggmice(penguins, aes(x = sex, y = bill_dep / 2.54)) +
  geom_point() +
  labs(y = "Bill depth (inches)")
```



Visualization of bill depth (in inches) by sex, showing missing values in red on the axis lines. Observed values for bill depth with missing data for sex are plotted on the vertical axis. The red point at the intersection of the axes indicates that at least one penguin has neither bill depth nor sex observed.

Another benefit of `ggplot` objects is that they may be adjusted using layers. If we would be interested in the sex differences in bill length between the penguin species, we can just add facets based on a clustering variable with:

```
ggmice(penguins, aes(x = sex, y = bill_dep)) +
  geom_point() +
  facet_wrap(~ species, labeller = label_both)
```



Visualization of bill depth by sex, split by penguin species. Observed values for bill depth with missing data for sex are plotted on the vertical axes. The red point at the intersection of the axes indicates that at least one penguin has neither bill depth nor sex observed.

Building imputation models

Imputation with `mice` requires an imputation model for each incomplete variable, consisting of an imputation method and imputation model predictors. These methods and predictors are either assigned implicitly in the `mice()` call, or supplied by the user as a methods vector and predictor matrix. The defaults in `mice()` are to assign an imputation method based on the column type of each incomplete variable, and to use all other variables as imputation model predictors.

The methods vector specifies an imputation method per variable. The default methods vector can be created using:

```
meth <- make.method(penguins)
meth
#>     species      island    bill_len    bill_dep flipper_len
#>          ""          ""       "pmm"       "pmm"       "pmm"
#>   body_mass         sex        year
#>       "pmm"    "logreg"          ""
```

In the default methods vector, imputation methods are based on column type, e.g., numeric data columns get assigned the semi-parametric imputation method 'predictive mean matching' (`pmm`), whereas dichotomous variables will be imputed using logistic regression.

The predictor matrix determines which variables will be used as predictors for imputing the incomplete variables. The default predictor matrix can be created with:

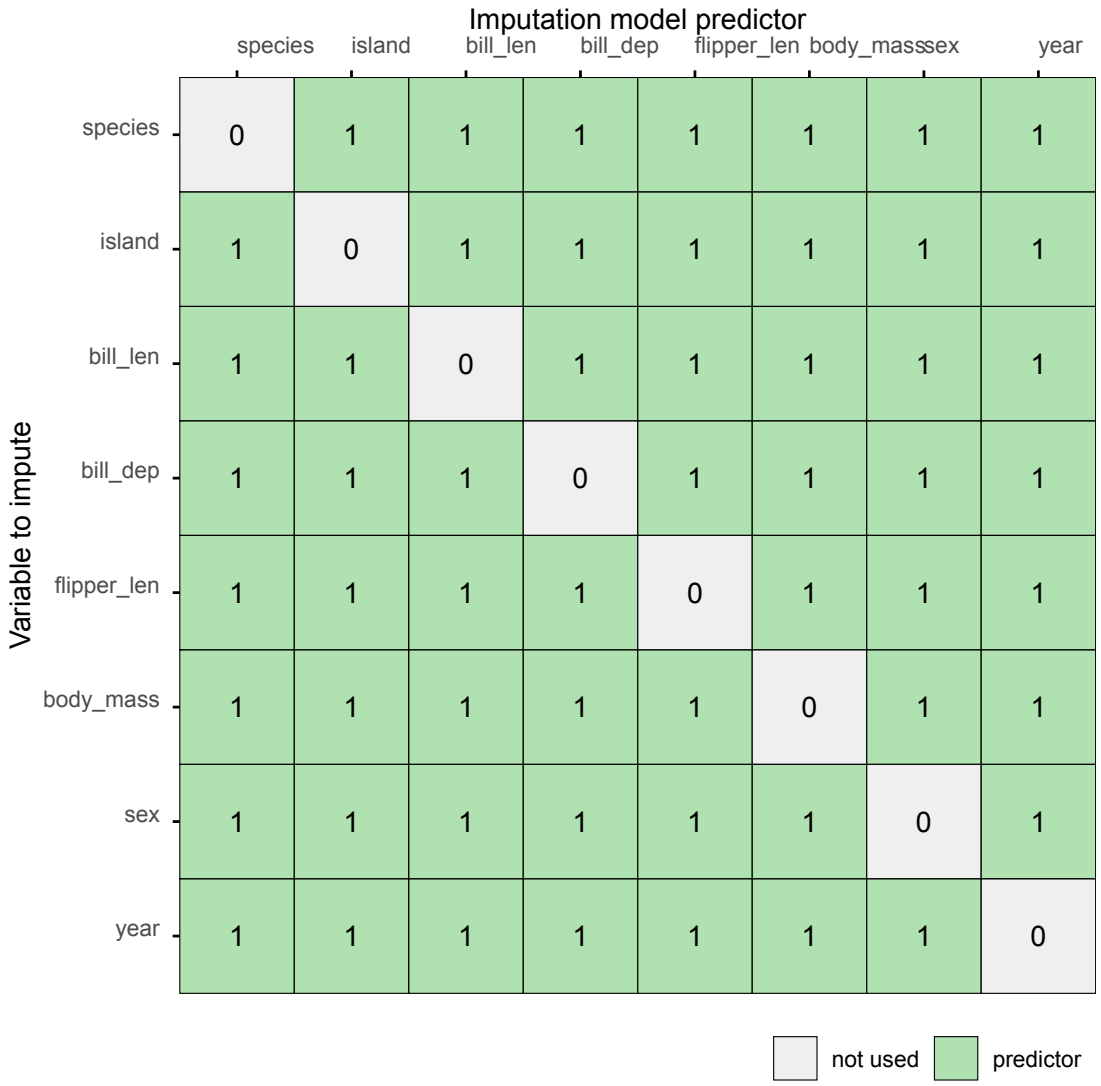
```
pred <- make.predictorMatrix(penguins)
pred
#>             species island bill_len bill_dep flipper_len
#> species           0      1        1        1           1
#> island            1      0        1        1           1
#> bill_len          1      1        0        1           1
#> bill_dep          1      1        1        0           1
#> flipper_len       1      1        1        1           0
#> body_mass         1      1        1        1           1
#> sex               1      1        1        1           1
#> year              1      1        1        1           1
#>             body_mass sex year
#> species             1   1    1
#> island              1   1    1
#> bill_len            1   1    1
#> bill_dep            1   1    1
#> flipper_len         1   1    1
#> body_mass           0   1    1
#> sex                 1   0    1
#> year                1   1    0
```

In the predictor matrix, rows represent variables to impute, and columns are potential imputation model predictors. By default, each variable is used as imputation model predictor for all other variables.

plot_pred()

The function `plot_pred()` displays `mice` predictor matrices, optionally paired with imputation methods. To create a predictor matrix plot, supply a predictor matrix via the `data` argument:

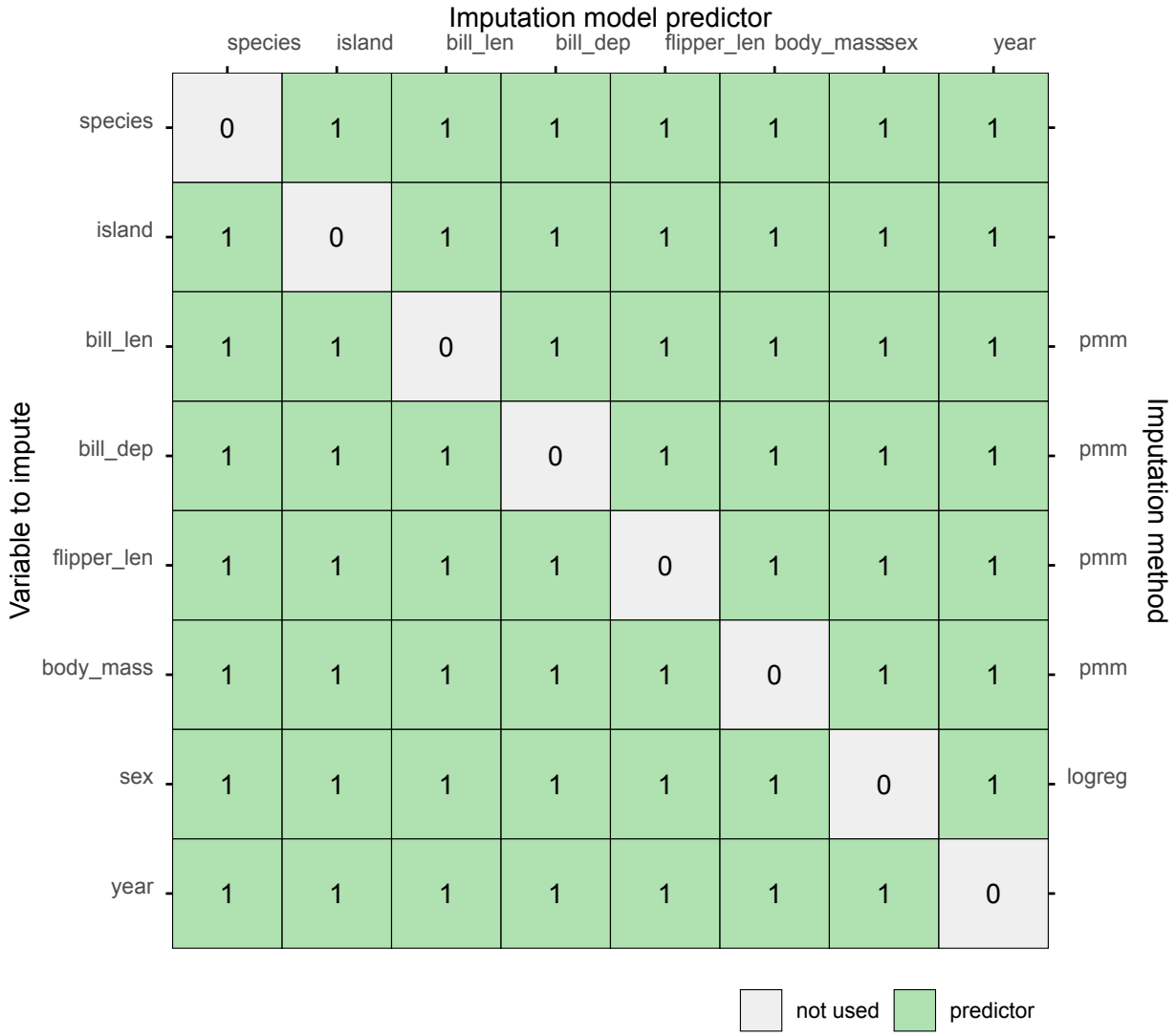
```
plot_pred(pred)
```



Predictor matrix with `mice` defaults.

Optional arguments may be added to the function call. For example, to show the full imputation model per incomplete variable, supply the methods vector via the optional argument `meth` :

```
plot_pred(pred, meth = meth)
```



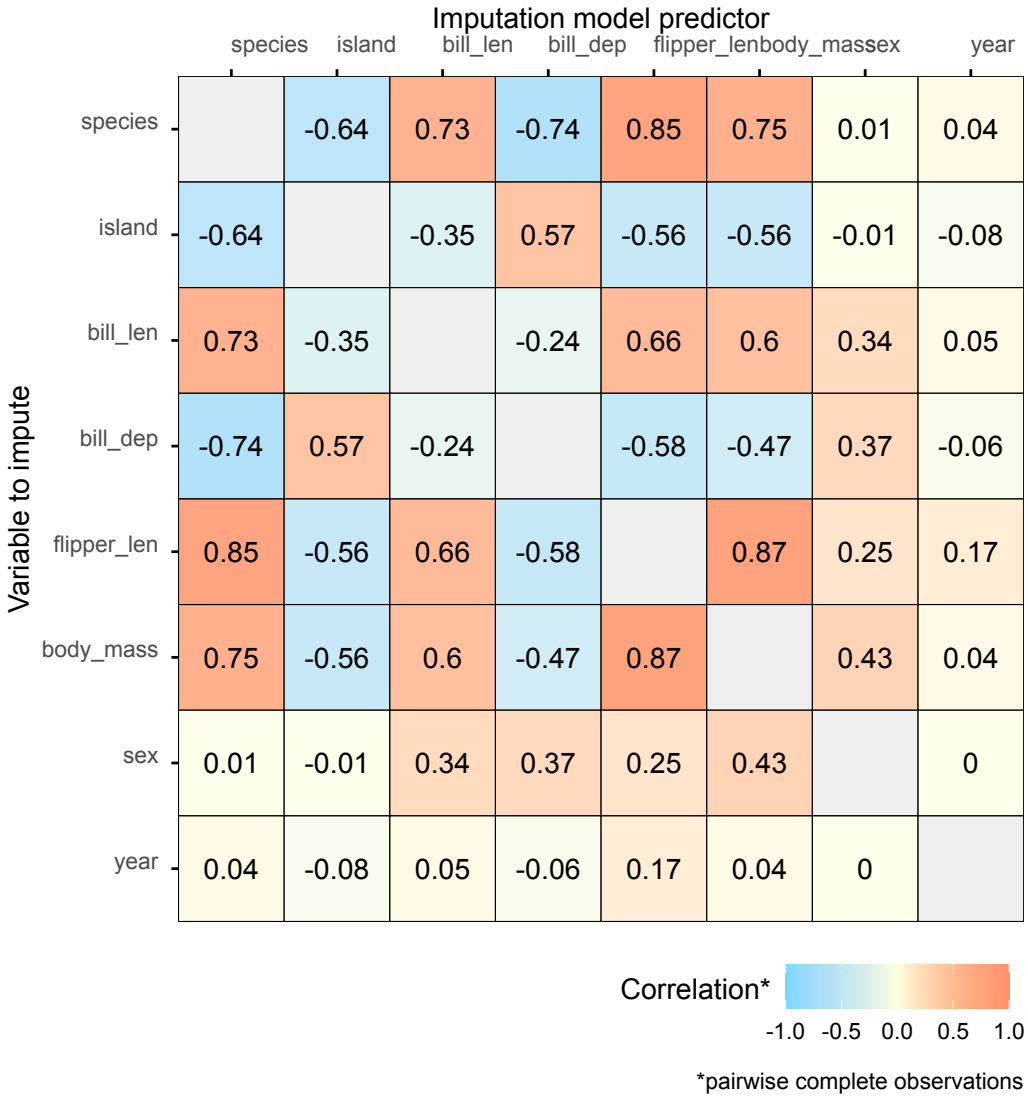
Predictor matrix with `mice` defaults including imputation methods.

Please refer to the function documentation for details.

`plot_corr()`

The function `plot_corr()` can be used to investigate relations between variables for the development of imputation models. The function requires incomplete dataset (via the argument `data`). All other arguments are optional, for example to add textual labels of the estimated correlations (via the `label` argument).

```
plot_corr(penguins, label = TRUE)
```

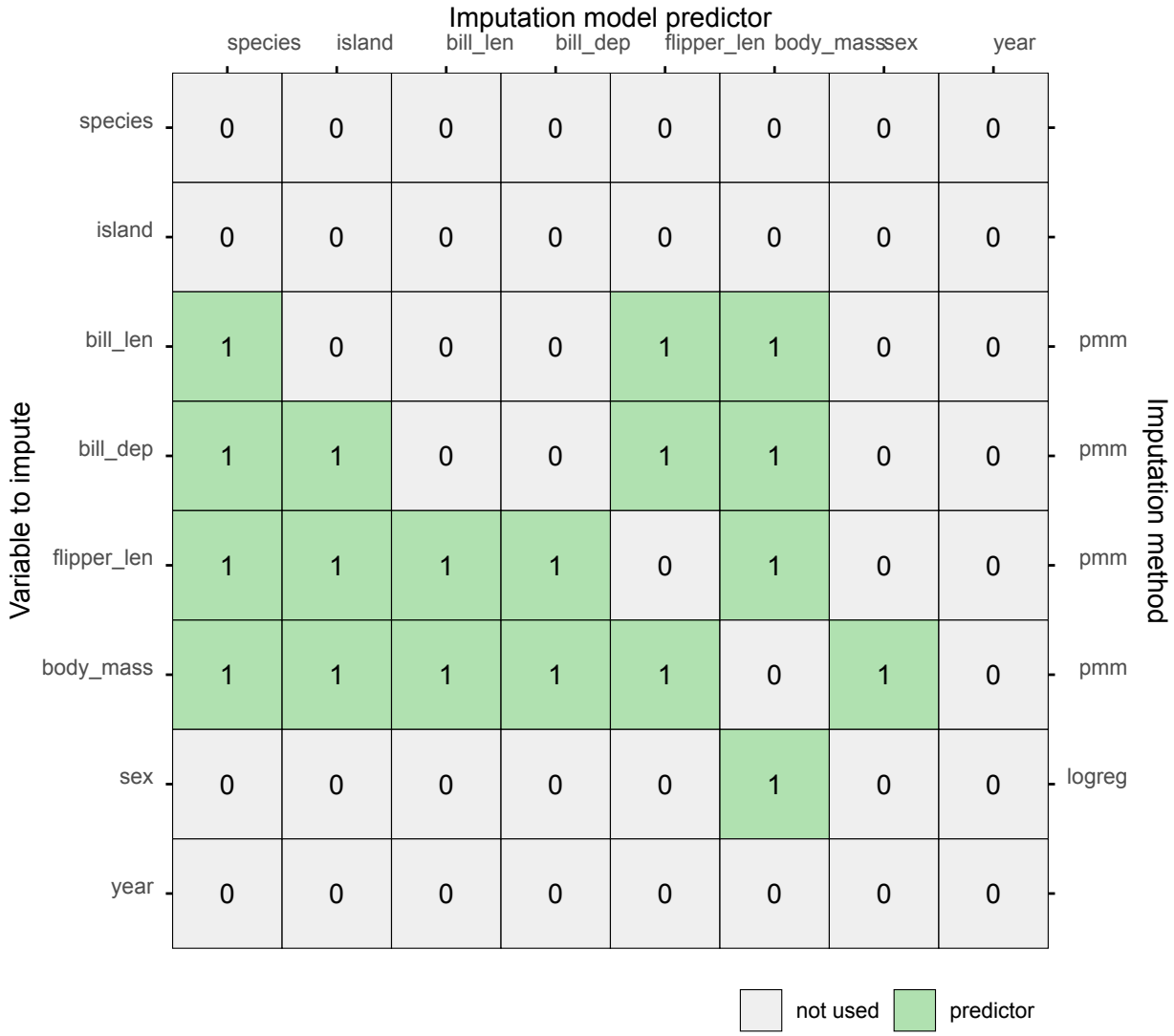


Visualization of bivariate correlations.

Based on these correlations, we can evaluate whether we have included all relevant imputation model predictors in the predictor matrix. With large datasets, for example, we might want to prune the predictor matrix, only to include the most relevant imputation model predictors.

The `quickpred()` function in `mice` selects imputation model predictors based on associations in the data. The function calculates correlations both with pairwise complete observations, as well as with the missingness indicators. Any potential imputation model predictor that surpasses a certain set threshold on either one of the two correlations, is selected into the predictor matrix.

```
pred <- quickpred(penguins, mincor = 0.4)
plot_pred(pred, method = meth)
```



Predictor matrix plot after pruning the predictor matrix.

The result is a pruned predictor matrix, that only includes imputation model predictors with a strong linear associations with the incomplete variables or missingness indicators.

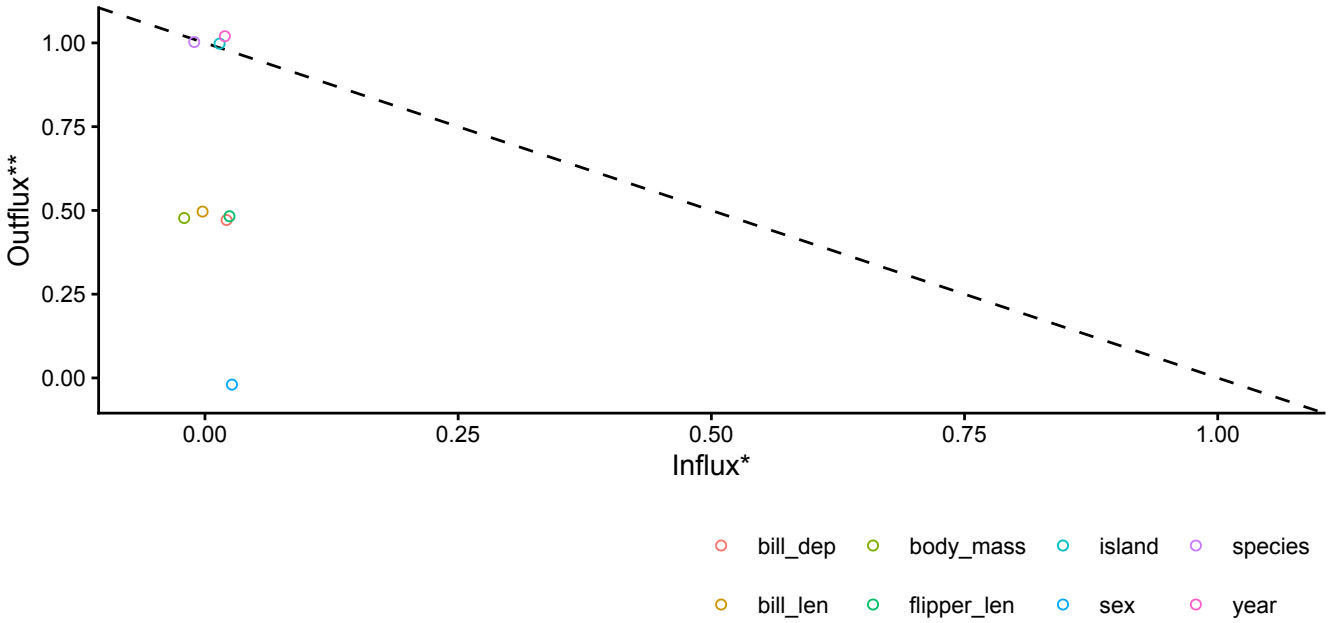
If we want to visualize this associations between observed data and missingness indicators, we can use an influx-outflux plot.

`plot_flux()`

The `plot_flux()` function produces an influx-outflux plot. The influx of a variable quantifies how well its missing data connect to the observed data on other variables. The outflux of a variable quantifies how well its observed data connect to the missing data on other variables. In general, higher influx and outflux values are preferred when building imputation models.

The plotting function requires an incomplete dataset (argument `data`), and takes optional arguments to adjust e.g., the legend and axis labels:

```
plot_flux(penguins, label = FALSE)
```



*connection of a variable's missingness indicator with observed data on other variables

**connection of a variable's observed data with missing data on other variables

Influx-outflux plot, showing connections between observed data and misisngness indicators. For example, `species`, `island`, and `year` have high outflux values and low influx, indicating that these observed variables may be quite informative for imputing incomplete variables.

For details, see the function documentation.

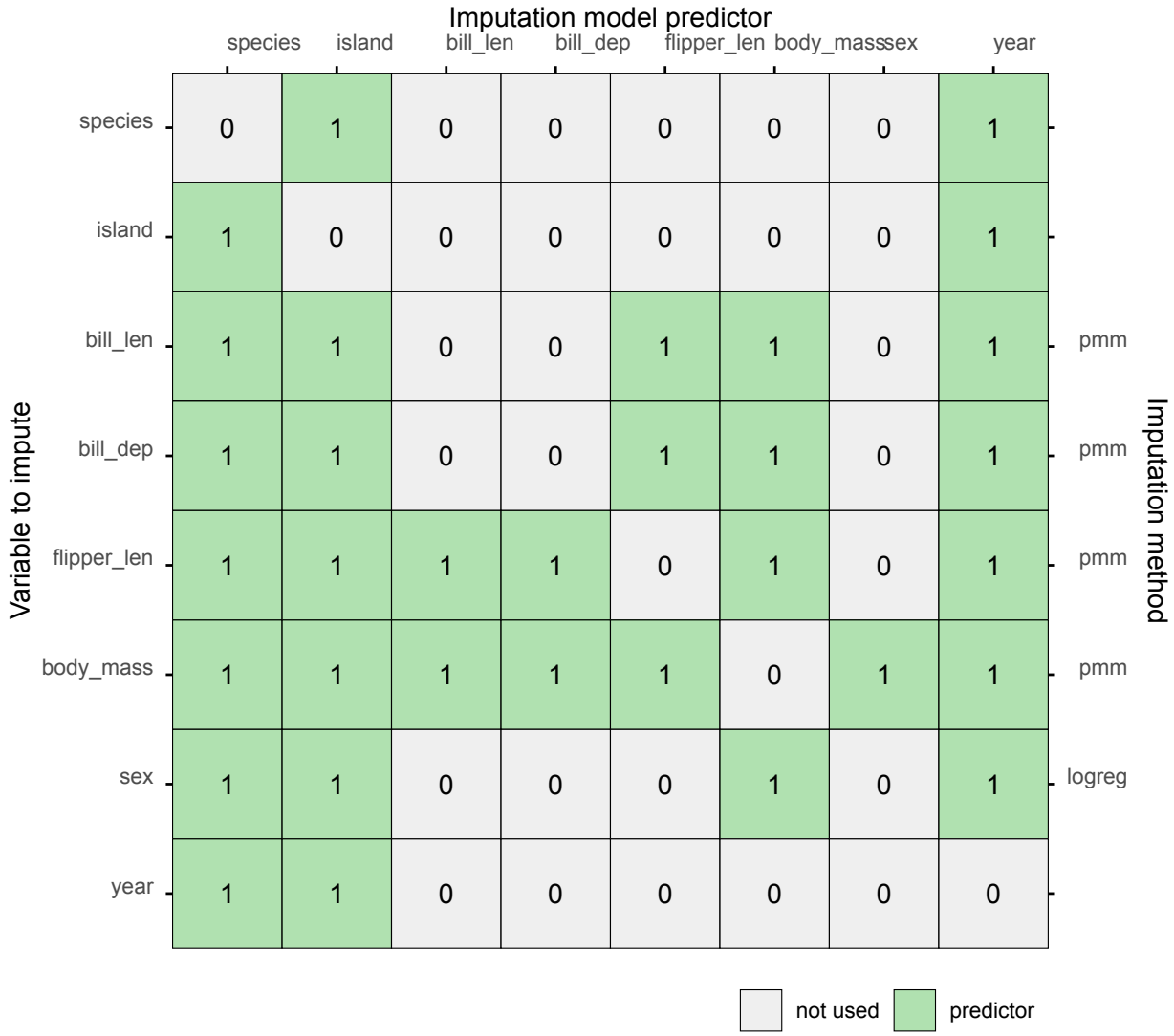
We can use this information to adjust the imputation models, via editing the predictor matrix. Based on the high outflux values, we add `species`, `island`, and `year` to the imputation models for all variables.

```
pred[, c("species", "island", "year")] <- 1
```

Subsequently, we can remove any variable that now became an imputation model predictor for itself:

```
diag(pred) <- 0
```

```
plot_pred(pred, method = meth)
```



Predictor matrix plot after adding high outflux variables to the predictor matrix.

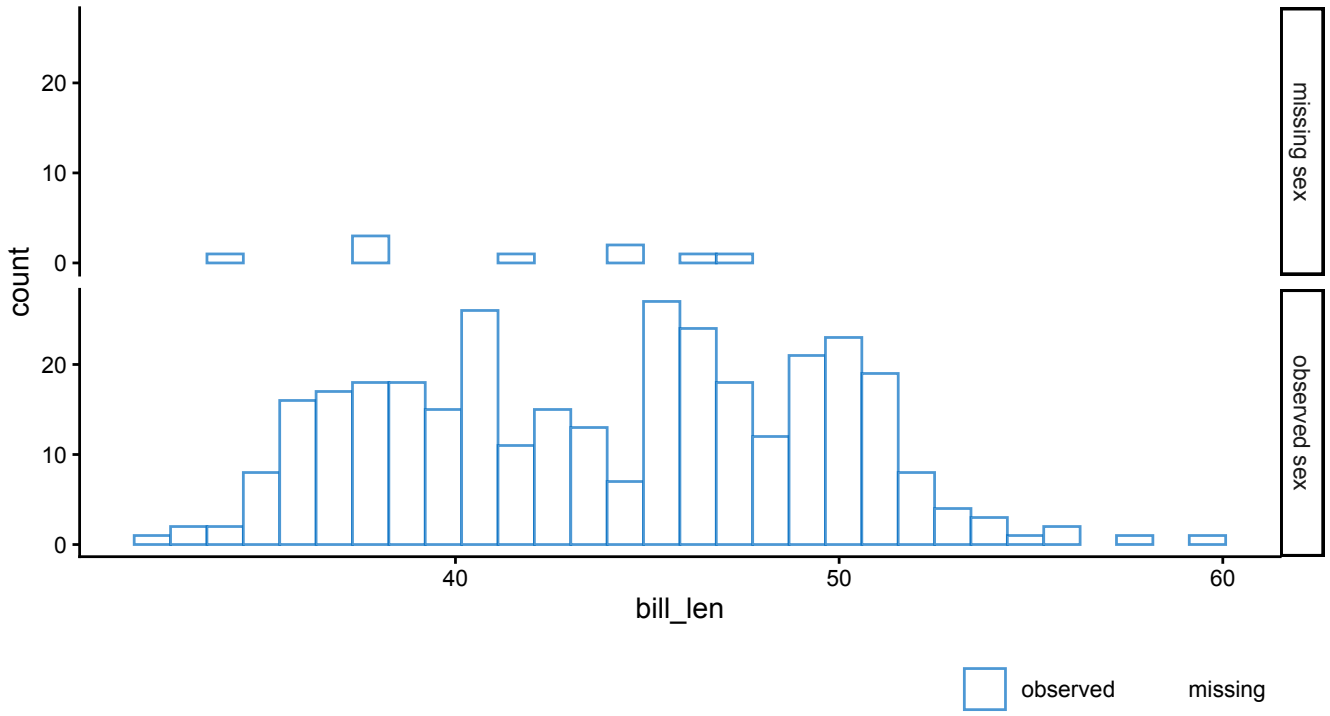
Another way to evaluate the associations with the missingness indicators is using `ggmice()`.

ggmice()

We can use `ggmice()` to visualize associations between observed and missing data, by applying the function to incomplete data and adding faceting based on a missingness indicator. This may help further explore the missingness mechanisms in the incomplete data.

For example, to investigate which variables may be informative for imputing the incomplete variable `sex`, we can create a graph split by the missingness indicator of `sex`. To visualize the association between the distribution of `bill_len` and the missingness indicator of `sex` we use a faceted design:

```
ggmice(penguins, aes(bill_len)) +
  geom_histogram(fill = "white") +
  facet_grid(factor(
    is.na(sex),
    levels = c(TRUE, FALSE),
    labels = c("missing sex", "observed sex")
    ) ~ .)
```



Distribution of observed bill length (in cm) split by the missingness indicator of the variable `sex`. Since the missingness indicator itself is always observed, this plot does not show any missing data.

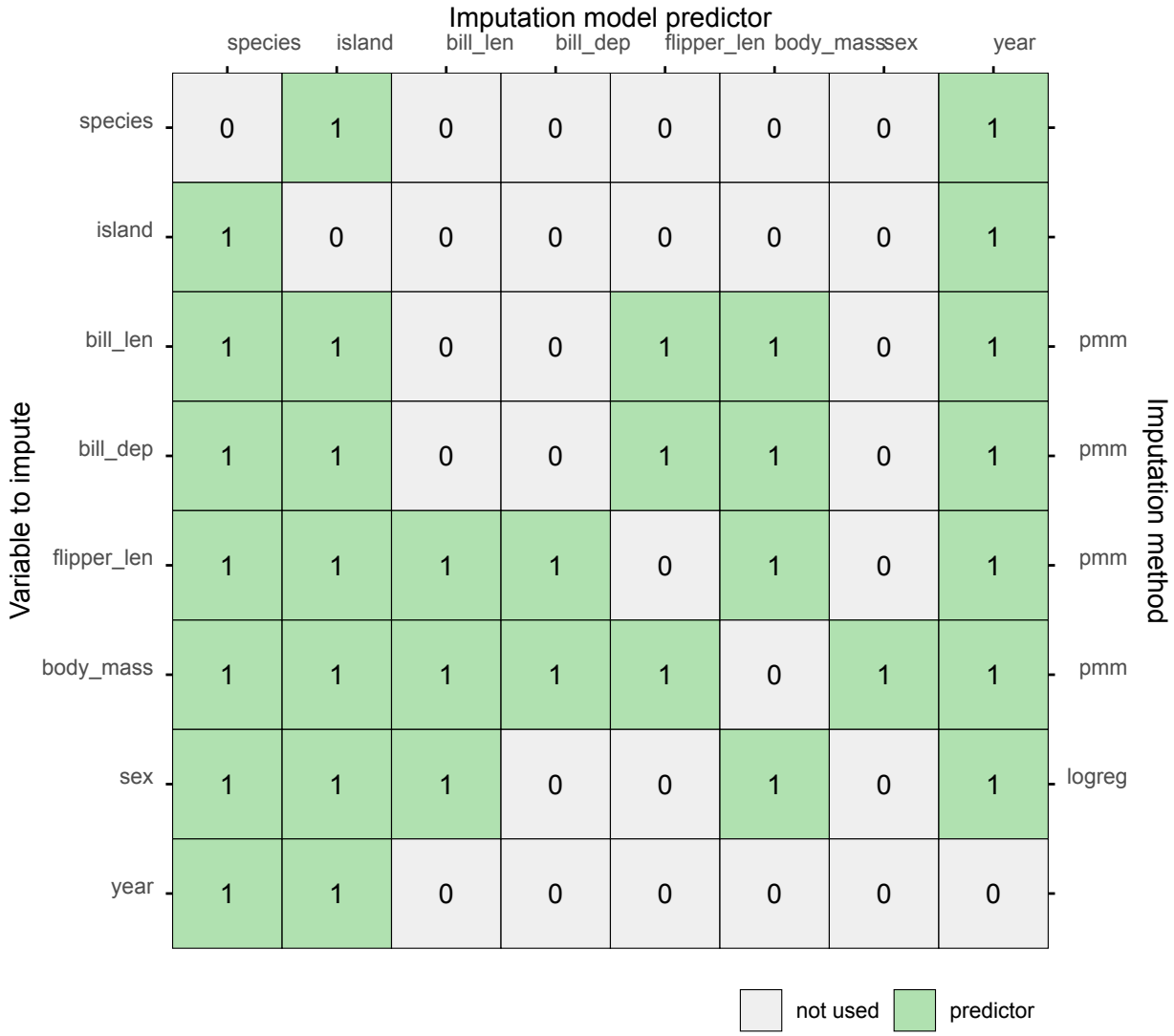
We can see there are some differences in the distribution of `bill_len` based on the missingness indicator of `sex`, which may imply that `bill_len` is informative for imputing `sex`.

We add this variable to the imputation model of `sex` by assigning `bill_len` as imputation model predictor in the predictor matrix:

```
pred["sex", c("bill_len", "species")] <- 1
```

Since we had already added some imputation model predictors, our predictor matrix now looks as follows:

```
plot_pred(pred, method = meth)
```



Predictor matrix plot after editing the predictor matrix.

This yields a final version of the imputation models, that we will use to impute the data.

Evaluating imputations

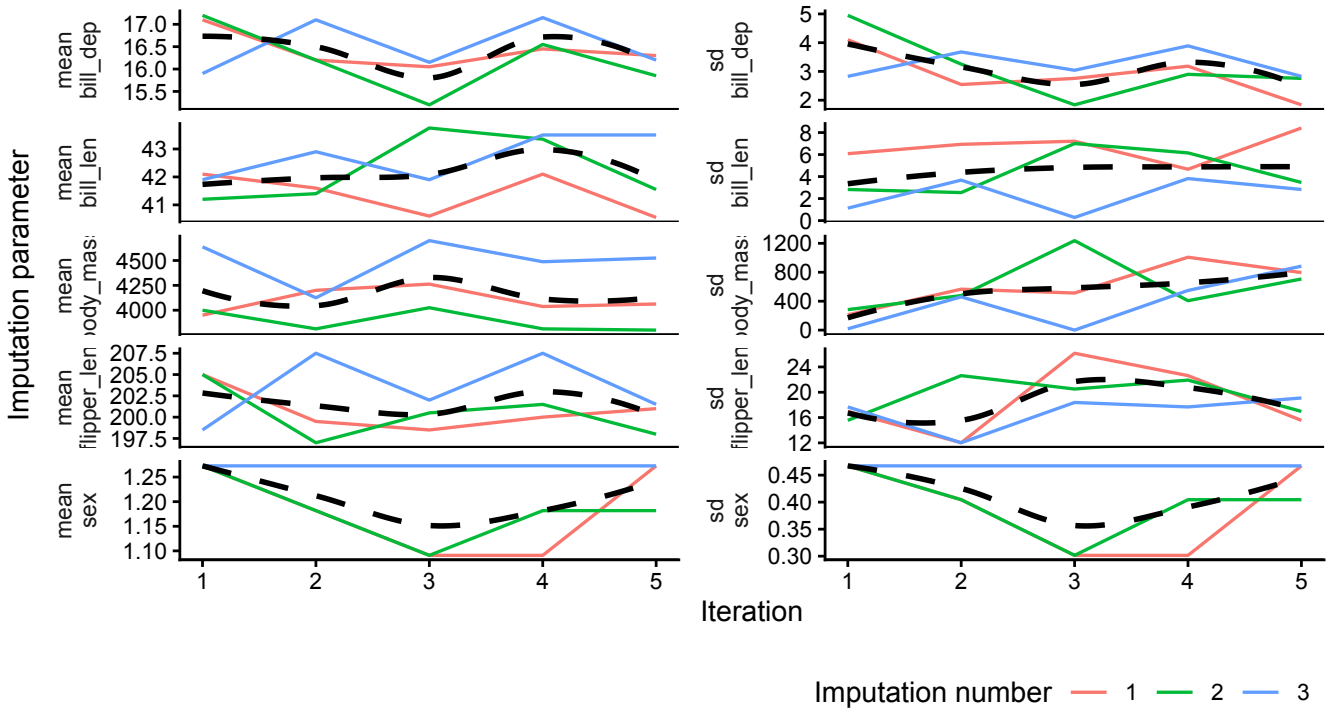
Run the imputation algorithm on the incomplete dataset, with the imputation models we built (i.e., the predictor matrix and methods vector), with three imputations.

```
imp <- mice(
  penguins,
  pred = pred,
  method = meth,
  m = 3,
  print = FALSE
)
```

plot_trace()

The function `plot_trace()` plots the trace lines of the MICE algorithm for convergence evaluation. The only required argument is `data` (to supply a `mice::mids` object). Optional arguments such as `trend` (to add a trend line that facilitates interpretation) are described in the function documentation.

```
plot_trace(imp, trend = TRUE)
```

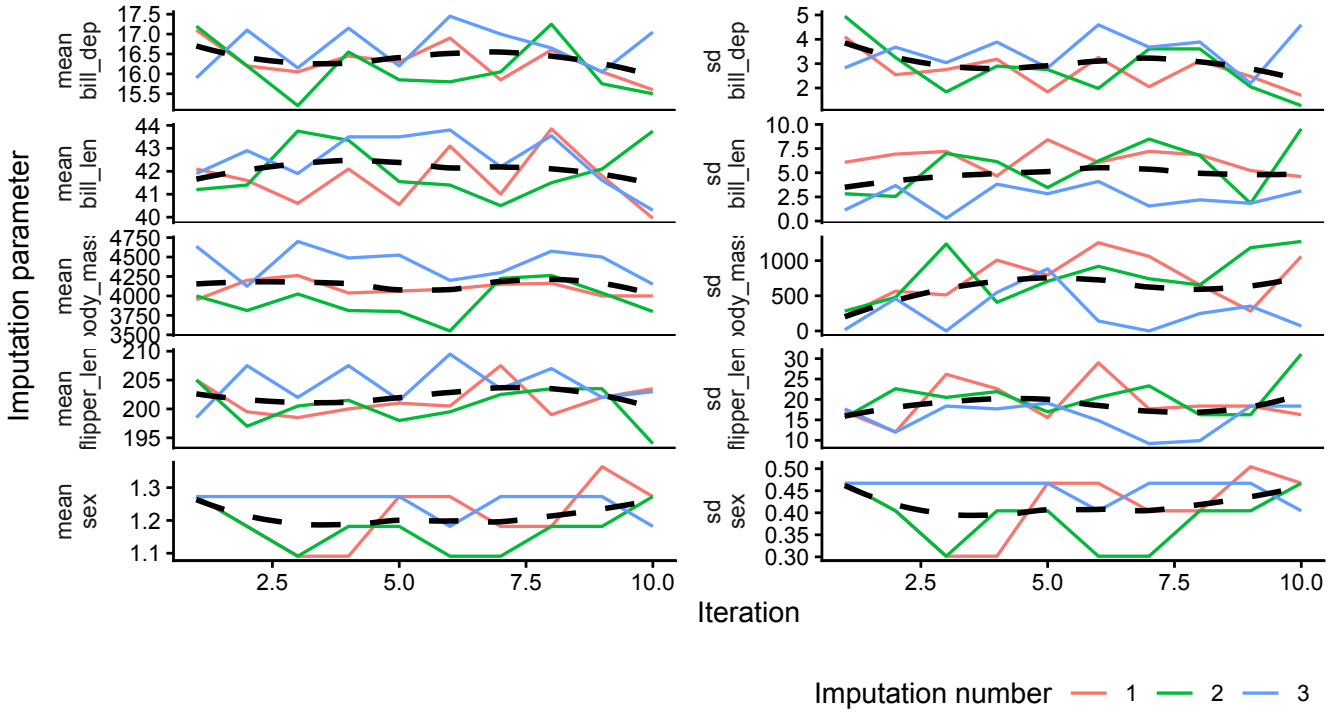


Trace plots for all variables.

For algorithmic convergence, the trace plot lines should be stationary (non-trending) and mixing (intermingling nicely). If we are unsure of the algorithmic convergence, we can add iterations to the imputation algorithm and re-evaluate:

```
imp <- mice.mids(imp, maxit = 5, print = FALSE)
```

```
plot_trace(imp, trend = TRUE)
```



Trace plots for all variables after adding iterations.

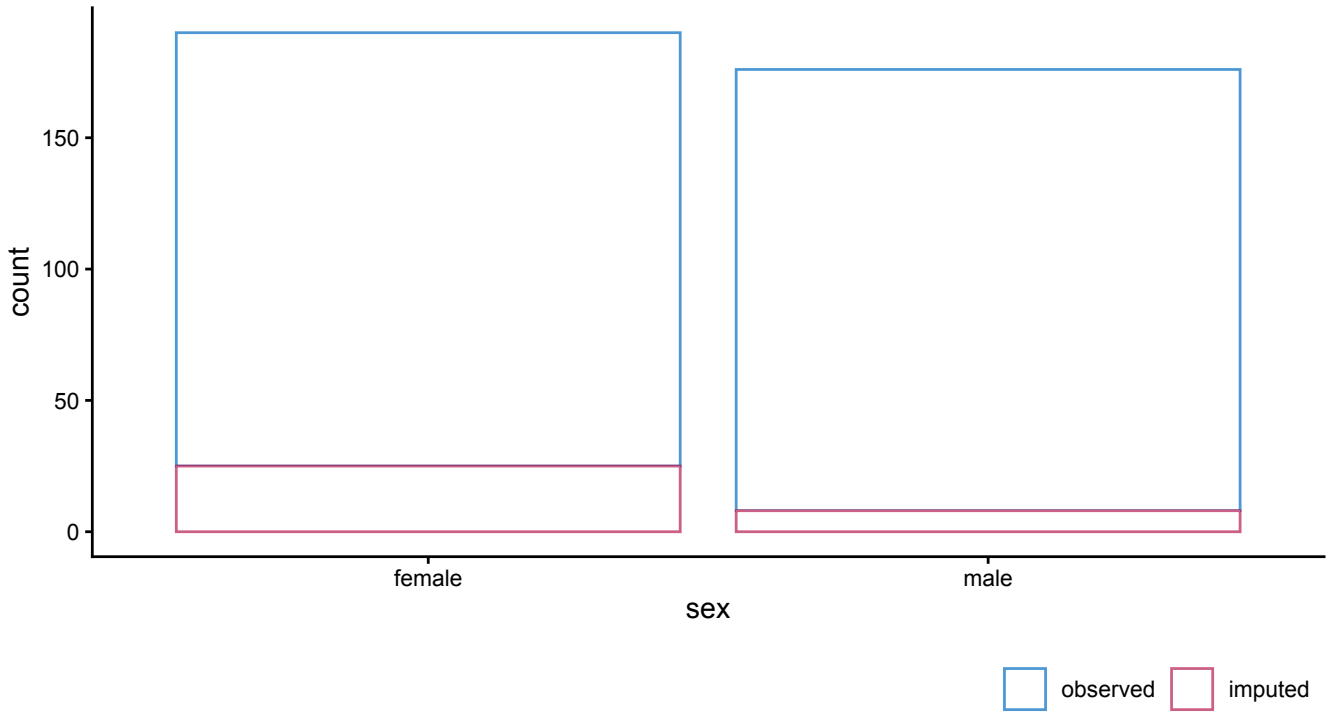
We can supplement the inspection of the traceplots with evaluations of the imputed values themselves. We can use `ggmice()` to create visual diagnostics for the imputed variables.

ggmice()

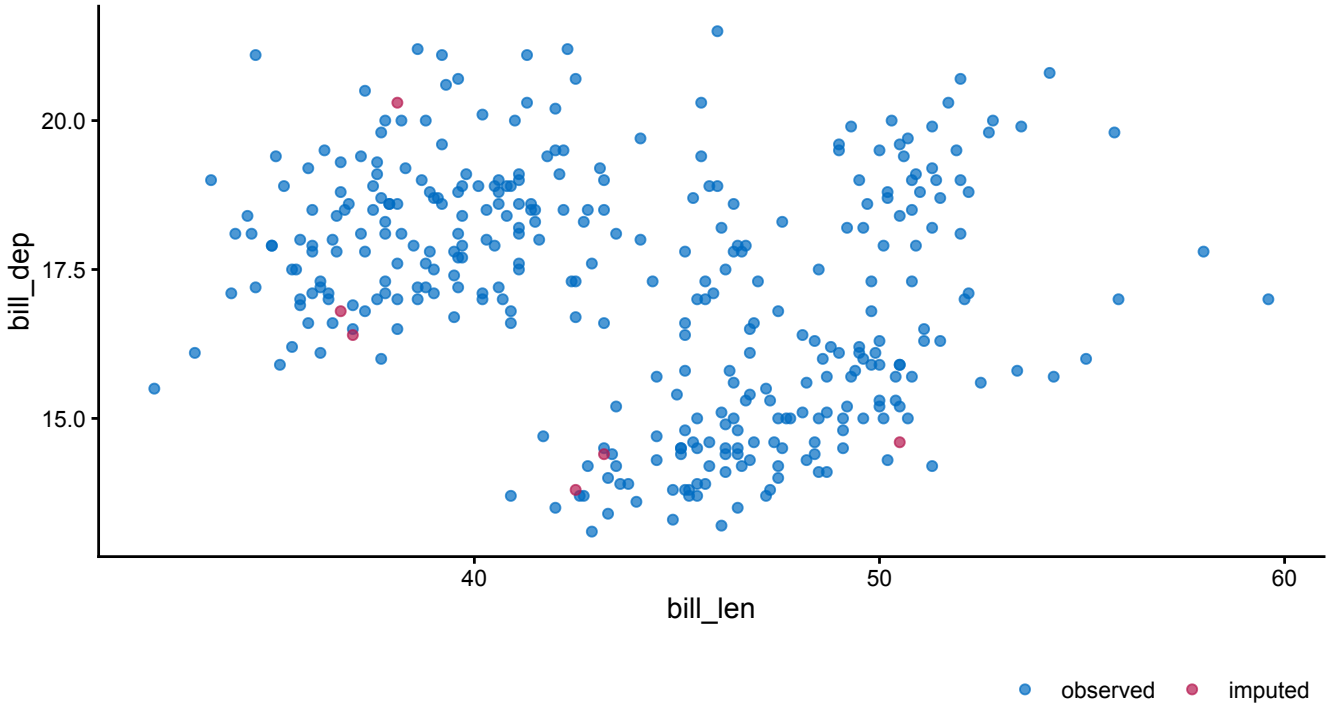
Plotting the imputed data can reveal unrealistic imputations or issues with the imputation models. Additionally, `ggmice()` allows for graphical comparisons between the incomplete and imputed data.

We can create the same plots as the ones on the incomplete data, but now on the imputed data:

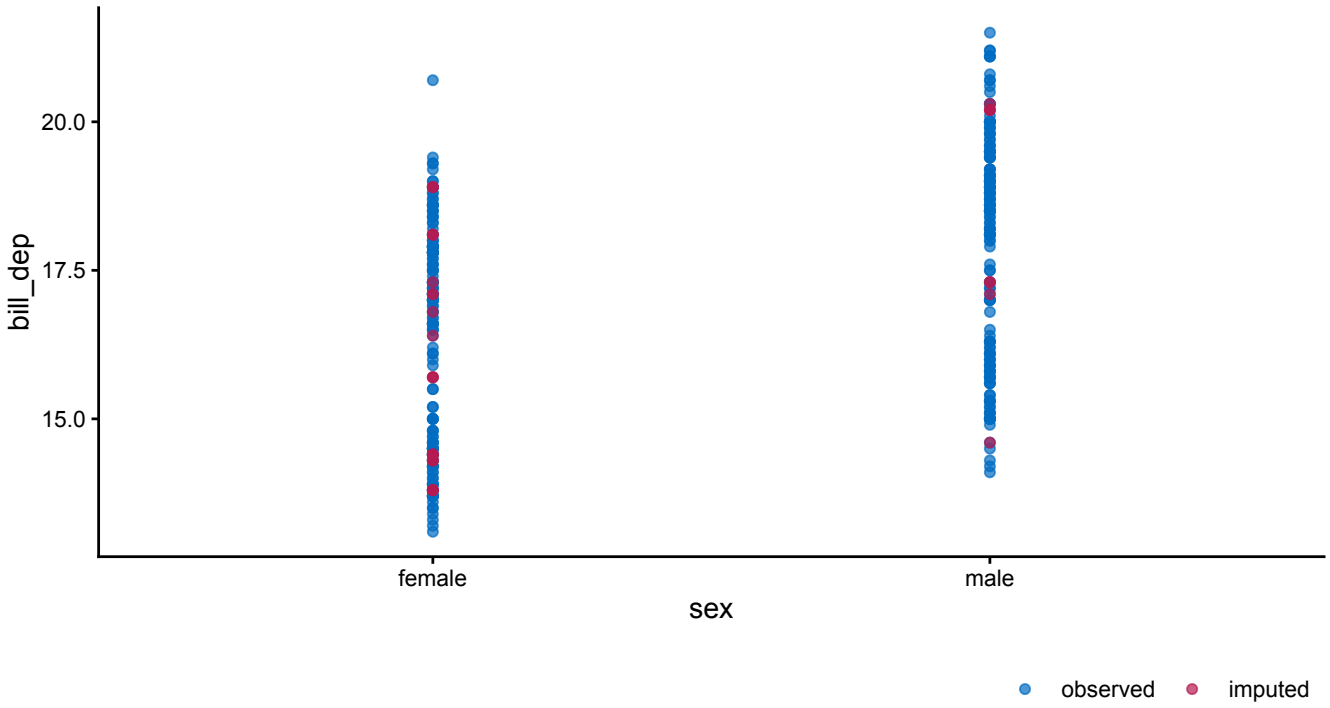
```
ggmice(imp, aes(x = sex)) +
  geom_bar(fill = " white")
```



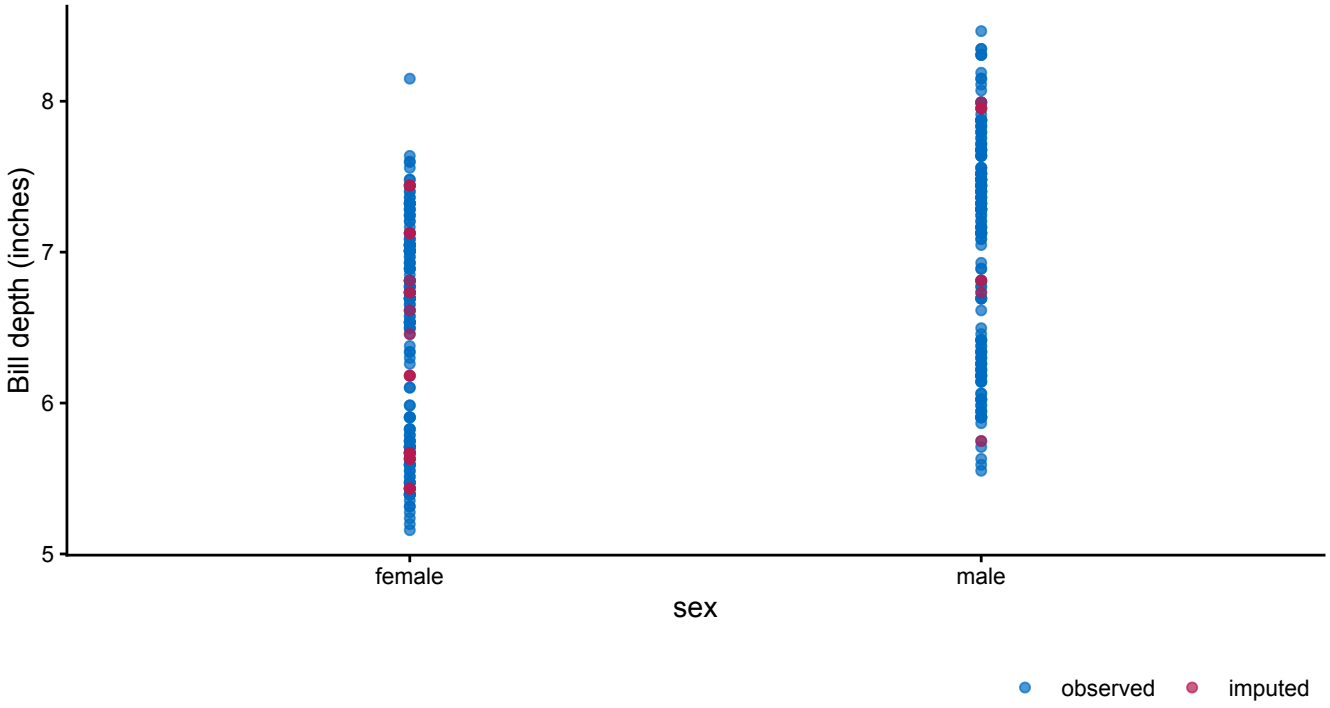

```
ggmice(imp, aes(x = bill_len, y = bill_dep)) +
  geom_point()
```



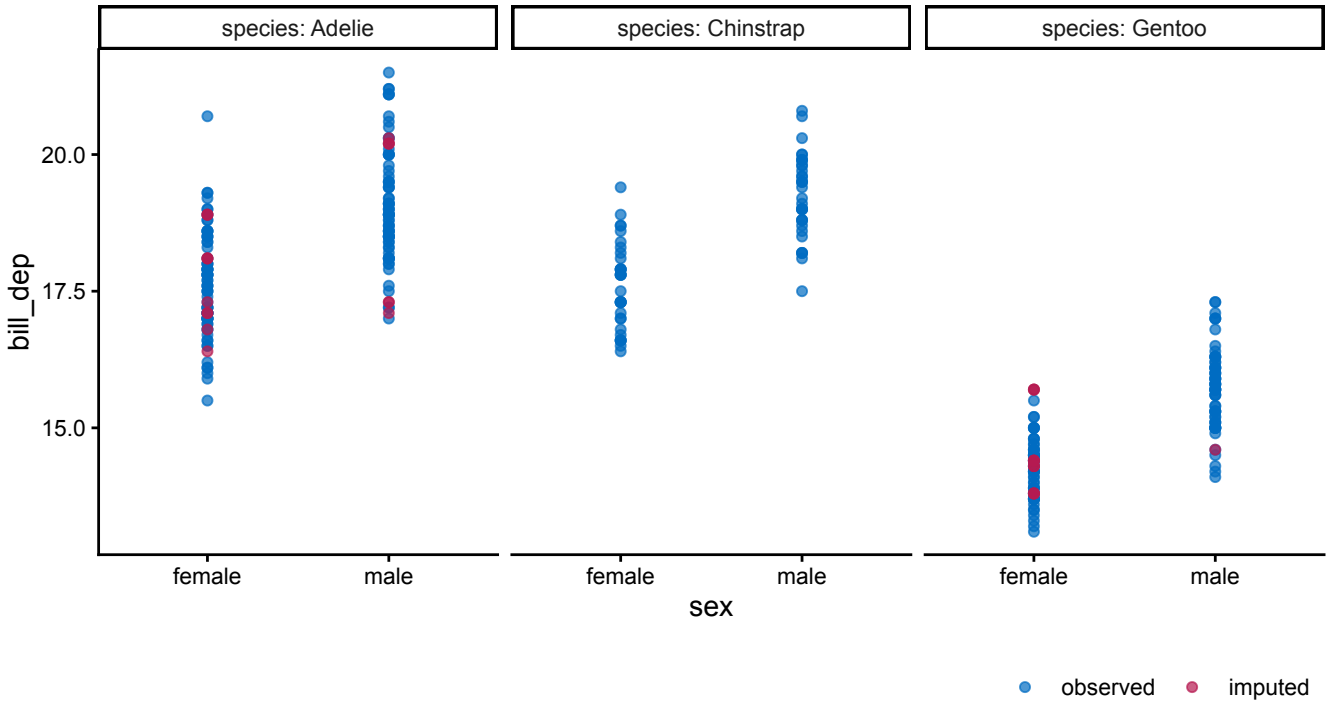

```
ggmice(imp, aes(x = sex, y = bill_dep)) +
  geom_point()
```




```
ggmice(imp, aes(x = sex, y = bill_dep / 2.54)) +
  geom_point() +
  labs(y = "Bill depth (inches)")
```



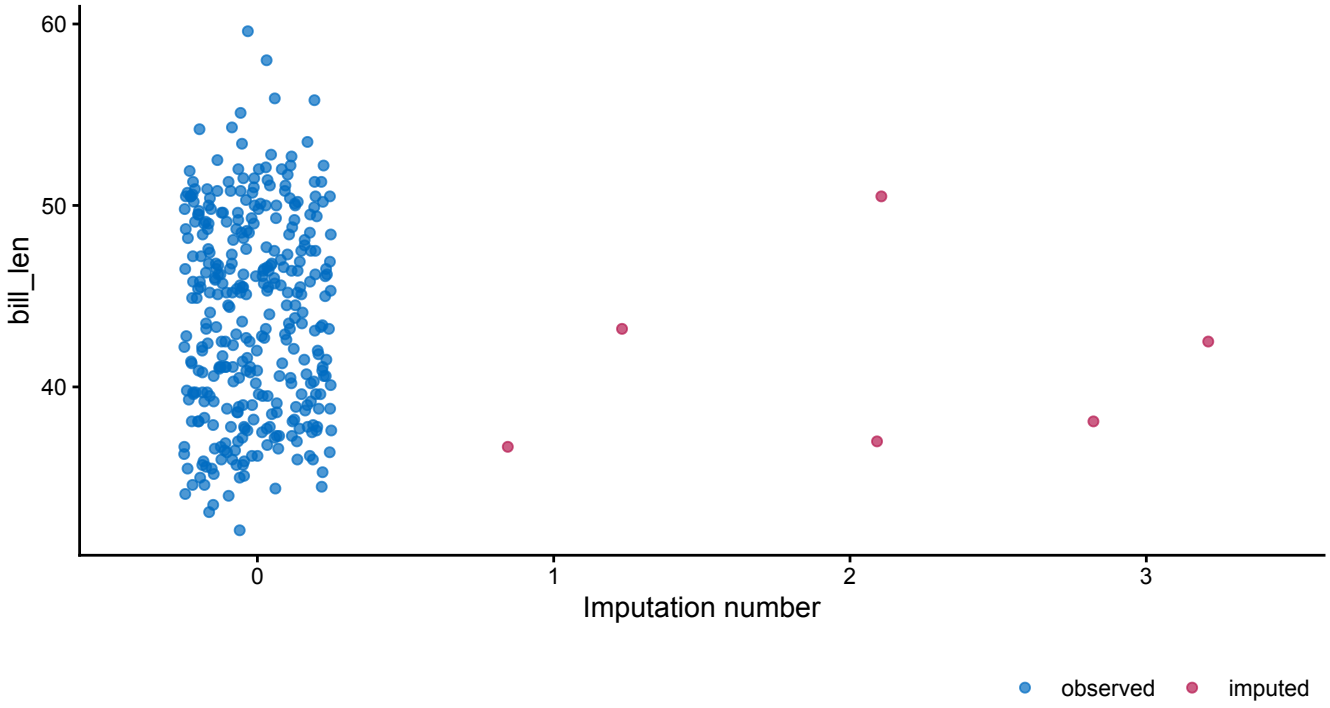

```
ggmice(imp, aes(x = sex, y = bill_dep)) +
  geom_point() +
  facet_wrap(~ species, labeller = label_both)
```



These figures show the observed data points once in blue, plus 3 imputed values in red for each missing entry.

In addition to recreating the graphs from the incomplete data exploration stage, it is also possible to use the imputation number as mapping variable in the plot. For example, we can create a stripplot of observed and imputed data with the imputation number `.imp` on the horizontal axis:

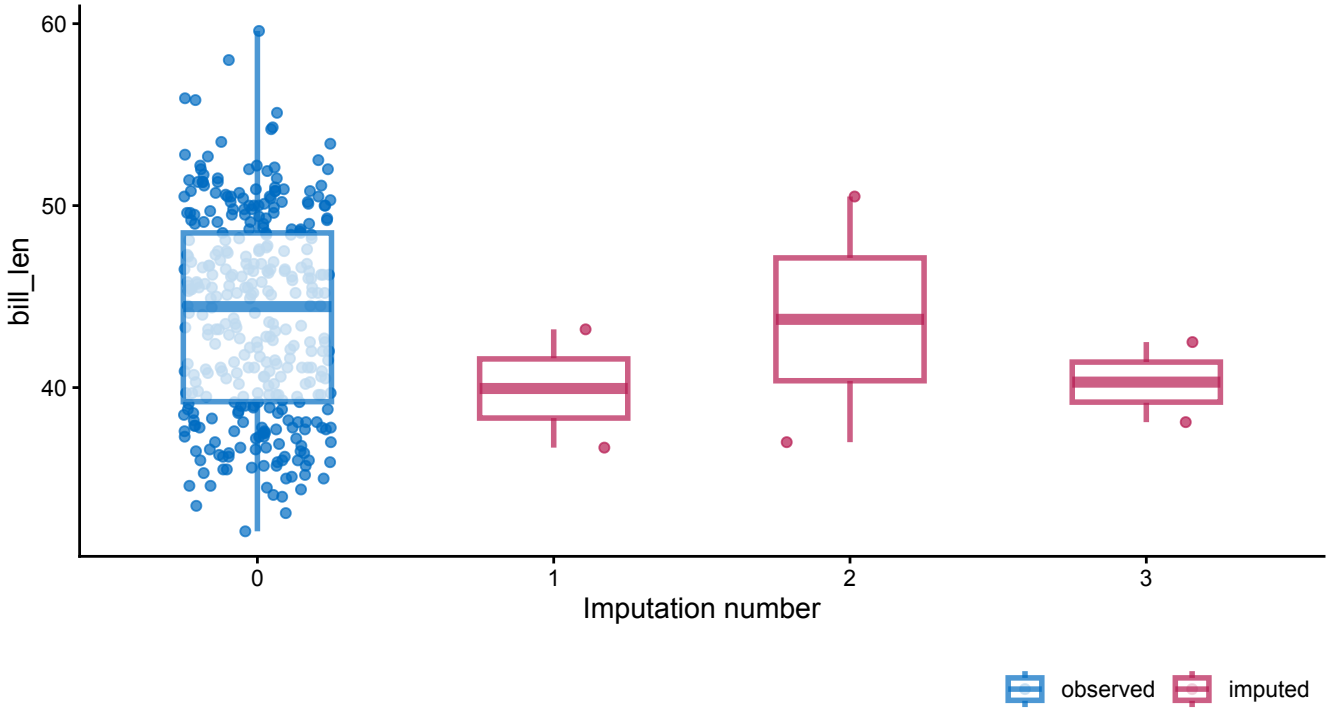
```
ggmice(imp, aes(x = .imp, y = bill_len)) +
  geom_jitter(height = 0, width = 0.25) +
  labs(x = "Imputation number")
```



Stripplot of bill length (in cm) by imputation.

A major advantage of `ggmice()` over the equivalent function `mice::stripplot()` is that `ggmice` allows us to add subsequent plotting layers, such as a boxplot overlay:

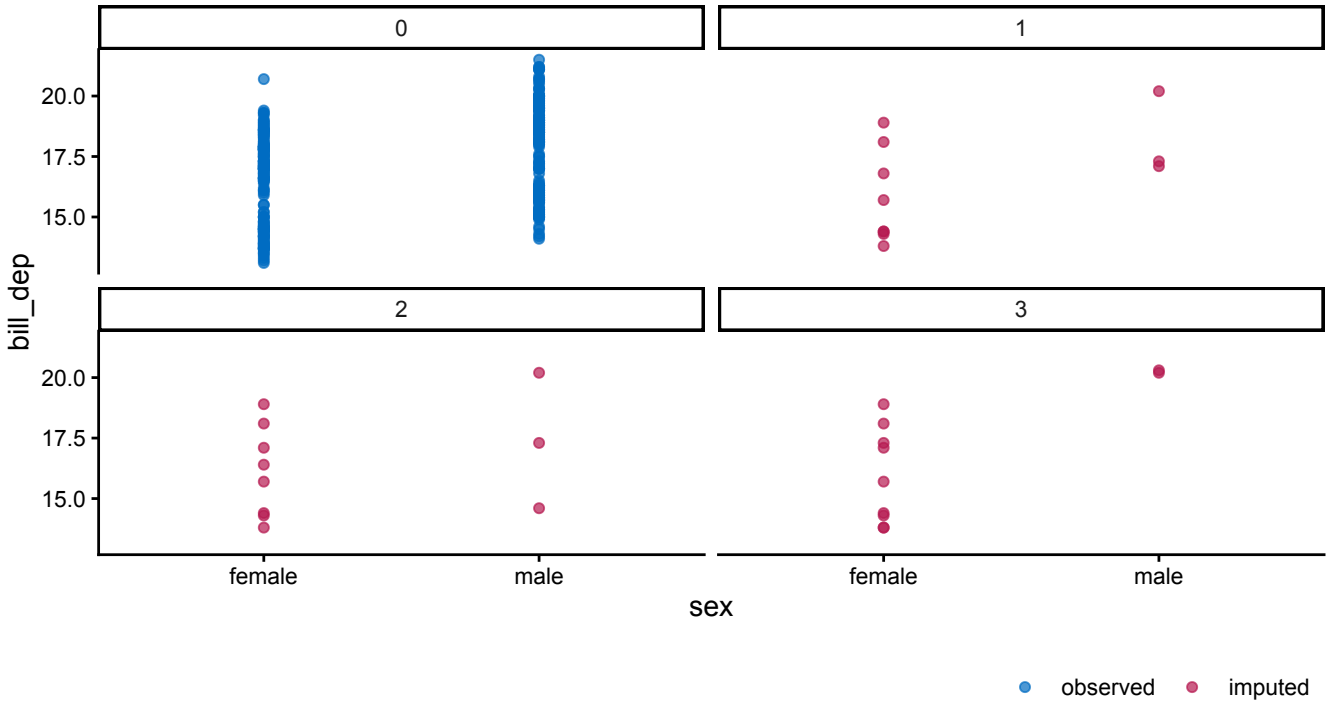
```
ggmice(imp, aes(x = .imp, y = bill_len)) +
  geom_jitter(height = 0, width = 0.25) +
  geom_boxplot(width = 0.5, linewidth = 1, alpha = 0.75, outlier.shape = NA) +
  labs(x = "Imputation number")
```



Stripplot combined with boxplot of bill length (in cm) by imputation.

Another advantage of the 'grammar of graphics' philosophy in `ggmice` is that we can use faceting to split plots by imputation number. This allows for further inspection of e.g., bivariate distributions within each imputation:

```
ggmice(imp, aes(x = sex, y = bill_dep)) +
  geom_point() +
  facet_wrap(~.imp)
```

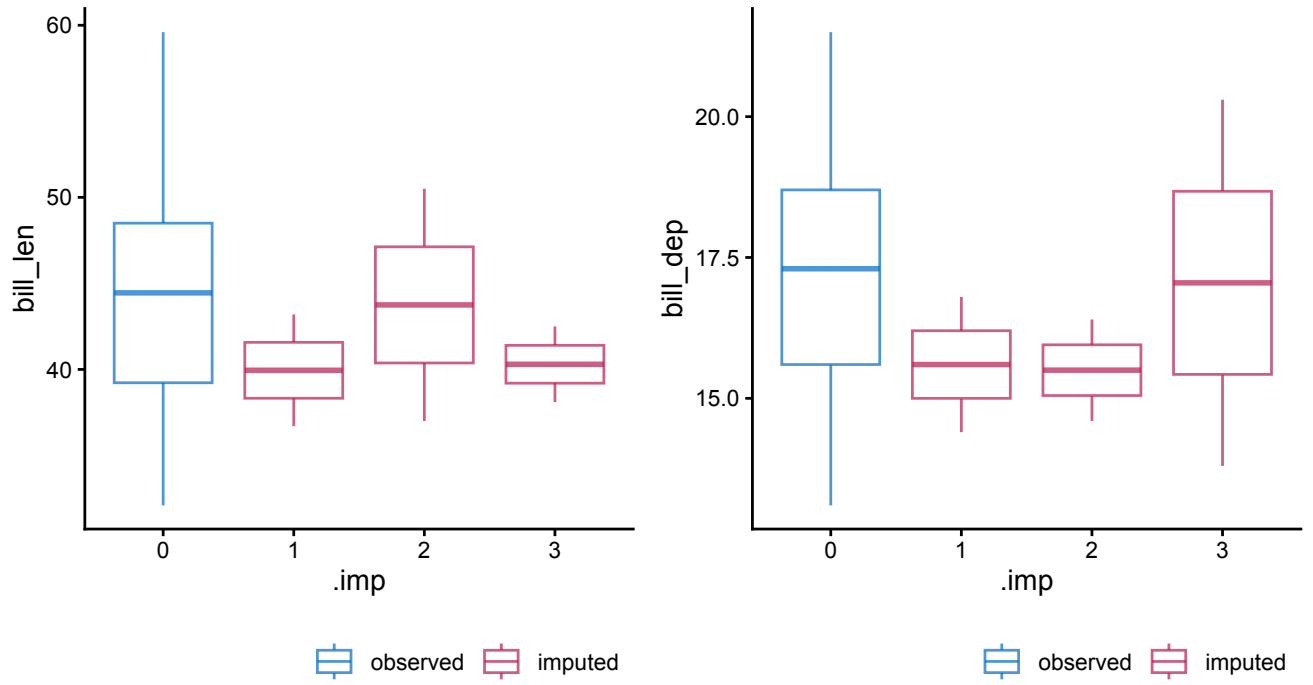


Scatterplots split by imputation.

When evaluating imputations, you may want to assess multiple variables at once. To create plots of multiple imputed variables, we can combine `ggmice` with the graphical formatting package `patchwork` and the functional programming package `purrr`.

You can combine figures with `patchworks`'s `wrap_plots()` function:

```
p1 <- ggmice(imp, aes(x = .imp, y = bill_len)) + geom_boxplot()
p2 <- ggmice(imp, aes(x = .imp, y = bill_dep)) + geom_boxplot()
patchwork::wrap_plots(p1, p2)
```



Combined boxplots.

And to plot many variables at once, we can use the `purrr` function `map()` (which works like a vectorized `for` loop). If we want to create stripplots of every imputed variable–and only imputed variables–we first need to know which variables were imputed. We generate a vector with the imputed variable names by extracting all variable names, and then indexing the variables where the number of imputed values is greater than zero:

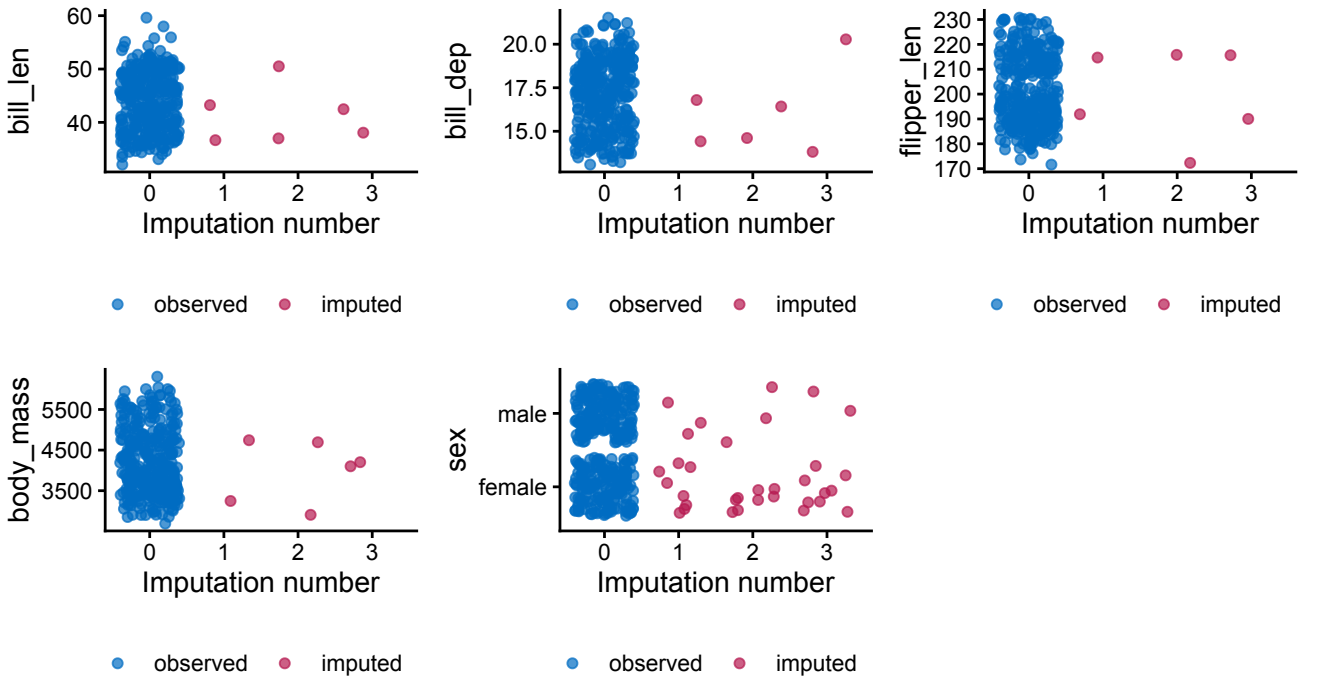
```
vrb_names <- names(imp$data)
imp_count <- colSums(imp$where)
vrb_imp <- vrb_names[imp_count > 0]
```

Subsequently, we can use functional programming to create a plot for each imputed variable:

```
plot_list <- purrr::map(vrb_imp, function(vrb){
  ggmice(imp, aes(x = .imp, y = .data[[vrb]])) +
    geom_jitter() +
    labs(x = "Imputation number")
})
```

And finally, we use `patchwork` to combine the plot into one object:

```
plot_list |>
  patchwork::wrap_plots()
```



Combined stripplots.

Take-aways

In this vignette, you have seen how `ggmice` can support each phase of a `mice` imputation workflow, from the first look at the incomplete data to evaluating the imputations.

In the exploration of the incomplete data, the `ggmice()` function allows you to visualize missing datapoints instead of silently discarding incomplete cases. Other missingness exploration functions, such as `plot_pattern()`, may help you understand the structure of the missing data problem and subsequently inform imputation model choices.

When building imputation models, `ggmice()` may be used to investigate associations between the observed data and missingness indicators. Other visualization functions that may aid imputation model building are `plot_pred()`, `plot_corr()`, and `plot_flux()`.

Applied to imputed data (`mids` objects), `ggmice()` visualizes observed and imputed values together, making it straightforward to compare the distributions before and after imputation. These graphs may also flag potential problems in the imputation model fit, such as implausible imputed values, or algorithmic non-convergence. Algorithmic convergence is a prerequisite for using `mice` imputations in any subsequent statistical analysis. Therefore, the `plot_trace()` function offers tools for visually diagnosing non-convergence.

Across these steps, an advantage of `ggmice` is that all visualizations are standard `ggplot` objects. This means you can adapt them to your own preferences and publication standards with the usual 'grammar of graphics' functionality.

Supporting information

This is the end of the vignette.

This document was generated using:

```
sessionInfo()
#> R version 4.5.0 (2025-04-11)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.6
#>
#> Matrix products: default
#> BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/ve
cLib.framework/Versions/A/libBLAS.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlap
ack.dylib;  LAPACK version 3.12.1
#>
#> locale:
#> [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
#>
#> time zone: Europe/Amsterdam
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods
#> [7] base
#>
#> other attached packages:
#> [1] pagedown_0.23     ggmice_0.1.1.9000 ggplot2_4.0.2
#> [4] mice_3.19.3
#>
#> loaded via a namespace (and not attached):
#>  [1] tidyselect_1.2.1   dplyr_1.2.0        farver_2.1.2
#>  [4] S7_0.2.1           fastmap_1.2.0      promises_1.5.0
#>  [7] digest_0.6.39      rpart_4.1.27       mime_0.13
#> [10] lifecycle_1.0.5    ellipsis_0.3.2     survival_3.8-6
#> [13] processx_3.8.7     magrittr_2.0.4     compiler_4.5.0
#> [16] sass_0.4.10        rlang_1.1.7        tools_4.5.0
#> [19] yaml_2.3.12        knitr_1.51         labeling_0.4.3
#> [22] pkgbuild_1.4.8     xml2_1.5.2         RColorBrewer_1.1-3
#> [25] websocket_1.4.4    pkgload_1.5.1      rsconnect_1.7.0
#> [28] withr_3.0.2        purrr_1.2.1        desc_1.4.3
#> [31] nnet_7.3-20        grid_4.5.0         fansi_1.0.7
#> [34] jomo_2.7-6         scales_1.4.0       iterators_1.0.14
#> [37] MASS_7.3-65        tinytex_0.59       cli_3.6.5
#> [40] rmarkdown_2.31     reformulas_0.4.4   generics_0.1.4
#> [43] otel_0.2.0         rstudioapi_0.18.0  sessioninfo_1.2.3
#> [46] minqa_1.2.8        cachem_1.1.0       stringr_1.6.0
#> [49] splines_4.5.0      vctrs_0.7.2        devtools_2.5.0
#> [52] boot_1.3-32        glmnet_4.1-10      Matrix_1.7-5
#> [55] jsonlite_2.0.0     callr_3.7.6        patchwork_1.3.2
#> [58] mitml_0.4-5        systemfonts_1.3.2  foreach_1.5.2
#> [61] servr_0.32         jquerylib_0.1.4    tidyr_1.3.2
#> [64] glue_1.8.0         nloptr_2.2.1       pkgdown_2.2.0
#> [67] pan_1.9            codetools_0.2-20   stringi_1.8.7
#> [70] shape_1.4.6.1      gtable_0.3.6       later_1.4.8
#> [73] downlit_0.4.5      lme4_2.0-1         tibble_3.3.1
#> [76] pillar_1.11.1      htmltools_0.5.9    R6_2.6.1
```

```
#> [79] textshaping_1.0.5  Rdpack_2.6.6       evaluate_1.0.5
#> [82] lattice_0.22-9     rbibutils_2.4.1    backports_1.5.0
#> [85] memoise_2.0.1      broom_1.0.12       httpuv_1.6.17
#> [88] bslib_0.10.0       Rcpp_1.1.1         svglite_2.2.2
#> [91] nlme_3.1-169       mgcv_1.9-4         whisker_0.4.1
#> [94] xfun_0.57          fs_2.0.1           usethis_3.2.1
#> [97] pkgconfig_2.0.3
```

Acknowledgements

The `ggmice` package is developed with guidance and feedback from Gerko Vink, Stef van Buuren, and others. This project has received funding from the European Union's Horizon 2020 research and innovation programme under ReCoDID grant agreement No 825746.

References

Buuren, Stef van, and Karin Groothuis-Oudshoorn. 2011. "mice: Multivariate Imputation by Chained Equations in R." *Journal of Statistical Software* 45 (3): 1–67. https://doi.org/10.18637/jss.v045.i03 (https://doi.org/10.18637/jss.v045.i03).

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. Manual. R Foundation for Statistical Computing. https://www.R-project.org/ (https://www.R-project.org/).

Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org (https://ggplot2.tidyverse.org).

5 Discussion

In this disseration, I have investigated how to evaluate imputation methodology. Instead of seeing imputation as a quick fix for treating missing data, I have approached it as a sociotachnical sysmtem: a methodological practice that requires continuous assessment and reflection. A central theme that runs through all chapters is that imputation methodology should be evaluated, whether computyationally or graphically. Evaluation is key both for methdolologists who develop imputation methodology, as well as for imputers who apply imputation in practice.

5.1 Main findings

The central theme of this dissertation is that imputation methodology requires verification and validation. Verification happens in simulation studies by means of computational evaluation of imputation methods. Validation happens in practice, by means of computational evaluation and graphical evaluation of the imputation workflows.

In chapter 2, we learnded that testing new imputation methods through simulation studies requires careful, standardized design. If this is not the case, simulation studies might not cover all relevant conditions, and results are not comparable between studies. To make imputation methods comparable across studies, we must standardize simulation designs so that different methods face similar, clearly documented scenarios, report design choices transparently, and publish research compendiums so that others can reproduce and extend the experiments. Even with such care, there is a limit to what simulation studies can tell us. We can know that a method worked under the scenarios we simulated, but we cannot be certain that it will work in practice. That is why we need validation.

In chapter 3 we studied the algorithmic convergence of the `mice` algorithm. Diagnosing non-convergence quantitatively is a form of computational evaluation of the imputation workflow. Since `mice` converges in distribution, and not to a point, non-convergence is hard to diagnose. We learned that commonly used metrics and thresholds for non-convergence in iterative algorithms such as `mice`, may not be directly applicable to FCS. Simulation results show that MICE reaches stable output before non-convergence metrics would indicate so. There is a mismatch between theoretical convergence criteria and practical algorithmic behavior. That is why we still have to rely on the current standard advised evaluation tool for algorithmic convergence: visual inspection. Inspecting traceplots for signs of non-convergence is a form of visual diagnostic evaluation of imputation workflows, or graphical evaluation. Graphical evaluation was the topic of the fourth chapter.

Visual inspection aids imputers in developing and evaluating imputation models, because formal tests are not available. The fifth chapter introduces the R package `ggmice`. `ggmice` offers tools for the graphical evaluation of the imputations models that were previously unavailable. `ggmice` functions aid imputers in visualizing incomplete data and imputed data side-by-side,

displaying missingness and imputation models in an interpretable way, and by producing plots that can be extended and customized for publication. These visual diagnostics support existing evaluations whenever quantitative metrics and substantive reasoning are unfeasible.

5.2 Recommendations

For colleagues who design and study imputation methods, this dissertation suggests the following:

- Design simulation studies in a structured and reproducible manner. Make sure that the simulation conditions and performance measures cover all relevant scenarios and are comparable across methods. Report all design choices, and publish code (and data if applicable) alongside methodological papers.
- Engage with applied researchers. Talk to your intended audience (i.e., applied researchers), or at least find out what they are struggling with via discussion forums or surveys. This should inform the design of methods and diagnostics.

For researchers who implement imputation in their own studies, the main advice can be summarized as:

- Think before you code. Do not treat `mice` or any other package as a black box that magically treats your missing data problem. Consider the missingness mechanisms and your analysis model(s) before specifying imputation methods and predictor sets. Your substantive knowledge is valueable for treating the missignness.
- Look at the data, plot the data. Inspect incomplete variables, missingness patterns, and imputed values using graphics. Use visualization to inform model building, diagnose misfit, and communicate uncertainty.
- Treat diagnostics as part of the analysis, not an afterthought. Convergence plots, distributional comparisons, and sensitivity analyses should be integrated into the workflow and reported.
- Please cite the software you use. For R packages, it's as easy as running `citation("ggmice")`.

5.3 Limitations

Computational and graphical evaluation are not a substitute for thinking. Simulation studies, diagnostic metrics and plots can offer detailed insight into imputation workflows, but they may also create a false sense of certainty. For example, imputed values that lie within the range of the observed data may give a false sense of certainty against a MNAR mechanism, which is an untestable assumption. No amount of computation or visualization can prove that the missing-data mechanism is correctly assumed. Instead, one should use computational evaluation as complement to (not a replacement for) substantive knowledge and expert insight.

This dissertation investiagtes imputation workflows at the methodological level, without systematic user feedback. I have not tested the evaulation tools I propose experimentally. Assessing their causal impact on user behavior and imputation quality in practice was not feasible. I did receive feedback via teaching, workshops, collaborations, and online forums GitHub and StackOverflow. This informal evaluation is an indication of user appreciation(?). And I have used this input for example to further develop `ggmice`. So even though it might be impossibel to draw evidence-based conclusions about the efficacy of the tools I developed, we may still call the development evidence-informed, maybe?

Future direction: automated sensitivity analyses and over-imputation. Sensitivity analyses that vary model assumptions, include alternative predictors, or impose plausible MNAR structures can reveal how fragile conclusions are to changes in imputation choices. Over-imputation can reveal imputation model misfit on observed part of the data. Since publishing, there has not been a pre-reg format

Since this dissertation relies on the R language and centers the `mice` package, it is not certain how results translate to the wider imputation ecosystem. Concepts may generalize beyond `mice` in R. The focus reflects the current dominance of R and `mice` in many applied fields, and allows for depth and specificity in tool development. While this set-up is popular, the data science landscape is expanding, with many machine learning and deep learning methods emerging as candidate imputation engines. These methods are not (yet) widely implemented within `mice`. They might be able to better approximate the posterior predictive distribution of the missing values than `mice` methods, but this requires dedicated research and evaluation, which is outside of the scope of this dissertation.

Some other directions for future research include computational evaluation of variable importance within imputation models, integration of `ggmice` with `naniar`, and better support for non-numeric variables. The visualizations implemented in `ggmice` are generally better suited for numeric data, as opposed to categorical variables (or even open text field data, which is not supported by `mice` currently). Future research may focus on associations of categorical variables with missingness indicators, and convergence parameter other than SD of imputed values. Some other directions for future development of `ggmice` include:

- Visualizing `mira` objects (analysis results across multiple imputations);
- Adding posterior predictive checks and over-imputation plots (Cai et al., 2023);
- Quantifying uncertainty at the pattern or cell level;
- Tools for sensitivity analyses (e.g. inducing MNAR structure in the observed data and inspecting effects on imputations and inferences);
- Investigating which data-level diagnostics are actually informative about imputation model adequacy in practice (i.e., when inferential validity cannot be evaluated).

These extensions further align the software with the philosophy of making imputation workflows and their consequences visible.

5.4 Outlook

Missingness will be a persistent problem in research data. There is no foreseeable future in which all empirical datasets are complete. I therefore expect imputation methodology to conBut the way we deal with data might change. But the way we deal with data might change.

With the rise of machine learning and deep learning methods (or "AI"), evaluation will become ever more important. These new methods will also require new verification and validation tools, especially if the framework for intended applications changes. Many novel methods are designed under a prediction framework, not a statistical inference framework. Instead of inferring from a sample to the population, the aim becomes to infer about an individual case in the fututre. Methods intended for prediction may excel at point prediction, but ignore uncertainty in estimates. Omitting sampling variance from the estimates and to failing to incorporate between-imputation variance underrepresents the uncertainty. When such methods are used naively for inference, they produce confidence intervals that are too narrow and p-values that are too small. The result is an increased risk of spurious findings. Therefore, all models must propagate uncertainty.

Any model used in scientific inference should propagate uncertainty. This includes (but is not limited to) integrating missing-data uncertainty into model outputs and reporting interval estimates that align with the uncertainty in the data and the modeling process. This also holds for graphical presentations.

Trust in science requires an honest reflection of uncertainty in our analyses. So please be open about uncertainty and limitations of scientific studies. And while we're at it, also please publish null results (e.g., via registered reports or pre-registrations), share data and code with other scientists and the public, and publish open access (including educational materials). At the same time, we should change the way we evaluate science. This dissertation is an example of the new 'recognition and rewards' vision. It includes software as an actual chapter, not something extra. This is part of the current movement of valuing diverse forms of contribution beyond traditional papers. I hope this can serve as an example for other PhD candidates and researchers to reflect on what they consider scientific output. I hope and to claim space for software, open data, and methodological infrastructure as central achievements. I hope to inspire others to claim space for 'alternative' output such as software and data as full-fledged research output.

Finally, it is important to note that imputation is not always the right response to missingness. Missing data can signal a mismatch between our data collection methods and the lived experiences of participants. Systematic patterns of non-response may point to issues in the survey design, accessibility, or trust. In such cases, treating missingness solely as a statistical nuisance to be 'fixed' risks overlooking important social and ethical dimensions of research, especially in the social and biomedical sciences, where rows in the data represent human beings. I would love to study missingness as a method to tell you about mismatches between data collection and participant perspectives.

In short, I hope this dissertation encourages to question defaults, to make evaluation practices explicit, and to contribute to a scientific culture that takes uncertainty—and our responsibilities in representing it—seriously.

References

[TODO: merge refs from chapters]

Abayomi, K., Gelman, A., & Levy, M. (2008). Diagnostics for Multivariate Imputations. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, *57*(3), 273–291. https://www.jstor.org/stable/20492604

Allison, P. D. (2002). *Missing Data*. SAGE Publications, Inc. https://doi.org/10.4135/9781412985079

Alsalti, T., Rohrer, J. M., & Arslan, R. C. (2026). Thinking Clearly About Sampling and Representation With the Total Survey Error Framework. *Social and Personality Psychology Compass*, *20*(7), e70160. https://doi.org/10.1111/spc3.70160

Bartlett, J. W., Seaman, S. R., White, I. R., & Carpenter, J. R. (2015). Multiple imputation of covariates by fully conditional specification: Accommodating the substantive model. *Statistical Methods in Medical Research*, *24*(4), 462–487. https://doi.org/10.1177/0962280214521348

Bondarenko, I., & Raghunathan, T. (2016). Graphical and numerical diagnostic tools to assess suitability of multiple imputations and imputation models. *Statistics in Medicine*, *35*(17), 3007–3020. https://doi.org/10.1002/sim.6926

Cai, M., van Buuren, S., & Vink, G. (2023). Graphical and numerical diagnostic tools to assess multiple imputation models by posterior predictive checking. *Heliyon*, *9*(6), e17077. https://doi.org/10.1016/j.heliyon.2023.e17077

Carpenter, J. R., & Smuk, M. (2021). Missing data: A statistical framework for practice. *Biometrical Journal*, *63*(5), 915–947. https://doi.org/10.1002/bimj.202000196

Curzon, P., Dorling, M., Ng, T., Selby, C., & Woollard, J. (2014). *Developing computational thinking in the classroom: A framework*. Computing at School.

Daniel, R. M., Kenward, M. G., Cousens, S. N., & De Stavola, B. L. (2012). Using causal diagrams to guide analysis in missing data problems. *Statistical Methods in Medical Research*, *21*(3), 243–256. https://doi.org/10.1177/0962280210394469

de Ruiter, J. P., & Albert, S. (2017). An Appeal for a Methodological Fusion of Conversation Analysis and Experimental Psychology. *Research on Language and Social Interaction*, *50*(1), 90–107. https://doi.org/10.1080/08351813.2017.1262050

Deming, W. E. (1944). On Errors in Surveys. *American Sociological Review*, *9*(4), 359–369. https://doi.org/10.2307/2085979

Ding, P., & Li, F. (2018). Causal Inference: A Missing Data Perspective. *Statistical Science*, *33*(2), 214–237. https://doi.org/10.1214/18-STS645

Dudek, G. (2004). Computational Evaluation. In G. Dudek (Ed.), *Collaborative Planning in Supply Chains: A Negotiation-Based Approach* (pp. 165–213). Springer. https://doi.org/10.1007/978-3-662-05443-7_7

Evaluation measures (information retrieval). (2025). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Evaluation_measures_(information_retrieval)&oldid=1329650960

Federal Committee on Statistical Methodology. (2001). *Statistical Policy Working Paper 31. Measuring and Reporting Sources of Error in Surveys*. https://doi.org/10.21949/1529874

Gelman, A., & Shalizi, C. R. (2013). Philosophy and the practice of Bayesian statistics. *British Journal of Mathematical and Statistical Psychology*, *66*(1), 8–38. https://doi.org/10.1111/j.2044-8317.2011.02037.x

Graham, J. W. (2012). *Missing Data*. Springer. https://doi.org/10.1007/978-1-4614-4018-5

Groves, R. M. (2002). *Survey nonresponse*. Wiley.

Hand, D. (2020). *Dark data: Why what you don't know matters*. Princeton University Press.

Hand, D. (2025). Statistics: An Overview. In *International Encyclopedia of Statistical Science* (pp. 2654–2659). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-662-69359-9_649

He, Y., Zaslavsky, A. M., Harrington, D. P., Catalano, P., & Landrum, M. B. (2010). Multiple Imputation in a Large-Scale Complex Survey: A Practical Guide. *Statistical Methods in Medical Research*, *19*(6), 653–670. https://doi.org/10.1177/0962280208101273

IEEE Standard for System, Software, and Hardware Verification and Validation (2025). https://doi.org/10.1109/IEEESTD.2025.11134780

ISO, & IEC. (2015). *ISO/IEC 2382:2015 — information technology — vocabulary*. ISO; IEC; International Standard;

Little, R. J. A., & Rubin, D. B. (2019). *Statistical analysis with missing data. Third edition*. John Wiley & Sons, Ltd. https://doi.org/10.1002/9781119482260

Little, & Rubin. (2020). *Statistical Analysis with Missing Data, Third Edition | Wiley Series in Probability and Statistics*. https://onlinelibrary.wiley.com/doi/book/10.1002/9781119482260

Liu, D., Oberman, H. I., Muñoz, J., Hoogland, J., & Debray, T. P. A. (2023). Quality Control, Data Cleaning, Imputation. In F. W. Asselbergs, S. Denaxas, D. L. Oberski, & J. H. Moore (Eds.), *Clinical Applications of Artificial Intelligence in Real-World Data* (pp. 7–36). Springer International Publishing. https://doi.org/10.1007/978-3-031-36678-9_2

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. https://doi.org/10.1017/CBO9780511809071

Meng, X.-L. (1994). Multiple-Imputation Inferences with Uncongenial Sources of Input. *Statistical Science*, *9*(4), 538–558. https://doi.org/10.1214/ss/1177010269

Molenberghs, G., Fitzmaurice, G., Kenward, M. G., Tsiatis, A., & Verbeke, G. (Eds.). (2014). *Handbook of Missing Data Methodology*. Chapman and Hall/CRC. https://doi.org/10.1201/b17622

Moreno-Betancur, M., Lee, K. J., Leacy, F. P., White, I. R., Simpson, J. A., & Carlin, J. B. (2018). Canonical Causal Diagrams to Guide the Treatment of Missing Data in Epidemiologic Studies. *American Journal of Epidemiology*, *187*(12), 2705–2715. https://doi.org/10.1093/aje/kwy173

Neyman, J. (1934). On the Two Different Aspects of the Representative Method: The Method of Stratified Sampling and the Method of Purposive Selection. *Journal of the Royal Statistical Society*, *97*(4), 558–625. https://doi.org/10.2307/2342192

Nguyen, C. D., Carlin, J. B., & Lee, K. J. (2017). Model checking in multiple imputation: An overview and case study. *Emerging Themes in Epidemiology*, *14*(1), 8. https://doi.org/10.1186/s12982-017-0062-6

Nguyen, C. D., Carlin, J. B., & Lee, K. J. (2021). Practical strategies for handling breakdown

of multiple imputation procedures. *Emerging Themes in Epidemiology*, *18*, 5. https://doi.org/10.1186/s12982-021-00095-3

Oberman, H. I. (2022). *ggmice: Visualizations for 'mice' with 'Ggplot2'* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.6532702

Oberman, H. I., University, U., Utrecht, U. M. C., Volker, T., Vink, G., Vink, P., Wallis, J., & Lang, K. (2025). *Ggmice: Visualizations for 'mice' with 'Ggplot2'* (Version 0.1.1) [Computer software]. https://cran.r-project.org/web/packages/ggmice/index.html

Oberman, H. I., van Buuren, S., & Vink, G. (2021, October 22). *Missing the Point: Non-Convergence in Iterative Imputation Algorithms*. https://arxiv.org/abs/2110.11951

Oberman, H. I., & Vink, G. (2024). Toward a standardized evaluation of imputation methodology. *Biometrical Journal*, *66*(1), 2200107. https://doi.org/10.1002/bimj.202200107

Raghunathan, T., Lepkowski, J., Hoewyk, J., & Solenberger, P. (2001). A Multivariate Technique for Multiply Imputing Missing Values Using a Sequence of Regression Models. *Survey Methodology*, *27*.

Rosenberg, A. (2018). *Philosophy of Social Science*. Routledge. https://books.google.com?id=GRdWDwAAQBAJ

Rubin, D. B. (1976). Inference and Missing Data. *Biometrika*, *63*(3), 581–592. https://doi.org/10.2307/2335739

Rubin, D. B. (1987). *Multiple Imputation for nonresponse in surveys*. Wiley.

Rubin, D. B. (1996). Multiple Imputation after 18+ Years. *Journal of the American Statistical Association*, *91*(434), 473–489. https://doi.org/10.1080/01621459.1996.10476908

Sarkar, D. (2008). *Lattice: Multivariate data visualization with R*. Springer. http://lmdvr.r-forge.r-project.org

Schafer, J. L. (1997). *Analysis of Incomplete Multivariate Data*. Chapman and Hall/CRC. https://doi.org/10.1201/9781439821862

Schafer, J. L., & Graham, J. W. (2002). Missing data: Our view of the state of the art. *Psychological Methods*, *7*(2), 147–177. https://www.ncbi.nlm.nih.gov/pubmed/12090408

Sommerville, I. (2015). *Software engineering* (Tenth edition). Pearson.

Systems and Software Engineering — Vocabulary (2017).

Tierney, N., & Cook, D. (2023). Expanding tidy data principles to facilitate missing data exploration, visualization and assessment of imputations. *Journal of Statistical Software*, *105*(7), 1–31. https://doi.org/10.18637/jss.v105.i07

van Buuren, S. (2007). Multiple imputation of discrete and continuous data by fully conditional specification. *Statistical Methods in Medical Research*, *16*(3), 219–242. https://doi.org/10.1177/0962280206074463

van Buuren, S. (2018). *Flexible imputation of missing data* (2nd ed.). CRC Press.

van Buuren, S., Brand, J. P. L., Groothuis-Oudshoorn, C. G. M., & Rubin, D. B. (2006). Fully conditional specification in multivariate imputation. *Journal of Statistical Computation and Simulation*, *76*(12), 1049–1064. https://doi.org/10.1080/10629360600810434

van Buuren, S., & Groothuis-Oudshoorn, C. G. M. (1999). *Flexible multivariate imputation by MICE* (Technical Report No. PG/VGZ/99.054). TNO Prevention and Health.

van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software*, *45*(3), 1–67. https://doi.org/10.18637/jss.

v045.i03

Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wilkinson, L. (2012). The Grammar of Graphics. In *Handbook of Computational Statistics* (pp. 375–414). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-21551-3_13

CV

[TODO: summarize, include]